

Multi-Agent Development with Claude Code

Subagents, Team Orchestration, and Long-Running Systems



**Orchestrate subagents.
Ship production agents.**

Multi-Agent Development with Claude Code

1. [Introduction](#)

1. [Who This Book Is For](#)
2. [How This Book Is Organized](#)
3. [Companion Repository](#)
4. [Conventions](#)
5. [A Note on Versioning](#)

2. [The Case for Multi-Agent Systems](#)

1. [The Limits of a Single Agent](#)
2. [How Multi-Agent Systems Address These Limits](#)
3. [Evidence from Anthropic's Own Systems](#)
4. [When Not to Use Multi-Agent Systems](#)
5. [What's Ahead](#)
6. [Summary](#)

3. [Five Workflow Patterns](#)

1. [The Complexity Staircase](#)
2. [The Building Block: The Augmented LLM](#)
3. [Pattern 1: Prompt Chaining](#)
4. [Pattern 2: Routing](#)
5. [Pattern 3: Parallelization](#)
6. [Pattern 4: Orchestrator-Workers](#)
7. [Pattern 5: Evaluator-Optimizer](#)
8. [Beyond Workflows: Autonomous Agents](#)
9. [Choosing a Pattern](#)
10. [Combining Patterns](#)
11. [Summary](#)

4. [How Subagents Work](#)

1. [What Subagents Are](#)
 2. [The Agent Tool](#)
 3. [Built-in Subagents](#)
 4. [Subagents in Practice](#)
 5. [Subagent Constraints](#)
 6. [Subagents vs. Agent Teams](#)
 7. [Common Failure Patterns](#)
 8. [Summary](#)
5. [Building Custom Subagents](#)
 1. [The Subagent Definition Format](#)
 2. [Scope and Priority](#)
 3. [Tool Restrictions](#)
 4. [Model Selection](#)
 5. [Permission Modes](#)
 6. [Persistent Memory](#)
 7. [MCP Server Integration](#)
 8. [Hooks for Fine-Grained Control](#)
 9. [Practical Example: A Multi-Subagent Project](#)
 10. [Common Failure Patterns](#)
 11. [Going Further](#)
 12. [Summary](#)
6. [Connecting Subagents to External Tools via MCP](#)
 1. [MCP Servers and Subagents](#)
 2. [Two Connection Methods](#)
 3. [Tool Inheritance and Scope Control](#)
 4. [Practical Example: Web Scraping Agent](#)
 5. [Practical Example: Code Review Agent](#)
 6. [Practical Example: Read-Only Database Agent](#)
 7. [MCP Scope Design Patterns](#)

8. [Best Practices](#)
9. [Common Failure Patterns](#)
10. [Summary](#)
7. [Orchestrator Design](#)
 1. [What an Orchestrator Does](#)
 2. [Task Decomposition](#)
 3. [Context Passing](#)
 4. [Implementation Patterns](#)
 5. [Lessons from Anthropic's Research System](#)
 6. [Anti-Patterns](#)
 7. [Common Failure Patterns](#)
 8. [Summary](#)
8. [Designing Tools for Agents](#)
 1. [What Is ACI?](#)
 2. [Six Principles of Tool Design](#)
 3. [Error Response Design](#)
 4. [Learning from Claude Code's Tool Design](#)
 5. [Building an MCP Server That Embodies ACI](#)
 6. [Tool Design Anti-Patterns](#)
 7. [Tool Evaluation](#)
 8. [MCP Server Development Checklist](#)
 9. [Summary](#)
9. [Writing Effective Agent Prompts](#)
 1. [From Prompt Engineering to Context Engineering](#)
 2. [Principle 1: Think Like the Agent](#)
 3. [Principle 2: Teach Orchestrators How to Delegate](#)
 4. [Principle 3: Match Effort to Query Complexity](#)
 5. [Principle 4: Prioritize Tool Design and Selection](#)
 6. [Principle 5: Let Agents Improve Themselves](#)

7. [Principle 6: Start Broad, Then Narrow](#)
 8. [Principle 7: Guide the Thinking Process](#)
 9. [Principle 8: Transform Speed with Parallel Tool Calls](#)
 10. [The Right Altitude](#)
 11. [Encoding Principles in CLAUDE.md](#)
 12. [Summary](#)
10. [Managing Long-Running Agent Sessions](#)
 1. [The Long-Running Agent Problem](#)
 2. [Harness Architecture](#)
 3. [Building a Harness with Five Scripts](#)
 4. [Long-Running Patterns in Claude Code](#)
 5. [Memory Management and Context Compression](#)
 6. [Lessons from the C Compiler Project](#)
 7. [Checkpoints and Recovery](#)
 8. [Long-Running Best Practices](#)
 9. [Summary](#)
11. [Error Recovery and Failure Modes](#)
 1. [Why Agent Failures Are Different](#)
 2. [Four Failure Mode Categories](#)
 3. [Retry Strategy Design](#)
 4. [Designing Human-in-the-Loop Timing](#)
 5. [Graceful Degradation](#)
 6. [Learning from Failures](#)
 7. [Summary](#)
12. [Getting Started with Agent Teams](#)
 1. [Where Subagents Fall Short](#)
 2. [What Agent Teams Are](#)
 3. [Enabling Agent Teams](#)
 4. [Starting a Team](#)

5. [Context and Communication](#)
6. [Reusing Subagent Definitions](#)
7. [Lessons from the C Compiler Project](#)
8. [Community Resources](#)
9. [Limitations](#)
10. [Summary](#)
13. [Agent Teams in Practice](#)
 1. [Four Team Design Patterns](#)
 2. [Display Modes](#)
 3. [Specifying Teammates and Models](#)
 4. [Plan Approval Flow](#)
 5. [Task Management](#)
 6. [Team Size](#)
 7. [Quality Gates with Hooks](#)
 8. [Avoiding File Conflicts](#)
 9. [Direct Interaction with Teammates](#)
 10. [Shutting Down](#)
 11. [Troubleshooting](#)
 12. [Summary](#)
14. [Evaluating Agent Performance](#)
 1. [Why Evaluation Matters](#)
 2. [Evaluation Structure](#)
 3. [Three Types of Graders](#)
 4. [Capability vs. Regression Evals](#)
 5. [Handling Non-Determinism: \$\text{pass}@k\$ and \$\text{pass}^k\$](#)
 6. [A Practical Evaluation Roadmap](#)
 7. [Implementing Evaluations in Claude Code](#)
 8. [Summary](#)
15. [Production Reliability](#)

1. [Why Production Is Different](#)
 2. [Stateful Error Compounding](#)
 3. [Monitoring and Observability](#)
 4. [Cost Management](#)
 5. [Security and Sandboxing](#)
 6. [Deployment Strategy](#)
 7. [Incident Response](#)
 8. [Production Checklist](#)
 9. [Summary](#)
16. [Afterword](#)
 1. [What I Wanted This Book to Convey](#)
 2. [What Anthropic's Projects Demonstrate](#)
 3. [Next Steps](#)
 4. [The Road Ahead](#)
17. [Appendix A: Quick Reference](#)
 1. [A.1 CLI Commands and Flags](#)
 2. [A.2 Subagent Definition File YAML Schema](#)
 3. [A.3 MCP Server Configuration Reference](#)
 4. [A.4 Agent Teams Configuration](#)
 5. [A.5 Long-Running Agent Files](#)
 6. [A.6 Environment Variables](#)
 7. [A.7 Official Documentation and Blog References](#)
 8. [A.8 Agent Tool Parameters Reference](#)
18. [Appendix B: CLI Native vs Agent SDK](#)
 1. [B.1 Core Comparison](#)
 2. [B.2 When to Use CLI Native](#)
 3. [B.3 When to Use the Agent SDK](#)
 4. [B.4 Invocation Methods Map](#)
 5. [B.5 Decision Guide](#)

6. B.6 Key Takeaway.

OceanofPDF.com

Introduction

Claude Code accelerates software development even in a single session. But as projects grow, one agent hits structural limits: context window boundaries, sequential bottlenecks, and the inability to specialize. Multi-agent systems are the answer.

This book is a practical guide to Claude Code's multi-agent capabilities. It covers three pillars – subagents, agent teams, and long-running harnesses – grounded in data from Anthropic's own engineering projects.

Who This Book Is For

- **Engineers using Claude Code** who know the basics but haven't explored multi-agent patterns
- **Developers interested in AI agents** who want to build agent systems natively from the CLI, without reaching for Python frameworks like LangGraph or AutoGen
- **Technical leads** who want to version-control subagent definitions and scale their team's productivity with Claude Code

You need enough programming experience to read Python. No prior knowledge of agent design is required.

How This Book Is Organized

The book has six parts and fourteen chapters.

Part I: Foundations (Chapters 1-2) establishes why multi-agent systems matter and introduces five workflow patterns. Anthropic’s research system and C compiler project provide real-world evidence.

Part II: Subagents (Chapters 3-5) covers Claude Code’s `Agent` tool for spawning subagents. You’ll progress from built-in subagent types through custom definitions (`.claude/agents/`) to MCP server integration.

Part III: Design and Orchestration (Chapters 6-8) tackles the principles that make multi-agent systems work well: orchestrator design, tool design following ACI (Agent-Computer Interface) principles, and eight prompt engineering principles for agents.

Part IV: Long-Running Agents (Chapters 9-10) addresses agents that span multiple sessions. Topics include harness architecture, checkpoint design, progress file management, and failure mode classification with recovery strategies.

Part V: Agent Teams (Chapters 11-12) introduces agent teams – multiple Claude Code instances working in parallel on a shared codebase with direct inter-agent communication.

Part VI: From Evaluation to Production (Chapters 13-14) covers quality evaluation and production reliability for multi-agent systems.

Two appendices round out the book. **Appendix A** is a quick reference for commands, configuration, and official documentation pointers. **Appendix B** compares the CLI-native approach with the Claude Agent SDK, for readers who want to embed the patterns from this book into CI scripts or programmatic workflows.

Companion Repository

All sample code is available on GitHub:

```
git clone https://github.com/driftnode986/claude-multiagent-guide-en-samples.git
cd claude-multiagent-guide-en-samples
```

Each chapter's samples are in a directory matching the chapter number. I recommend cloning the repository and running the examples as you read.

Conventions

- **Code blocks** show executable commands and code examples.
- **WHY blocks** (indented quotes) explain the design rationale behind a decision. You can skip them without losing the main thread, but they answer the question you'll inevitably ask: "Why not just do it the simpler way?"
- **Diagrams** visualize architecture and concepts for each chapter.

Command examples assume macOS or Linux. On Windows, use Git Bash or WSL.

A Note on Versioning

Claude Code is under active development. This book reflects the state of the tool as of April 2026. Agent teams in particular are an experimental feature and may change in future versions. For the latest specifications, consult the Claude Code documentation at <https://code.claude.com/docs/>.

The book marks volatile details with a standardized callout: "**At time of writing** (Claude Code vX.Y)." grep for this phrase to find sections that may need updating.

[OceanofPDF.com](https://oceanofpdf.com)

The Case for Multi-Agent Systems

Claude Code handles most development tasks just fine in a single session. This chapter explains why some tasks break that model – and when splitting work across multiple agents is the right architectural response.

The Limits of a Single Agent

A single Claude Code session excels at focused work: editing files, running tests, debugging a function, exploring a codebase. You interact with one agent that reads context, makes decisions, and acts. For the vast majority of daily development tasks, this is all you need.

Problems appear when the scope of a task outgrows what one session can handle. Three structural limits surface repeatedly.

The Context Window Ceiling

Every LLM operates within a finite context window – the total amount of information it can hold in one conversation. As you load files, read test output, and accumulate tool results, the window fills up. Once it's full, earlier information gets compressed or dropped. The agent starts “forgetting” what it read at the beginning of the session.

Consider a refactoring task that touches 50 files across a monorepo. The agent reads the first file, analyzes its dependencies, plans a change. By the time it reaches the twentieth file, the details of the first file have faded from context. This isn't a bug – it's a hard constraint of the architecture.

Why this is structural, not incidental According to Anthropic Engineering Blog (2025-06), token usage alone explains 80% of the performance variance in their BrowseComp evaluation. Techniques like summarization and selective loading help, but they're workarounds. They delay the problem rather than solve it.

Sequential Execution

A single agent does one thing at a time. If you need a security review, a performance audit, and a style check on the same codebase, the agent runs them

sequentially. Each additional concern adds linearly to the total time.

This matters less for a quick bug fix. It matters a lot when you're doing comprehensive code review across a large pull request, or running multiple independent research queries. The tasks don't depend on each other – they could run simultaneously – but a single session can't parallelize them.

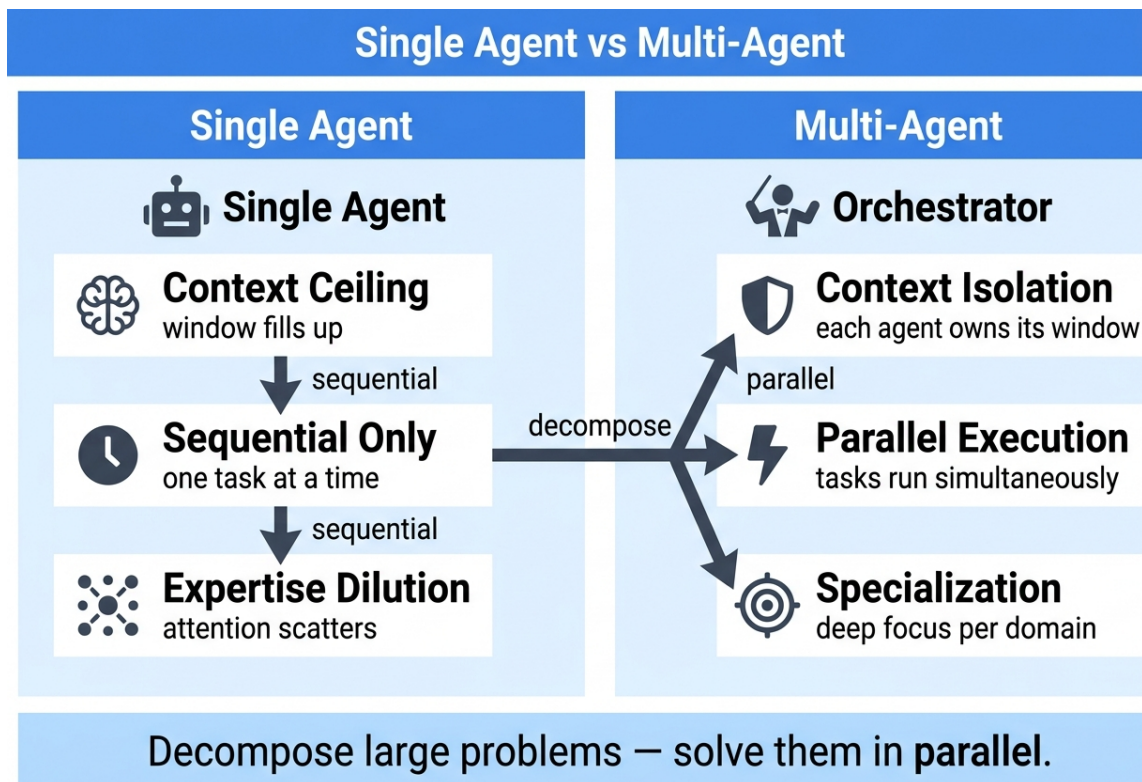
Expertise Dilution

A single prompt that says “handle everything” produces shallow results across every dimension. When one agent is responsible for frontend design, backend API structure, database schema, and CI/CD configuration, attention scatters. The agent spends tokens switching between domains instead of going deep in any one of them.

According to Anthropic Engineering Blog (2025-06), this is a documented failure mode. Single agents tasked with broad responsibilities tend to exhaust their implementation budget midway through the context, or declare completion while significant work remains undone.

| How Multi-Agent Systems Address These Limits

Multi-agent architecture doesn't add magic. It applies a familiar engineering principle: decompose a large problem into smaller, independent pieces and solve them separately.



Single Agent vs Multi-Agent

Context Isolation

When you split work across multiple agents, each one gets its own context window. A lead agent (sometimes called the orchestrator) decomposes the task and delegates subtasks to subagents. Each subagent focuses on its piece of the problem, using its full context window for that narrow scope. It returns only the relevant results – not everything it processed along the way.

Anthropic's own research system uses this pattern. The lead agent analyzes a query, develops a strategy, and spawns subagents to explore different aspects simultaneously. Each subagent acts as what the team calls an "intelligent filter" – iteratively searching, gathering information, and compressing the most important findings before returning them to the lead agent.

Why context isolation works According to Anthropic Engineering Blog (2025-06), “the essence of search is compression.” Subagents don’t just expand memory – they serve as information compressors, distilling large volumes of raw data into the tokens that matter most. The lead agent synthesizes these compressed results without needing to process the raw sources itself.

Parallel Execution

Independent subtasks can run simultaneously when each one has its own agent. Anthropic’s research system introduced two levels of parallelization:

1. The lead agent spawns 3 to 5 subagents in parallel (not sequentially).
2. Each subagent uses 3 or more tools in parallel within its own session.

The result: complex queries that previously took hours completed in minutes – a reduction of up to 90% in research time.

This maps directly to software development scenarios. Reviewing security, performance, and style concerns in parallel. Running independent test suites simultaneously. Searching multiple documentation sources at once.

Specialization

Each agent can be given a focused role with a tailored prompt. Instead of one generalist agent juggling frontend and backend concerns, you create separate agents with specific expertise and instructions.

The C compiler project (covered in detail later in this chapter) used this approach with 16 parallel agents. Some focused on implementing new compiler features. Others specialized in consolidating duplicate code, optimizing performance, or maintaining documentation. Each agent brought its full context window to bear on a narrow domain – far more effective than one agent trying to do everything.

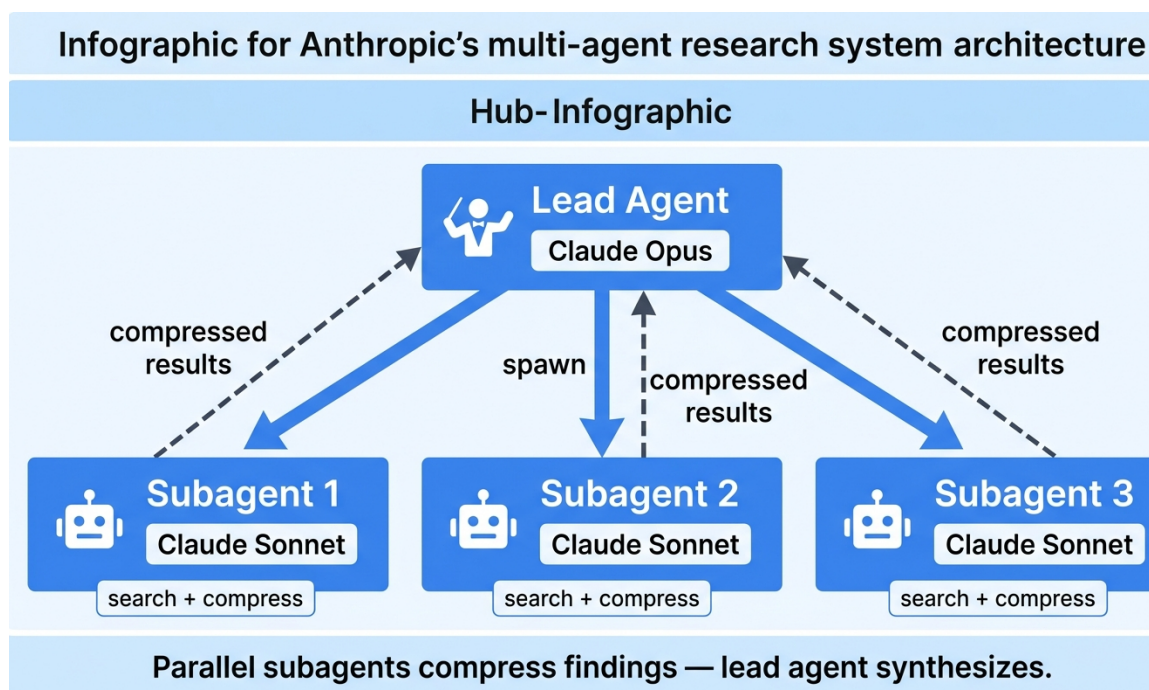
Why specialization beats generalism here This follows the same principle behind Unix pipes and microservices: do one thing well. A specialized agent produces higher-quality output because its prompt is focused, its context isn't diluted, and its success criteria are clear. The coordination cost is real, but for sufficiently complex tasks, it pays for itself.

Evidence from Anthropic's Own Systems

The multi-agent approach isn't theoretical. Anthropic has built and operated multi-agent systems at scale and published their results.

The Multi-Agent Research System

In June 2025, Anthropic published the architecture behind their research capabilities. The system uses an orchestrator-worker pattern: a lead agent running Claude Opus coordinates multiple subagents running Claude Sonnet.



Research System Architecture

How it works:

The lead agent receives a query, analyzes its complexity, and decides how many subagents to spawn. Simple fact-finding might need just one agent with a few

tool calls. Complex research might use more than ten subagents, each with a clearly divided responsibility. The subagents search, gather, and compress their findings. The lead agent synthesizes a final answer.

What the data shows:

- The multi-agent system outperformed a single Claude Opus agent by 90.2% on Anthropic’s internal research evaluation.
- Parallel tool calling cut research time by up to 90% for complex queries.
- Token usage explained 80% of performance variance in the BrowseComp evaluation. The number of tool calls and model choice explained most of the remaining 15%.

That last finding is particularly revealing. Performance scales with how many tokens you can effectively deploy – and multi-agent architecture is the mechanism for deploying them across independent context windows.

Why this pattern? The orchestrator-worker pattern works because research tasks are naturally decomposable. Investigating “all board members of S&P 500 IT companies” breaks into hundreds of independent lookups. A single agent doing them sequentially hits both time and context limits. Multiple agents in parallel finish faster and find more. Alternative: A single agent with a larger context window would still be sequential and would spend most of its tokens on raw search results rather than analysis.

The C Compiler Project

In February 2026, Anthropic researcher Nicholas Carlini published an experiment: 16 parallel Claude agents building a Rust-based C compiler from scratch.

Scale of the project:

- Nearly 2,000 Claude Code sessions over two weeks
- 2 billion input tokens, 140 million output tokens
- Total API cost: approximately \$20,000
- 100,000 lines of generated code

The resulting compiler can build Linux 6.9 on x86, ARM, and RISC-V. It compiles QEMU, FFmpeg, SQLite, PostgreSQL, and Redis. It passes 99% of

most compiler test suites, including the GCC torture test suite.

How the team operated:

The harness is surprisingly simple. Each agent runs in a Docker container with an infinite loop: pick a task, work on it, commit, pull other agents' changes, repeat. There's no central orchestrator. Coordination happens through Git – agents claim tasks by writing lock files to a shared directory, and Git's merge mechanics handle conflicts.

```
#!/bin/bash
# Each agent runs in its own Docker container
while true; do
    COMMIT=$(git rev-parse --short=6 HEAD)
    LOGFILE="agent_logs/agent_${COMMIT}.log"

    # Agent autonomously selects and executes tasks
    claude --dangerously-skip-permissions \
        -p "$(cat AGENT_PROMPT.md)" \
        --model claude-opus-X-Y &> "$LOGFILE"
done
```

When many independent tests are failing, parallelization is natural – each agent picks a different failing test. The challenge comes when agents converge on a single large problem (like compiling the Linux kernel), where every agent tries to fix the same bug. The team solved this with differential debugging: using GCC as an oracle to isolate which files contained bugs, so each agent could work on a different file.

Why no central orchestrator? The C compiler project deliberately avoided an orchestrator. With high-quality tests serving as ground truth, each agent can independently assess what needs fixing and verify its own work. The tests are the orchestrator. This works when the problem has clear, measurable success criteria (compiler test suites) but wouldn't work for tasks with ambiguous goals. Alternative: An orchestrator-worker pattern (like the research system) adds coordination overhead but handles ambiguous task decomposition better. Chapter 6 covers when to choose each approach.

When Not to Use Multi-Agent Systems

Multi-agent architecture adds real complexity and cost. According to Anthropic Engineering Blog (2024-12), the right approach is to “find the simplest solution possible, and only increase complexity when needed.”

Single-Agent Tasks

Some tasks genuinely don't benefit from decomposition:

- **Single-file changes:** A bug fix, refactor, or feature addition that stays within one file. The overhead of spawning and coordinating agents exceeds any benefit.
- **Sequential dependencies:** When step B requires the complete output of step A, parallelization doesn't help. You'd just be running agents that wait on each other.
- **Quick tasks:** Work that completes in minutes. The setup cost of multi-agent coordination isn't justified.
- **Shared-context tasks:** When every part of the work needs access to the same information, context isolation provides no advantage – you'd end up duplicating context across agents.

The Cost Equation

Multi-agent systems use significantly more tokens than single-agent sessions. According to Anthropic Engineering Blog (2025-06), token usage alone explains 80% of performance variance – more tokens deployed across parallel agents means better results, but cost scales accordingly. This investment is justified only when:

- The task is valuable enough to warrant the investment (days of manual research, codebase-wide changes).
- Significant parallelism exists (many independent subtasks).
- The information volume exceeds a single context window.

Why “don’t use it” matters as much as “use it” According to Anthropic Engineering Blog (2024-12), “for many applications, optimizing single LLM calls with retrieval and in-context examples is usually enough.” The impulse to build complex systems when simple ones suffice is a common engineering trap. Multi-agent architecture is a tool for specific problems, not a default mode of operation.

A Decision Framework

Use this flow to decide whether a task benefits from multi-agent execution:

```

Does the task fit in a single context window?
YES -> Use a single agent.
NO  -> Can it be split into independent subtasks?
      YES -> Multi-agent parallelization. (Ch2-5)
      NO  -> Does it need sustained execution over hours?
            YES -> Long-running agent with harness. (Ch9)
            NO  -> Single agent with better tool selection.
Does it span multiple specialist domains?
YES -> Specialized subagents with focused prompts. (Ch4)
NO  -> Single agent with domain-specific tools.

```

This isn’t a rigid algorithm – it’s a starting checklist. The chapters ahead cover each branch in detail.

What’s Ahead

This chapter established the structural case for multi-agent systems: context isolation, parallel execution, and specialization address limits that no amount of single-agent optimization can overcome. The two Anthropic case studies – the research system and the C compiler – demonstrate the approach at production scale.

Chapter 2 introduces five workflow patterns that Anthropic has identified as the building blocks for agentic systems: prompt chaining, routing, parallelization, orchestrator-workers, and evaluator-optimizer. The research system you saw here is an instance of the orchestrator-worker pattern. The C compiler project is an early example of agent teams, covered in depth in Chapters 11 and 12.

Note: The agent teams feature used in the C compiler project is experimental. Chapters 11 and 12 cover this feature in detail, including the setup requirements and current limitations.

Summary

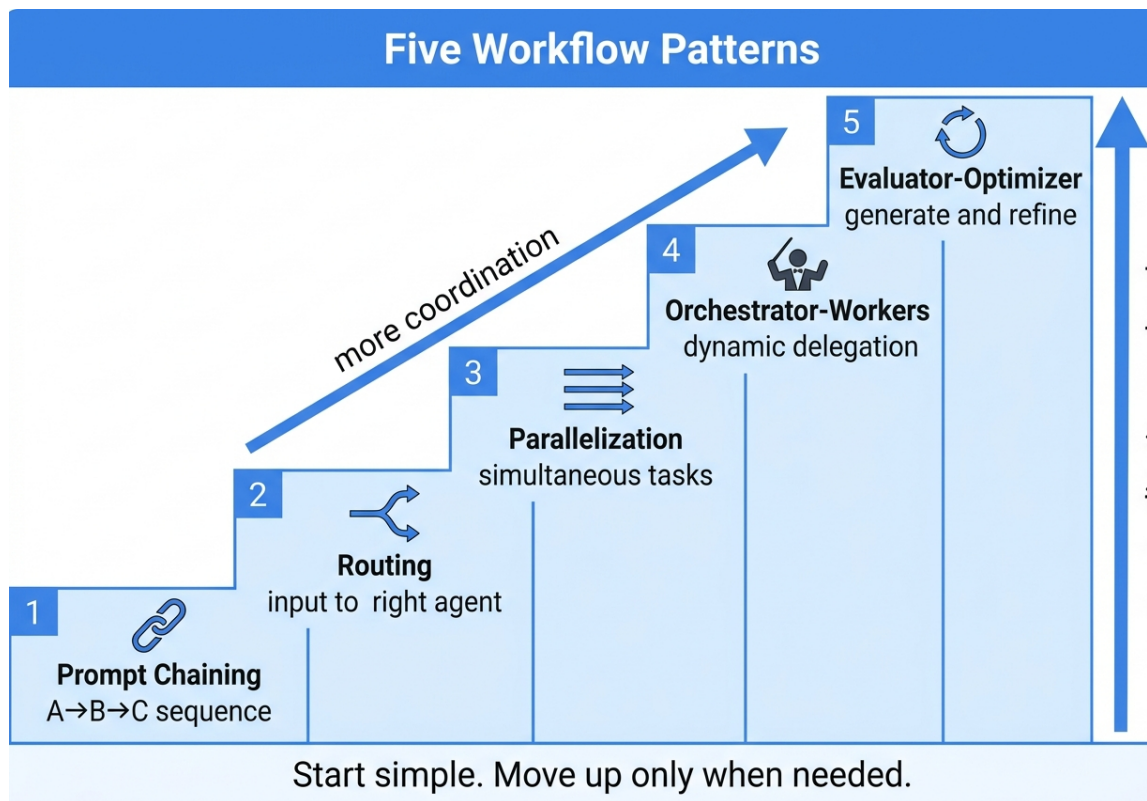
- Single agents hit three structural limits as task scope grows: context window ceiling, sequential execution, and expertise dilution.
- Multi-agent systems solve these through context isolation, parallel execution, and role specialization – not through better prompts or bigger models.
- Anthropic’s research system achieved a 90.2% performance improvement over a single agent by distributing work across an orchestrator and parallel subagents.
- The C compiler project demonstrated autonomous multi-agent development at scale: 16 agents, 2,000 sessions, 100,000 lines of code, two weeks.
- Multi-agent architecture is an investment, not a default. Use it when tasks exceed a single context window, offer meaningful parallelism, or require specialist decomposition. For everything else, a single agent is simpler, cheaper, and faster.

Five Workflow Patterns

Anthropic has identified five workflow patterns from working with dozens of teams building agent systems. This chapter explains each pattern, when it fits, and how it maps to Claude Code – giving you a vocabulary for the architectural decisions in the rest of the book.

The Complexity Staircase

The five patterns form a progression from simple to complex. Each step adds flexibility at the cost of coordination overhead.



Five Workflow Patterns

The critical insight: more complex doesn't mean better. According to Anthropic Engineering Blog (2024-12), the right approach is to “find the simplest solution possible, and only increase complexity when needed.” Start at the bottom of the staircase. Move up only when a simpler pattern can't handle your task.

Why exactly five? These aren't theoretical categories. They're patterns extracted from production systems Anthropic observed across customer projects. Other organizations (Microsoft, for example) have defined similar taxonomies. The five patterns here are deliberately simple and map cleanly to Claude Code's architecture.

The Building Block: The Augmented LLM

Every pattern is built from the same unit: an LLM enhanced with retrieval, tools, and memory. Anthropic calls this the “augmented LLM.”

Claude Code is a direct implementation of this concept:

- **Retrieval:** `Read`, `Grep`, and `Glob` tools for exploring codebases
- **Tools:** `Bash`, `Edit`, and `Write` for acting on files and systems
- **Memory:** `CLAUDE.md` and project memory for persistent context

Add MCP (Model Context Protocol) servers, and the augmented LLM gains access to databases, APIs, browsers, and any external system with an MCP integration. Every node in the patterns below is one of these augmented LLMs.

Pattern 1: Prompt Chaining

Decompose a task into a fixed sequence of steps, where each step's output feeds the next. Optionally insert programmatic checks (“gates”) between steps to verify the process stays on track.

```
[Step 1] -> Gate -> [Step 2] -> Gate -> [Step 3]
```

When to use it: The task breaks cleanly into fixed substeps, and you want accuracy over speed. Each LLM call handles a simpler subtask than one monolithic prompt would.

Claude Code implementation:

```
#!/bin/bash
# Prompt chaining: generate code, then validate, then test

# Step 1: Generate the change
claude -p "Refactor the auth module in src/auth.ts to use JWT" \
  --output-format json > step1.json

# Gate: type check
npx tsc --noEmit || { echo "Type errors. Aborting."; exit 1; }

# Step 2: Generate tests for the change
claude -p "Add unit tests for the JWT changes in src/auth.ts" \
  --output-format json > step2.json

# Gate: run tests
npm test
```

Why gates matter Without gates, errors compound through the chain. A type error in Step 1 propagates silently into Step 2, producing a test suite that validates broken code. Gates catch failures at each boundary, keeping the cost of a retry low. Breaking this causes: cascading errors where each step builds on the previous step's mistakes, resulting in output that's confidently wrong.

Pattern 2: Routing

Classify the input and direct it to a specialized handler. Each handler has a focused prompt optimized for its category.

```
[Input] -> [Classifier] -> Route A -> [Handler A]
                        -> Route B -> [Handler B]
                        -> Route C -> [Handler C]
```

When to use it: Distinct categories exist, each benefits from different handling, and classification is reliable.

Claude Code implementation:

```
#!/bin/bash
# Routing: direct tasks to the right model based on complexity

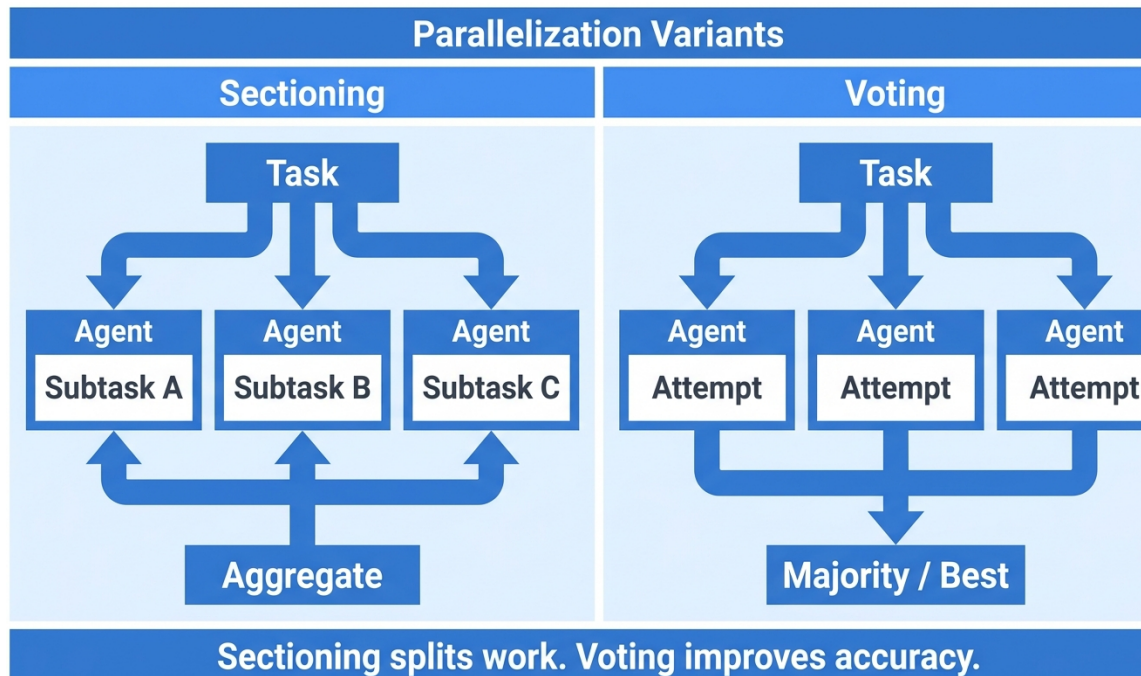
TASK_TYPE=$(claude -p "Classify this task as: simple_question, \
code_change, or architecture_design. Task: $USER_QUERY" \
--output-format json | jq -r '.classification')

case $TASK_TYPE in
  simple_question)
    claude -p "$USER_QUERY" --model haiku # fast, cheap
    ;;
  code_change)
    claude -p "$USER_QUERY" --model sonnet # balanced
    ;;
  architecture_design)
    claude -p "$USER_QUERY" --model opus # maximum capability
    ;;
esac
```

Why routing improves quality A single prompt optimized for every input type makes tradeoffs – improving performance on one category often hurts another. Routing separates concerns: each handler’s prompt is tuned for its specific category without compromise. Alternative: A single agent with a long system prompt covering all categories. This works for low-volume systems but degrades as input diversity grows.

Pattern 3: Parallelization

Run independent tasks simultaneously and aggregate results.



Parallelization Variants

Sectioning splits a task into independent subtasks that run in parallel:

```
[Task] -> [Subtask A] -> [Aggregate results]
        -> [Subtask B] ->
        -> [Subtask C] ->
```

Voting runs the same task multiple times for diverse outputs:

```
[Task] -> [Attempt 1] -> [Select best / majority vote]
        -> [Attempt 2] ->
        -> [Attempt 3] ->
```

When to use it: Subtasks are genuinely independent (sectioning), or you need higher confidence through multiple perspectives (voting).

Claude Code implementation:

Subagents are the natural mechanism for parallelization in Claude Code. When you ask Claude Code to review a pull request from multiple perspectives, it spawns separate subagents – each with its own context window – and runs them in parallel:

```
# Parallelized code review via subagents
claude -p "Review this pull request from three perspectives,
using a separate subagent for each:
1. Security vulnerabilities
2. Performance issues
3. Coding standard violations

Target: $(git diff main...HEAD)"
```

Claude Code receives this prompt, creates three subagents using the `Agent` tool, runs them concurrently, and synthesizes their findings.

Why parallelization is especially effective for LLMs According to Anthropic Engineering Blog (2024-12), “LLMs generally perform better when each consideration is handled by a separate LLM call, allowing focused attention on each specific aspect.” A single LLM call juggling security, performance, and style concerns will miss things that three focused calls catch.

Pattern 4: Orchestrator-Workers

A central LLM (the orchestrator) dynamically decomposes a task, delegates subtasks to workers, and synthesizes results.

```
[Orchestrator] -> [Worker A] -> [Synthesize]
                -> [Worker B] ->
                -> [Worker C] ->
```

How it differs from parallelization: In parallelization, subtasks are predefined. In orchestrator-workers, the orchestrator decides at runtime what subtasks are needed based on the input. The number and nature of workers aren’t known in advance.

When to use it: You can’t predict the subtasks ahead of time. A coding task might need changes to three files or thirty, depending on the request.

Claude Code implementation:

Claude Code itself is an orchestrator-worker system. When you say “migrate the authentication system to OAuth2,” Claude Code dynamically determines which files need changes, identifies dependencies between them, decides what can be parallelized, and delegates independent work to subagents.

```
# Orchestrator-worker: Claude Code decides the decomposition
claude -p "Migrate the auth system to OAuth2.
- First analyze the impact scope and identify all files needing changes
- Delegate independent file changes to subagents in parallel
- Handle dependent changes sequentially
- Run the full test suite after all changes complete"
```

The Anthropic research system from Chapter 1 is a production instance of this pattern. The lead agent (Claude Opus) analyzes queries and dynamically spawns subagents (Claude Sonnet) to investigate different aspects – the number and focus of subagents varies per query.

Why orchestrator-workers is the most versatile pattern According to Anthropic Engineering Blog (2024-12), this pattern is “well-suited for complex tasks where you can’t predict the subtasks needed.” Parallelization is simpler but rigid – it requires fixed subtask definitions. Orchestrator-workers trade that simplicity for the flexibility to handle open-ended, variable-structure tasks. Alternative: If your subtasks are predictable, use parallelization instead. The orchestrator adds overhead that isn’t justified for fixed-structure work.

Pattern 5: Evaluator-Optimizer

One LLM generates output while another evaluates it and provides feedback. The generator incorporates the feedback and tries again. The loop repeats until the evaluator approves or iteration hits the maximum count.

```
[Generator] -> [Output] -> [Evaluator] -> Feedback -> [Generator] (loop)
                        |
                        Pass -> [Final output]
```

When to use it: Clear evaluation criteria exist, iterative refinement produces measurable improvement, and the task is one where human feedback demonstrably helps (meaning LLM feedback can, too).

Claude Code implementation:

```
#!/bin/bash
# Evaluator-optimizer: iterative code improvement

MAX_ITERATIONS=3
ITERATION=0

while [ $ITERATION -lt $MAX_ITERATIONS ]; do
  # Generate or improve
  if [ $ITERATION -eq 0 ]; then
    claude -p "Optimize src/api/handler.ts for performance" \
      --output-format json > current.json
  else
    FEEDBACK=$(cat feedback.json)
    claude -p "Improve src/api/handler.ts based on this feedback: \
      $FEEDBACK" --output-format json > current.json
  fi

  # Evaluate with a separate call
  claude -p "Evaluate src/api/handler.ts against these criteria:
  - Response time optimization (1-10)
  - Memory efficiency (1-10)
  - Code readability (1-10)
  Pass threshold: all scores >= 7.
  Output JSON with scores and pass boolean." \
    --output-format json > feedback.json

  PASS=$(jq -r '.pass' feedback.json)
  if [ "$PASS" = "true" ]; then
    echo "Passed at iteration $ITERATION"
    break
  fi

  ITERATION=$((ITERATION + 1))
done
```

Why separate the generator and evaluator? When the same context generates and evaluates, the model tends to rate its own output favorably. A separate evaluation call brings fresh perspective. This mirrors human processes: writers benefit from editors, code benefits from code review. Breaking this causes: the loop converges prematurely because the evaluator “agrees” with the generator’s choices.

Beyond Workflows: Autonomous Agents

The five patterns above are workflows – systems where LLMs and tools follow predefined code paths. Anthropic draws a distinction between workflows and agents:

- **Workflows:** LLMs orchestrated through predefined code paths

- **Agents:** LLMs that dynamically direct their own processes and tool usage

Claude Code in interactive mode is an autonomous agent. You describe what you want. Claude Code plans, selects tools, executes, evaluates results, and adapts – all without a predefined path. The five workflow patterns are building blocks you use when you need more predictability than a fully autonomous agent provides.

Why the distinction matters Workflows offer predictability and consistency. Agents offer flexibility. According to Anthropic Engineering Blog (2024-12), “for many applications, optimizing single LLM calls with retrieval and in-context examples is usually enough.” Autonomous agents add cost and error-compounding risk. Use them for open-ended problems where you can’t predict the steps and you trust the model’s decision-making.

Choosing a Pattern

The right pattern depends on the structure of your task, not its difficulty. Use this decision flow:

```
Are the steps fixed and predictable?
YES -> Prompt chaining.
NO  -> Does the input fall into distinct categories?
      YES -> Routing.
      NO  -> Can it be split into independent subtasks?
              Subtasks known in advance -> Parallelization.
              Subtasks decided at runtime -> Orchestrator-workers.
      Does iterative refinement help?
      YES -> Evaluator-optimizer.
      NO  -> Autonomous agent.
```

The guiding principle: minimum viable complexity. Don’t use orchestrator-workers for a task that prompt chaining handles cleanly.

Combining Patterns

Production systems rarely use a single pattern in isolation. The Anthropic research system from Chapter 1 combines three:

- **Orchestrator-workers:** The lead agent decomposes queries and delegates to subagents.
- **Parallelization:** Multiple subagents run simultaneously.

- **Evaluator-optimizer:** The lead agent evaluates results and decides whether additional research rounds are needed.

The C compiler project combines two:

- **Parallelization:** 16 agents work on independent failing tests in parallel.
- **Evaluator-optimizer:** The test suite continuously evaluates code quality, driving agents toward passing tests.

Think of the five patterns as composable building blocks, not mutually exclusive categories. The chapters ahead show how to implement each pattern using Claude Code's subagent system (Chapters 3-5), orchestrator design (Chapter 6), and agent teams (Chapters 11-12).

Summary

- Anthropic defines five workflow patterns for agentic systems: prompt chaining, routing, parallelization, orchestrator-workers, and evaluator-optimizer.
- Each pattern trades simplicity for flexibility. Start with the simplest pattern that handles your task.
- Claude Code's subagent system provides native support for parallelization and orchestrator-worker patterns.
- Beyond workflows, autonomous agents (like Claude Code in interactive mode) dynamically control their own execution – use them for open-ended problems.
- Production systems typically combine multiple patterns. The five patterns are building blocks, not categories.

How Subagents Work

This chapter covers the mechanics of subagents – the mechanism Claude Code uses to spawn independent child agents, run them in isolated context windows, and collect their results. You’ll learn the `Agent` tool’s lifecycle, the three built-in subagent types, and the constraints that shape how you use them.

Companion code: [ch03-subagent-basics](#)

| What Subagents Are

The orchestrator-worker and parallelization patterns from Chapter 2 need a runtime mechanism. In Claude Code, that mechanism is the subagent.

A subagent is a child agent spawned within a Claude Code session. It gets its own context window, its own system prompt, and its own tool access. It runs its task independently, then returns only the result to the parent. The parent never sees the raw intermediate work – just the compressed summary.

The three benefits from Chapter 1 map directly to subagent mechanics: context isolation (each subagent has its own window), parallel execution (multiple subagents can run simultaneously), and specialization (each subagent can have a focused prompt and restricted tools).

Why subagents matter Subagents implement multi-agent patterns without external frameworks. You don't need a separate orchestration layer, a message queue, or a process manager. Claude Code's `Agent` tool handles spawning, communication, and cleanup internally. Alternative: The Claude Agent SDK provides a programmatic API for subagent creation (`agents` parameter in `query()` options). This book covers the CLI-native approach; Appendix B compares the two.

The Agent Tool

When Claude Code spawns a subagent, it internally calls the `Agent` tool. Understanding this tool's parameters helps you write better prompts that guide subagent behavior.

Parameters

The `Agent` tool accepts these parameters:

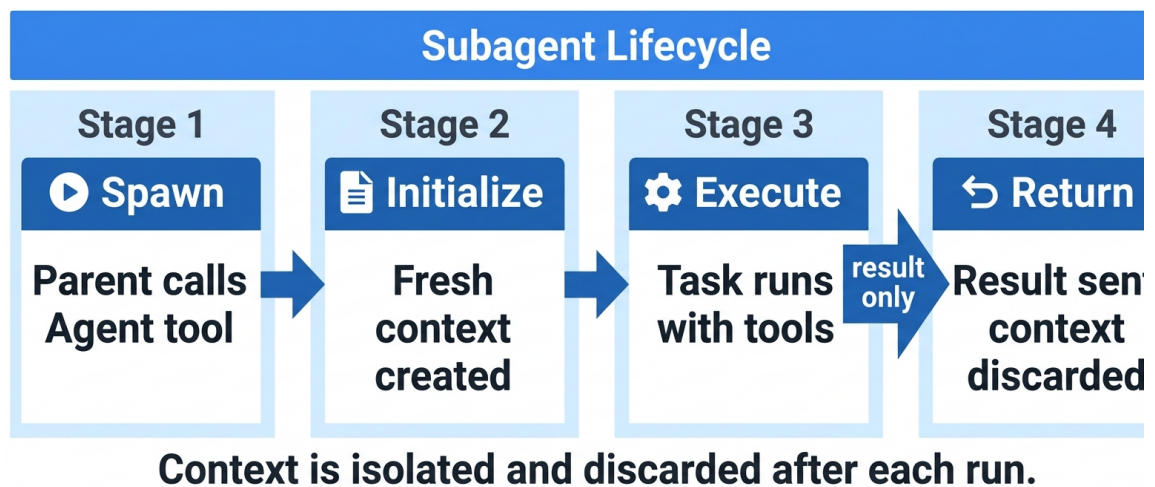
- `description`: A short label (3-5 words) displayed to the user. Helps you identify which subagent is running.
- `prompt`: The instructions for the subagent. This is the most important parameter – it determines what the subagent does and how.
- `subagent_type`: Which built-in subagent to use (e.g., `"Explore"`, `"Plan"`). Omit this for the general-purpose type.
- `model`: Override the model (`sonnet`, `opus`, `haiku`). Defaults to the parent's model.
- `isolation`: Set to `"worktree"` to run in a temporary Git worktree, keeping the main working tree untouched.
- `run_in_background`: Set to `true` for background execution. The parent continues working without waiting.

Claude Code sets these parameters automatically based on your prompt. You don't write JSON to invoke them directly. Instead, you influence subagent behavior through natural language (“use a separate subagent to investigate...”) or through subagent definition files (Chapter 4).

Why this design? Exposing raw JSON parameters to the user would create a brittle interface. Natural language delegation lets Claude Code choose the right parameters based on context – the same way a team lead assigns work by describing the task, not by filling out a form. Breaking this causes: users hard-coding parameters in prompts, which bypasses Claude Code’s optimization logic and often produces worse results.

The Subagent Lifecycle

A subagent goes through four phases:



Subagent Lifecycle

1. **Spawn**
The parent agent calls the Agent tool.
2. **Context initialization**
A fresh context window is created.
The subagent's system prompt + CLAUDE.md are loaded.
The full Claude Code system prompt is NOT loaded -- the subagent sees only its own prompt plus basic environment details like the working directory.
3. **Task execution**
The subagent uses its assigned tools to complete the task.
Its work doesn't touch the parent's context.
4. **Result return**
The subagent returns its result to the parent.
The subagent's context is not merged into the parent's -- only the returned summary lands in the parent's window.
(The transcript itself is persisted on disk, and you can resume a subagent via the experimental agent teams path; see Chapter 11.)

The key detail is step 4: the subagent's context is destroyed after it returns. The parent receives a text summary, not the full content of every file the subagent read. A subagent that reads 50 files adds only a paragraph of results to the parent's context – not 50 files' worth of tokens.

Why context disposal? This is the “intelligent filter” mechanism from Chapter 1. Subagents compress information by doing extensive work in their own context and returning only what matters. Without disposal, spawning subagents would just move the context bloat problem from one window to another.

Foreground and Background Execution

Subagents run in one of two modes:

Foreground (default): The parent waits for the subagent to finish before continuing. Use this when the parent needs the result to decide its next step.

Background: The parent continues working immediately. It receives a notification when the subagent completes. Use this for independent tasks that can run in parallel.

```
Foreground:
  Parent -> [subagent running...] -> result -> next step

Background:
  Parent -> spawn subagent -> continue own work
           |                   |
           |                   |
  [running...]             [working...]
           |                   |
  done -----> check result
```

Background execution is how you implement the parallelization pattern from Chapter 2. Spawn three subagents in background mode, and all three run concurrently – each in its own context window, each working independently.

Built-in Subagents

Claude Code includes three primary built-in subagents. It automatically selects the right one based on your prompt.

Explore

- Model: Haiku (fast, cheap)
- Tools: Read-only (no Write, no Edit)

- Purpose: File discovery, code search, codebase exploration

Explore handles tasks where Claude needs to search or understand a codebase without making changes. Explore supports three thoroughness levels – `quick`, `medium`, and `very thorough` – selected automatically based on query complexity.

Plan

- Model: Inherits from parent
- Tools: Read-only
- Purpose: Codebase research during plan mode

In plan mode, Claude delegates research to this subagent – gathering codebase context before presenting a plan.

General-purpose

- Model: Inherits from parent
- Tools: All tools
- Purpose: Complex tasks requiring both exploration and modification

General-purpose runs when the task needs reading, analysis, and action – the full set of capabilities.

Why Explore uses Haiku while others inherit the parent model

Codebase exploration is primarily pattern matching and information extraction – it doesn't require deep reasoning. Haiku runs faster and costs significantly less than Opus or Sonnet. For exploration tasks that Claude runs frequently, this cost difference adds up. Plan and General-purpose inherit the parent model because planning and implementation require the reasoning capability of the model you chose for your session.

The automatic selection isn't deterministic – it depends on prompt wording. “Show me the project structure” routes to Explore. “Analyze the project

structure and suggest improvements” likely routes to General-purpose. Being aware of this helps you write prompts that trigger the right subagent.

Subagents in Practice

The companion repository includes a sample codebase and prompts for these three examples. See `ch03-subagent-basics/demo-prompts.md` for the full prompts.

Example 1: Codebase Investigation

You ask Claude Code to migrate an auth middleware to JWT. Before making changes, it needs to understand the current implementation.

Claude Code spawns an Explore subagent with a focused prompt: “Identify all authentication-related files and summarize how authentication is used in each.” The Explore subagent reads files in its own context, discovers four relevant files (`auth.ts`, `protected.ts`, `token.ts`, `auth.test.ts`), and returns a concise summary.

The parent agent receives the summary and plans its changes – without the raw file contents consuming its context window.

Example 2: Parallel Review

You ask Claude Code to review a pull request. It spawns three general-purpose subagents in background mode, each reviewing from a different perspective: security, performance, and coding standards.

All three run simultaneously in independent contexts. When they finish, the parent synthesizes their findings into a unified report. Three focused reviews run faster and catch more than a single review juggling all three concerns.

Example 3: Git Worktree Isolation

You want to experiment with a performance optimization without risking your working tree. Claude Code spawns a subagent with `isolation: "worktree"`. The subagent works in a temporary Git worktree – a separate copy of your repository. If the experiment succeeds, it returns the worktree path and branch name so you can merge the changes. If it fails or makes no changes, the worktree is automatically cleaned up.

Why worktree isolation? Experimental changes are inherently risky. Worktree isolation lets you try ideas without the overhead of manual `git stash` or branch juggling. Chapter 9 revisits worktree isolation in the context of long-running agents, where it becomes even more valuable.

Subagent Constraints

Subagents have three design constraints. Understanding them prevents frustration.

No Nesting

A subagent can't spawn another subagent. The hierarchy is exactly one level deep.

```
Parent agent
+-- Subagent A (allowed)
|   +-- Subagent B (NOT allowed)
+-- Subagent C (allowed)
```

Why no nesting? Unrestricted nesting creates uncontrollable cost growth and makes debugging nearly impossible. A single level of delegation provides enough decomposition for most tasks while keeping the execution tree predictable. Breaking this causes: runaway token consumption and opaque failure chains where you can't tell which level of nesting produced an error.

No Shared Context

Subagents can't see each other's context. Subagent A and Subagent B operate in complete isolation. If Subagent B needs information from Subagent A's results, the parent must include that information in Subagent B's prompt.

```
Subagent A  <-X->  Subagent B
    |               |
    v               v
Parent (must relay information)
```


For tasks that require agents to share findings and challenge each other’s conclusions, agent teams (Chapters 11-12) provide direct inter-agent communication.

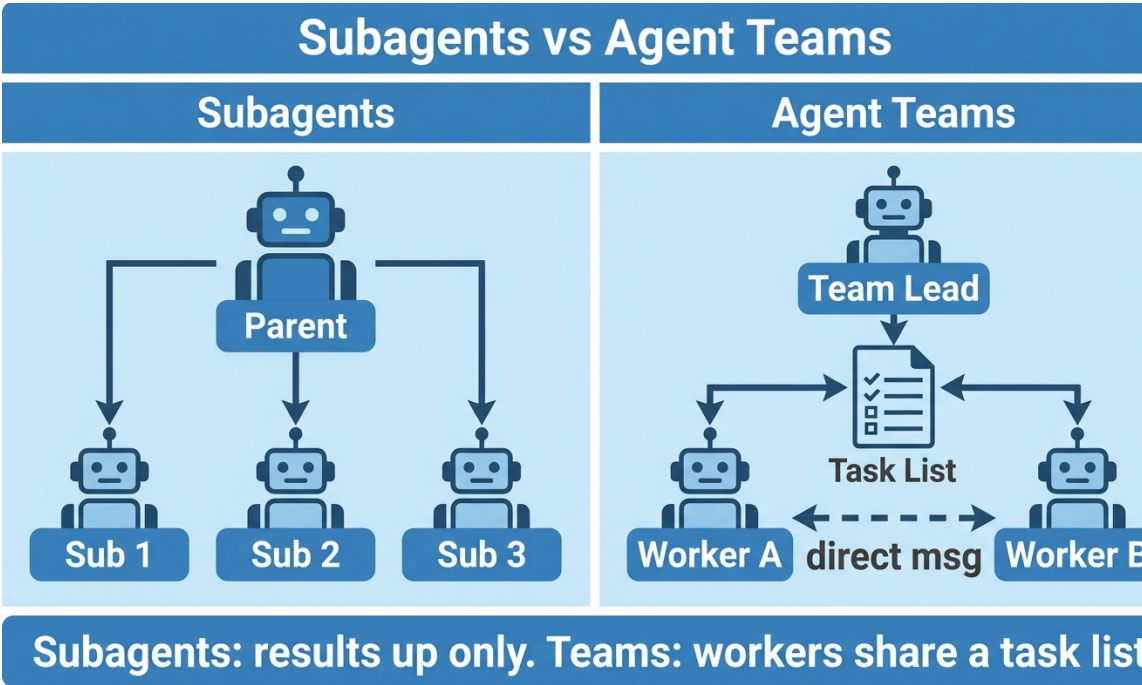
No Automatic Persistence Across Invocations

By default, each subagent invocation creates a new instance with a fresh context. If you spawn the “same” subagent again, it starts without memory of the previous run.

Two escape hatches exist. Chapter 4 covers the `memory` field in subagent definitions, which gives a subagent a persistent directory that survives across conversations. For continuing an in-flight subagent rather than starting a new one, Claude Code exposes a resume mechanism via the `SendMessage` tool – but that tool is only available when the experimental agent teams path is enabled (`CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS=1`; see Chapter 11). In the common case, treat each subagent invocation as a clean start.

Subagents vs. Agent Teams

Claude Code offers two multi-agent mechanisms. Choosing the right one depends on whether your workers need to communicate.



Subagents vs Agent Teams

Subagents run within a single session. Each subagent reports results only to the parent. The parent manages all coordination. Subagents are best for focused tasks where only the result matters – investigations, reviews, isolated experiments.

Agent teams coordinate across separate sessions. Teammates share a task list, claim work, and message each other directly. They’re best for complex work requiring discussion and collaboration – competing hypotheses, cross-layer coordination, architectural exploration.

At time of writing: Agent teams are experimental and disabled by default. Enable with `CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS` in `settings.json`. Requires Claude Code v2.1.32 or later.

The cost difference is significant. Subagent results are compressed back into the parent’s context (lower token usage). Each agent team member is a fully independent Claude instance (higher token usage). Default to subagents. Use agent teams when workers genuinely need to communicate with each other.

Common Failure Patterns

“Subagent ignored my instructions”: The subagent didn’t receive the prompt you expected. Claude Code auto-generates the `prompt` parameter based on your natural language request. If the generated prompt is too vague, be more explicit about what you want the subagent to do. For critical tasks, use a custom subagent definition (Chapter 4) with a precise system prompt.

“Subagent took too long”: A general-purpose subagent with access to all tools may explore more broadly than necessary. Restrict tools (Chapter 4) or use Explore for read-only tasks. You can also set `maxTurns` to cap the subagent’s execution length.

“Results were too shallow”: The subagent’s model may lack the reasoning depth needed for the task. Check whether Explore (Haiku) was selected when you needed General-purpose (your session’s model). Rephrase the prompt to signal analysis, not just search.

“Background subagents disappeared”: Background subagents notify the parent when they complete. If you end the session before they finish, their results are lost. Check active subagents with `/agents` before ending a session.

Summary

- Subagents are child agents spawned within a Claude Code session, each with its own context window. They implement the multi-agent patterns from Chapter 2.
- The `Agent` tool manages spawning, with parameters for description, prompt, model, isolation, and background execution. Claude Code sets these automatically based on your natural language instructions.
- Three built-in subagents cover the common cases: Explore (fast, read-only search), Plan (research for planning), and General-purpose (full capabilities).
- Subagent context is destroyed after the task completes. Only the result text returns to the parent – this is the compression mechanism that makes multi-agent systems efficient.
- Foreground execution blocks the parent; background execution enables parallelism.
- Three constraints shape subagent usage: no nesting (one level only), no shared context (parent must relay), and no session persistence (fresh start each time).
- For tasks requiring inter-agent communication, use agent teams (Chapters 11-12) instead of subagents.

Building Custom Subagents

Chapter 3 covered built-in subagents. This chapter teaches you to build your own – project-specific subagents defined as Markdown files in `.claude/agents/`. You'll learn the definition format, tool restrictions, model selection, persistent memory, and how to combine these features into production-ready subagent configurations.

Companion code: [ch04-custom-subagent](#)

The Subagent Definition Format

A custom subagent is a Markdown file with YAML frontmatter:

```
---
name: code-reviewer
description: Reviews code for quality and best practices
tools: Read, Glob, Grep
model: sonnet
---

You are a code reviewer. When invoked, analyze the code and
provide specific, actionable feedback on quality, security,
and best practices.
```

The frontmatter (`---` block) configures the subagent. The body becomes the system prompt. This format is version-control friendly – diffs are readable, and configuration lives alongside instructions in a single file.

Required Fields

Only two fields are mandatory:

- **name**: A unique identifier using lowercase letters and hyphens (e.g., `code-reviewer`).
- **description**: Tells Claude Code when to delegate to this subagent.

The `description` field is critically important. Claude Code reads every subagent's description and selects the best match for the current task. A vague description means the subagent won't be selected when you expect it.

```
# Vague -- Claude Code can't tell when to use this
description: Helps with code

# Specific -- Claude Code knows exactly when to delegate
description: >
  Reviews code for quality and best practices.
  Use proactively after code changes.
```

Why description drives selection The main agent treats each subagent's `description` as its contract. When a task arrives, the main agent scans all descriptions and delegates to the best match. A well-written description is the difference between a subagent that gets used and one that's ignored.

Scope and Priority

Store subagent files in different locations depending on how broadly they should apply. When multiple subagents share the same name, the higher-priority location wins.

- **Project** (`.claude/agents/`): Available to everyone working on this project. Check these into version control.
- **User** (`~/.claude/agents/`): Your personal subagents, available in all projects.
- **CLI flag** (`--agents '{...}'`): Session-only definitions for testing or automation scripts. Gone when the session ends.

```
your-project/
+-- .claude/
|   +-- agents/
|       +-- code-reviewer.md
|       +-- test-writer.md
|       +-- security-auditor.md
+-- src/
```

Why separate scopes? Project-specific rules belong in the project. Personal preferences belong in your user directory. This separation keeps subagent definitions focused – a team's security auditor doesn't need your personal coding style agent, and vice versa.

You can also create subagents interactively with the `/agents` command, which provides a guided setup flow including Claude-generated prompts.

Tool Restrictions

Controlling which tools a subagent can access is the primary safety mechanism.

Allow List

The `tools` field explicitly lists permitted tools:

```
---
name: safe-researcher
description: Research agent with read-only access
tools: Read, Grep, Glob, Bash
---
```

This subagent can read and search but can't write or edit files.

Deny List

The `disallowedTools` field removes specific tools while inheriting everything else:

```
---
name: no-writes
description: Full capabilities except file modifications
disallowedTools: Write, Edit
---
```

Why two mechanisms? Use `tools` when you want a short allow list. Use `disallowedTools` when you want most tools but need to exclude a few. If you specify both, `disallowedTools` is applied after `tools`.

Restricting Subagent Spawning

When running a subagent as the main agent via `--agent`, you can restrict which other subagents it can spawn:

```
tools: Agent(worker, researcher), Read, Bash
```

This limits the agent to spawning only `worker` and `researcher` subagents.

Model Selection

The `model` field controls which Claude model the subagent uses:

```
model: haiku      # Fast, low cost -- exploration, classification
model: sonnet     # Balanced -- code review, test generation
model: opus       # Maximum capability -- architecture, reasoning
model: inherit    # Same as parent session (default)
```

Matching the model to the task is a cost optimization lever. A codebase exploration subagent doesn't need Opus. A security audit subagent shouldn't run on Haiku.

Why model selection matters in multi-agent systems According to Anthropic Engineering Blog (2025-06), model choice is one of three factors (alongside token usage and tool calls) that explain 95% of performance variance. Routing simple tasks to cheaper models and complex tasks to capable models optimizes the cost-performance ratio across the entire system.

Permission Modes

The `permissionMode` field controls how the subagent handles permission requests:

- `default`: Standard permission prompts
- `acceptEdits`: Auto-approve file edits (except protected directories)
- `plan`: Read-only mode
- `bypassPermissions`: Skip all permission checks

For automated workflows, `acceptEdits` works well for subagents that write tests or fix lint errors – tasks where you trust the changes. For sensitive operations like database access, keep `default` to require explicit approval.

Persistent Memory

By default, a subagent starts fresh every time (Chapter 3). The `memory` field changes this – it gives the subagent a persistent directory where it

accumulates knowledge across conversations.

```
---
name: project-expert
description: Deep knowledge agent for this project
memory: project
---

You are a project expert. Before starting any task:
1. Check your memory for relevant past findings
2. Use this context to provide better answers

After completing each task:
1. Save new discoveries to your memory
2. Update patterns and conventions you identified
3. Note any architectural decisions and their rationale
```

Three memory scopes are available:

- **user** : Stored in `~/.claude/agent-memory/<name>/` . Shared across all projects.
- **project** : Stored in `.claude/agent-memory/<name>/` . Project-specific, can be version-controlled.
- **local** : Stored in `.claude/agent-memory-local/<name>/` . Project-specific, excluded from version control.

When memory is enabled, the subagent's `MEMORY.md` file (first 200 lines) is loaded at startup, and Read/Write/Edit tools are automatically available for memory management.

Why persistent memory? After the first run, the subagent starts with accumulated knowledge – “this project uses the Result type for error handling,” “the team prefers integration tests over mocks.” Each invocation becomes more effective than the last.

MCP Server Integration

The `mcpServers` field connects MCP (Model Context Protocol) servers to the subagent:


```

---
name: browser-tester
description: Tests features in a real browser
mcpServers:
  - playwright:
      type: stdio
      command: npx
      args: ["-y", "@playwright/mcp@latest"]
  - github
---

```

Use the Playwright tools to navigate, screenshot, and test.

Inline MCP definitions (like `playwright` above) start when the subagent launches and stop when it finishes. Named references (like `github`) connect to servers already configured in `settings.json`. Chapter 5 covers MCP integration patterns in detail.

Why scope MCP servers to subagents? Token-heavy MCP servers like Playwright add tool definitions to the context window. Scoping them to the subagent that actually needs them keeps the main session's context clean.

Hooks for Fine-Grained Control

The `hooks` field adds lifecycle hooks scoped to the subagent. The most common use case is `PreToolUse` – validating tool inputs before execution.

```

---
name: db-reader
description: Execute read-only database queries
tools: Bash
hooks:
  PreToolUse:
    - matcher: "Bash"
      hooks:
        - type: command
          command: "./scripts/validate-readonly-query.sh"
---

```

The validation script blocks SQL write operations:

```
#!/bin/bash
# PreToolUse hook: block SQL write operations

INPUT=$(cat)
COMMAND=$(echo "$INPUT" | jq -r '.tool_input.command // empty')

# Block SQL write operations (case-insensitive)
if echo "$COMMAND" | grep -iE \
  '\b(INSERT|UPDATE|DELETE|DROP|CREATE|ALTER|TRUNCATE)\b' \
  > /dev/null; then
  echo "Blocked: Only SELECT queries are allowed"
  exit 2 # exit 2 = block the tool call
fi

exit 0 # exit 0 = allow the tool call
```

Why hooks instead of tool restrictions? The `tools` field works at the tool level – allow Bash or don’t. Hooks work at the input level – allow Bash, but block specific commands. This granularity is necessary for cases like “the subagent needs Bash for queries but must never execute writes.”

Practical Example: A Multi-Subagent Project

Combining the features from this chapter, here’s how a project might organize its subagents:

```
your-project/
+-- .claude/
|   +-- agents/
|       +-- test-writer.md           # Sonnet, write access
|       +-- security-auditor.md      # Sonnet, read-only
|       +-- doc-generator.md         # Haiku, write access
|       +-- perf-analyzer.md         # Sonnet, Bash allowed
```

When you say “implement this feature and review it,” the main agent:

1. Implements the feature itself.
2. Delegates to `test-writer` for test generation.
3. Delegates to `security-auditor` for vulnerability scanning.
4. Synthesizes all results into a unified report.

Each subagent runs at the appropriate model tier with the minimum required tools. This is the “specialization through decomposition” principle

from Chapter 1, made concrete at the project level.

Common Failure Patterns

“My subagent is never selected”: The `description` doesn’t match the tasks you’re assigning. Make it specific and include trigger phrases like “Use proactively when...” or “Use after...”.

“Subagent has too many tools”: Omitting the `tools` field gives the subagent access to everything. For read-only agents, always specify `tools: Read, Grep, Glob` explicitly.

“Memory isn’t loading”: Memory requires the subagent’s `MEMORY.md` to exist. The first run creates it. If it’s not loading, check that the memory directory exists and the scope matches (`project` vs `user` vs `local`).

“Hook script isn’t running”: Hooks require the script to be executable (`chmod +x`). Check the path is relative to the project root and the script reads from stdin (hooks pipe tool input via stdin).

Going Further

The Claude Code community maintains open-source collections of subagent definitions, agent team configurations, and orchestration patterns on GitHub. Search for “awesome claude code” to find curated lists that complement the patterns in this chapter.

The companion repository for this book includes working examples:

<https://github.com/driftnode986/claude-multiagent-guide-en-samples>

Summary

- Custom subagents are Markdown files with YAML frontmatter. Only `name` and `description` are required. Store them in `.claude/agents/` (project) or `~/.claude/agents/` (user).
- The `description` field is the selection mechanism – write specific, actionable descriptions so Claude Code delegates to the right subagent.
- Tool restrictions (`tools` and `disallowedTools`) enforce least-privilege access. Hooks add input-level validation for finer control.
- Model selection optimizes cost: Haiku for exploration, Sonnet for analysis, Opus for complex reasoning.

- Persistent memory lets subagents accumulate project knowledge across conversations.
- MCP servers scoped to subagents keep token-heavy tool definitions out of the main session's context.
- Check project subagents into version control so teams share and improve them collaboratively.

[OceanofPDF.com](https://oceanofpdf.com)

Connecting Subagents to External Tools via MCP

Chapters 3 and 4 covered subagent fundamentals and custom definitions. This chapter connects subagents to MCP (Model Context Protocol) servers – the mechanism that gives Claude Code access to databases, browsers, APIs, and other external systems. You’ll learn two connection methods, scope control, safety hooks, and design patterns for combining MCP servers with subagents.

Companion code: [ch05-subagent-mcp](#)

| MCP Servers and Subagents

MCP servers expose external tools to Claude Code. Database queries, browser automation, Slack messages, GitHub operations – these capabilities are impossible without MCP. On their own, subagents can only use built-in tools like Read, Write, Grep, and Bash.

Combining subagents with MCP creates agents that are both specialized *and* externally connected. A subagent with Playwright MCP can navigate real web pages. A subagent with GitHub MCP can review pull requests and post comments. The subagent’s prompt provides the expertise; MCP provides the reach.

The cost is real, though. Every MCP server adds tool definitions to the context window. Playwright MCP alone defines over 20 tools – thousands of tokens. According to Anthropic Engineering Blog (2025-09), excessive tool definitions cause agents to choose suboptimal strategies. The key design question is: where do those tool definitions live?

Why scope MCP servers to subagents? Inline MCP definitions load only when the subagent runs and unload when it finishes. The main session's context stays clean. This is the same resource isolation principle from Chapter 3, applied to tool definitions.

Two Connection Methods

A subagent connects to MCP servers in two ways: by reference or by inline definition.

Reference: Sharing an Existing Server

When the main session already has an MCP server configured, the subagent can reference it by name:

```
---
name: slack-reporter
description: Post reports to the appropriate Slack channel
mcpServers:
  - slack
---

Post the report to the relevant Slack channel.
```

The subagent shares the main session's MCP connection. This is simple but has a tradeoff: the tool definitions already exist in the main session's context, so you don't save any context.

Why use references? References work best for MCP servers that the main session uses frequently anyway – GitHub, Slack, or Linear. The tool definitions are already in context. Adding them to a subagent by reference costs nothing extra.

Inline Definition: A Subagent-Local Server

Inline definitions configure the MCP server directly in the subagent's frontmatter:

```

---
name: browser-tester
description: Run browser tests with Playwright
mcpServers:
  - playwright:
      type: stdio
      command: npx
      args: ["-y", "@playwright/mcp@latest"]
---

```

Use the Playwright tools to navigate pages and validate behavior.

The MCP server starts when the subagent launches and stops when it finishes. The tool definitions exist only in the subagent's context window – the main session never sees them.

Why prefer inline definitions? Inline definitions are the default choice for MCP servers that only one subagent needs. Playwright, database connectors, monitoring dashboards – these are task-specific tools. Keeping them inline avoids polluting the main session's context with irrelevant tool definitions. Alternative: Reference the server from `settings.json`. This is simpler but adds tool definitions to every session, whether they're needed or not.

Mixing Both Methods

A single subagent can use both connection methods:

```

---
name: issue-triage
description: Triage GitHub issues and sync with Linear
mcpServers:
  - github
  - linear:
      type: stdio
      command: npx
      args: ["-y", "@linear/mcp-server"]
      env:
        LINEAR_API_KEY: "${LINEAR_API_KEY}"
---

```

Check GitHub issues, cross-reference with Linear projects, and set appropriate labels and priority.

Here `github` is a reference to the main session's existing connection. `linear` is an inline definition scoped to this subagent. The mixed approach is common when a subagent needs one widely-used service plus one task-specific one.

Tool Inheritance and Scope Control

Default Behavior

When no `tools` field is specified, a subagent inherits all tools from the main session – including MCP tools:

```
---
name: general-helper
description: Handle miscellaneous tasks
# No tools field = inherits everything (MCP included)
---
```

If the main session has `github` and `slack` MCP servers connected, this subagent can use both.

Allowlisting with `tools`

The `tools` field acts as an allowlist. Only listed tools are available:

```
---
name: read-only-analyzer
description: Analyze code without making changes
tools: Read, Grep, Glob
---
```

This subagent cannot use any MCP tools. The `tools` field is strict – if a tool is not listed, it is blocked. This is the least-privilege principle from Chapter 4.

Allowing Specific MCP Tools

To allow particular MCP tools, include them in the `tools` field. MCP tool names follow the pattern `mcp__<server>__<tool>`:


```

---
name: github-reader
description: Read-only access to GitHub issues and PRs
tools: Read, Grep, Glob, mcp_github_list_issues,
      mcp_github_get_pull_request
---

```

This subagent can read issues and PRs but cannot create comments, merge PRs, or perform any other GitHub operation. Fine-grained MCP tool selection is useful when a server exposes both read and write capabilities but the subagent should only read.

Why restrict MCP tools? According to Anthropic Engineering Blog (2025-09), agents with fewer tools make better decisions. A subagent with access to 50 GitHub tools will occasionally choose the wrong one. A subagent with access to 2 specific tools won't. Breaking this causes: unintended writes (a “read-only” agent that accidentally closes an issue) and wasted context on tool definitions the agent never uses.

Blocklisting with `disallowedTools`

The inverse approach: inherit everything *except* specific tools:

```

---
name: safe-browser
description: Browse the web without modifying local files
disallowedTools: Write, Edit
mcpServers:
  - playwright:
      type: stdio
      command: npx
      args: ["-y", "@playwright/mcp@latest"]
---

```

This subagent inherits all tools (including Playwright MCP) but cannot write or edit files. Use `disallowedTools` when you want broad access minus a few dangerous operations.

Practical Example: Web Scraping Agent

Build a subagent that scrapes data from web pages using Playwright MCP. Create `.claude/agents/web-scraper.md`:

```
---
name: web-scraper
description: Scrape data from web pages.
  Specify a URL to begin extraction
tools: Read, Write, Bash
model: sonnet
mcpServers:
  - playwright:
      type: stdio
      command: npx
      args: ["-y", "@playwright/mcp@latest"]
---
```

You are a web scraping specialist.

Procedure

1. Navigate to the specified URL with Playwright
2. Take a snapshot to understand page structure
3. Extract the requested data
4. Save results in JSON format

Constraints

- Check robots.txt and confirm scraping is permitted
- Space requests to avoid overloading the server
- Do not collect personal information
- Save results to the `output/` directory

Invoke it from the main session:

```
You: Scrape all product names and prices
    from https://example.com/products

Claude Code: Delegating to web-scraper subagent...
```

Claude Code matches the task to `web-scraper` based on its description. The subagent's execution flow:

1. Playwright MCP starts (subagent launch).
2. `browser_navigate` opens the URL.
3. `browser_snapshot` captures page structure.
4. The agent extracts data using `browser_click` and `browser_scroll` as needed.
5. Results are saved as JSON.
6. Playwright MCP shuts down (subagent exit).

The critical point: Playwright's 20+ tool definitions never appear in the main session's context. This is the context isolation benefit of inline MCP definitions.

Practical Example: Code Review Agent

Build a subagent that reviews pull requests using GitHub MCP. Create `.claude/agents/pr-reviewer.md`:

```
---
name: pr-reviewer
description: Review a GitHub PR. Analyze the diff
  and post review comments
tools: Read, Grep, Glob, Bash
model: opus
mcpServers:
  - github
---
```

You are an experienced code reviewer.

Review Criteria

1. **Security**: SQL injection, XSS, authentication
2. **Performance**: N+1 queries, loops, memory leaks
3. **Maintainability**: Naming, function length, SRP
4. **Testing**: Coverage, edge cases

Output Format

- Report critical issues as BLOCKER
- Report improvement suggestions as SUGGESTION
- Include the line number and a proposed fix

GitHub Integration

- Fetch the PR diff and analyze it
- Post review comments directly on the PR
- Approve if no issues are found

Invoke it:

```
You: Review PR #42

Claude Code: Delegating to pr-reviewer subagent...
```

This example uses a reference (`github`) rather than an inline definition. GitHub MCP is typically configured at the session level because the main session uses it too. Code review demands strong reasoning – detecting

security vulnerabilities and architectural issues – so Opus is the right model choice.

Why Opus for code review? Code review requires identifying subtle security flaws, spotting architectural drift, and understanding cross-file dependencies. Sonnet handles straightforward tasks well, but review quality improves significantly with Opus-level reasoning. The per-review cost is justified by the value of catching a vulnerability before it ships.

Practical Example: Read-Only Database Agent

A subagent that queries a database needs strict safety controls. The MCP server provides access; hooks enforce boundaries.

```
---
name: db-analyst
description: Execute SQL queries and produce analysis reports
tools: Bash, Read, Write
model: sonnet
mcpServers:
  - postgres:
      type: stdio
      command: npx
      args: ["-y", "@modelcontextprotocol/server-postgres"]
      env:
        POSTGRES_CONNECTION_STRING: "${DATABASE_URL}"
hooks:
  PreToolUse:
    - matcher: "Bash"
      hooks:
        - type: command
          command: "./scripts/validate-readonly-query.sh"
---
```

You are a data analyst. Query the database and produce clear, actionable reports.

Constraints

- SELECT queries only (INSERT/UPDATE/DELETE are blocked)
- Limit results to 1,000 rows per query
- Mask personally identifiable information in output

Two safety mechanisms work together here:

Environment variables: The `${DATABASE_URL}` syntax pulls the connection string from the environment. Credentials never appear in the subagent definition file, which is safe to check into version control.

PreToolUse hooks: The validation script inspects every Bash command before execution:

```
#!/bin/bash
# PreToolUse hook: block SQL write operations

INPUT=$(cat)
COMMAND=$(echo "$INPUT" | jq -r '.tool_input.command // empty')

# Block SQL write operations (case-insensitive)
if echo "$COMMAND" | grep -iE \
  '\b(INSERT|UPDATE|DELETE|DROP|CREATE|ALTER|TRUNCATE)\b' \
  > /dev/null; then
  echo "Blocked: only SELECT queries are allowed"
  exit 2 # exit 2 = block the tool call
fi

exit 0 # exit 0 = allow the tool call
```

The `tools` field (Chapter 4) controls access at the tool level. Hooks control access at the *input* level. “Bash is allowed, but only for SELECT queries” is a constraint that requires hooks.

Why hooks for database safety? The `tools` field is binary: allow Bash or don’t. But the subagent needs Bash to run SQL queries. Hooks add the missing granularity – Bash is available, but destructive SQL is blocked before it reaches the database. Breaking this causes: accidental data mutations. A subagent “analyzing” sales data could accidentally DELETE records if the only protection is a prompt instruction.

MCP Scope Design Patterns

Three patterns emerge for organizing MCP servers across subagents.

Specialization: One MCP per Subagent

Each subagent owns exactly one MCP server:

```
[Main agent]
+-- web-scraper (Playwright MCP)
+-- db-analyst (PostgreSQL MCP)
+-- slack-reporter (Slack MCP)
+-- github-reviewer (GitHub MCP)
```

Each subagent is optimized for one external system. Its prompt, tool restrictions, and model selection all target that specific integration. This is the most common pattern and the easiest to reason about.

Integration: Multiple MCPs per Subagent

A single subagent combines multiple MCP servers for cross-system workflows:

```
---
name: deployment-checker
description: Verify application health after a deployment
mcpServers:
  - playwright:
      type: stdio
      command: npx
      args: ["-y", "@playwright/mcp@latest"]
  - datadog:
      type: stdio
      command: npx
      args: ["-y", "@datadog/mcp-server"]
---
```

After a deployment, verify the following:

1. Use Playwright to navigate critical frontend screens
2. Check error rates and latency in Datadog
3. Report any anomalies immediately

Use this pattern when a single workflow crosses system boundaries. Post-deploy verification naturally spans both the browser (user-facing behavior) and monitoring (backend metrics). Splitting this across two subagents would fragment the workflow.

Why limit to two or three MCP servers? According to Anthropic Engineering Blog (2025-09), agents perform better with fewer tools. Each MCP server adds tool definitions to the subagent's context. Beyond two or three servers, the tool count becomes a drag on decision quality. If you need more than three, consider splitting into multiple subagents.

Escalation: Lightweight First, Heavy Later

Start with a cheap subagent. Escalate to an expensive one only if needed:

Step 1: code-analyzer (no MCP, Read/Grep only)
-> Problem detected

Step 2: browser-debugger (Playwright MCP)
-> Reproduce in browser

Step 3: monitoring-checker (Datadog MCP)
-> Check production metrics

Why escalate? MCP servers have startup overhead. Playwright launches a browser process. Database connectors open TCP connections. Starting everything for a task that might only need static analysis wastes time and resources. Escalation defers the cost until it's justified by actual findings.

Best Practices

Prefer inline definitions. If only one subagent needs the MCP server, define it inline. This keeps the main session's context clean and makes the subagent self-contained.

Manage credentials with environment variables. Use `${ENV_VAR}` syntax for API keys and connection strings. The subagent definition file should be safe to commit to version control.

Add hooks for write-capable MCP servers. Any MCP server that can modify external state – databases, issue trackers, messaging systems – should have a PreToolUse hook that validates inputs. Prompt instructions (“only use SELECT”) are not reliable safety controls.

Test against a staging environment first. Switch the `env` field to point at a test environment before targeting production:

```
mcpServers:
  - postgres:
      type: stdio
      command: npx
      args: ["-y", "@modelcontextprotocol/server-postgres"]
      env:
        POSTGRES_CONNECTION_STRING: "${TEST_DATABASE_URL}"
```

Keep tool counts low. Restrict MCP tools with the `tools` field when a server exposes more capabilities than the subagent needs. Two or three MCP servers per subagent is a practical ceiling.

In the Agent SDK: The same MCP patterns apply. Pass `mcpServers` in the `options` object when calling `query()`. Use `allowedTools` with wildcards (`mcp_github_*`) or specific tool names for fine-grained access. The SDK supports stdio, HTTP, and SSE transports – inline definitions in CLI correspond to inline `mcpServers` in SDK code.

Common Failure Patterns

“MCP tools aren’t available”: The `tools` field is an allowlist. If you specified `tools: Read, Grep, Glob`, MCP tools are blocked. Either omit `tools` to inherit everything, or add the specific MCP tool names you need.

“Connection refused” on MCP startup: The MCP server binary isn’t installed. For `npx`-based servers, ensure Node.js is available. For HTTP-based servers, check the URL and authentication headers.

“Subagent is slow to start”: Each inline MCP server starts a process at subagent launch. Multiple inline servers compound the startup time. Consider the escalation pattern if startup latency is a problem.

“Hook script isn’t blocking”: The script must read from stdin, parse JSON with `jq`, and exit with code 2 to block. Exit code 0 means “allow.” Check that the script is executable and the path is relative to the project root.

Summary

- Subagents connect to MCP servers via reference (shared with main session) or inline definition (subagent-local). Inline definitions are the

default choice for task-specific MCP servers.

- The `tools` field allowlists MCP tools by name (`mcp__<server>__<tool>`). The `disallowedTools` field blocklists specific tools while inheriting the rest.
- PreToolUse hooks enforce input-level safety controls that the `tools` field cannot express, such as “allow Bash but block SQL writes.”
- Store credentials in environment variables (`${ENV_VAR}`), not in subagent definition files.
- Three design patterns organize MCP across subagents: specialization (one MCP per subagent), integration (multiple MCPs for cross-system workflows), and escalation (lightweight first, heavy later).
- Keep MCP server counts low per subagent. Two or three servers is a practical ceiling before tool overload degrades decision quality.

[OceanofPDF.com](https://oceanofpdf.com)

Orchestrator Design

Part II built subagents – specialized workers that handle focused tasks. Part III shifts to the design of multi-agent systems. This chapter covers the orchestrator: the agent that decomposes tasks, selects subagents, designs context, and integrates results. You’ll learn task decomposition principles, context passing strategies, three implementation patterns, and lessons from Anthropic’s own multi-agent research system.

Companion code: [ch06-orchestrator-design](#)

| What an Orchestrator Does

Recall the orchestrator-workers pattern from Chapter 2. A central LLM decomposes a task, delegates subtasks to workers, and synthesizes the results. In Claude Code, the orchestrator is either the main session itself or a custom agent launched with `claude --agent`.

An orchestrator has five responsibilities:

1. **Task decomposition:** Split the user’s request into subagent-sized units.
2. **Subagent selection:** Choose the right subagent for each subtask.
3. **Context design:** Decide what information each subagent receives.
4. **Result integration:** Combine subagent outputs into a coherent deliverable.
5. **Error handling:** Detect subagent failures and execute recovery strategies.

Why does orchestrator design matter? According to Anthropic Engineering Blog (2025-06), early agents made errors like spawning 50 subagents for simple queries and performing the exact same searches as other agents. Poor orchestrator design makes individual subagent quality irrelevant – the system fails at the coordination layer. Breaking this causes: wasted tokens, contradictory outputs, and tasks that take longer with multiple agents than with one.

Task Decomposition

Choosing the Right Granularity

Task granularity is the most consequential design decision in a multi-agent system.

Too coarse – the subagent’s context overflows:

```
You: Refactor this entire project  
  
Orchestrator: -> Delegates everything to 1 subagent
```

The subagent drowns in context. Quality degrades as the conversation grows. The multi-agent architecture adds overhead with zero benefit.

Too fine – startup costs dominate:

```
You: Refactor this entire project  
  
Orchestrator:  
-> Subagent 1: Change line 3 of auth.ts  
-> Subagent 2: Change line 7 of auth.ts  
-> Subagent 3: Change line 12 of auth.ts  
-> ... (100 subagents)
```

Each subagent has startup overhead. Cross-subagent consistency becomes impossible. The orchestrator spends more time coordinating than the subagents spend working.

Right-sized – one subagent per module or concern:

```
You: Refactor this entire project  
  
Orchestrator:  
-> Subagent 1: Refactor the auth module  
-> Subagent 2: Refactor the API routing layer  
-> Subagent 3: Refactor the database layer
```

According to Anthropic Engineering Blog (2025-06), their research system scales effort to query complexity with explicit rules:

- Simple fact-finding: 1 subagent, 3-10 tool calls.
- Comparative analysis: 2-4 subagents, 10-15 tool calls each.
- Complex research: 10+ subagents with clearly divided responsibilities.

Why embed scaling rules in prompts? LLMs are poor judges of appropriate effort. Without explicit guidelines, they overinvest in simple queries and underinvest in complex ones. Scaling rules in the orchestrator's prompt prevent both failure modes. Alternative: Let the model decide dynamically. This works with Opus-class models but is unreliable with smaller models.

Ensure Task Independence

Subtasks delegated to subagents should be as independent as possible:

```
Good decomposition (independent tasks):  
- Subagent 1: Add frontend tests  
- Subagent 2: Add backend tests  
- Subagent 3: Validate infrastructure config  
  
Bad decomposition (dependent tasks):  
- Subagent 1: Change the database schema  
- Subagent 2: Update the API (depends on #1)  
- Subagent 3: Update the UI (depends on #2)
```

Dependent tasks running in parallel produce inconsistent results. When dependencies exist, the orchestrator must sequence them: complete the schema change first, then delegate the API and UI updates with the new schema as context.

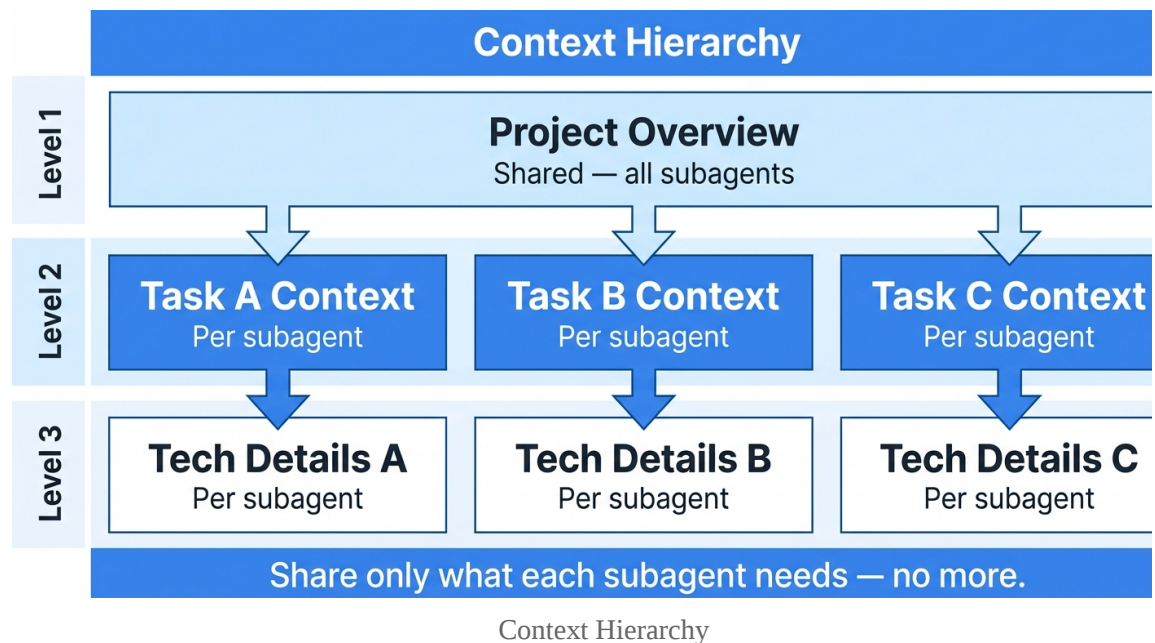
Decomposition Strategies

Four strategies cover most scenarios:

- **By module:** Split along architectural boundaries. Auth, API, database, UI each become a subtask. Best for refactoring and feature implementation.
- **By phase:** Split along workflow stages. Research, design, implement, test run sequentially. Best for greenfield work.

- **By file group:** Split large-scale changes into file batches. Best for migrations, bulk formatting, or dependency upgrades.
- **By concern:** Split by analytical lens. Security, performance, accessibility each become a subtask. Best for reviews and audits.

Context Passing



Designing the Subagent Prompt

The prompt the orchestrator passes to each subagent is the single most important factor in subagent success. According to Anthropic Engineering Blog (2025-06), effective subagent instructions include four elements:

1. **Objective:** What to achieve.
2. **Output format:** How to return results.
3. **Tool and source guidance:** Which tools and resources to use.
4. **Task boundaries:** What *not* to do.

Compare these two prompts:

Bad prompt:
"Research the semiconductor shortage"

Good prompt:
"Investigate the 2025 automotive semiconductor supply chain.
- Check production capacity for TSMC, Samsung, and Intel
- Quantify the impact on Toyota, VW, and GM
- Use web search; prioritize industry reports over blogs
- Return results as JSON: supplier, capacity, YoY change
- Stay under 3,000 tokens"

Why specify output format? Verbose subagent output wastes the orchestrator's context window. Constraints like "return JSON" or "5 bullet points maximum" control the token cost of integration. Without format constraints, subagents default to lengthy prose that the orchestrator must parse and compress.

Balancing Information Volume

Pass too much context and the subagent's window fills before it starts working. Pass too little and it wastes time rediscovering what the orchestrator already knows.

Too much:

Prompt: "Analyze all contents of the following files:
[10,000 lines of source code]
..."

The subagent has no room left for reasoning.

Too little:

Prompt: "Fix the bug"

The subagent spends its first 20 tool calls locating the problem the orchestrator already found.

Right-sized:

Prompt: "Fix the token validation bug in src/auth/jwt.ts, validateToken function (around line 42). Expired tokens are passing validation. Test file: tests/auth/jwt.test.ts. Error log: 'Token validation skipped for expired token'. Fix the bug and confirm tests pass."

File paths and line numbers eliminate the subagent's exploration phase. The orchestrator has already done the investigation – it should share those findings.

Context Hierarchy

For complex projects, structure context in layers:

```
Level 1: Project overview (shared across all subagents)
- Project purpose
- Tech stack
- Coding conventions

Level 2: Task-specific context (per subagent)
- Target file paths
- Related issue or PR numbers
- Expected behavior

Level 3: Technical details (per subagent)
- Error logs and stack traces
- Test results
- Dependency information
```

In custom subagent definitions, the system prompt (Markdown body) carries Level 1 context. The `prompt` parameter passed at invocation time carries Levels 2 and 3.

Implementation Patterns

Pattern 1: Main Session as Orchestrator

The simplest approach. The Claude Code main session acts as the orchestrator, delegating to subagents as needed:

```
[User] -> [Main session (orchestrator)]
      +-- Agent(code-reviewer)
      +-- Agent(test-writer)
      +-- Agent(doc-generator)
```

This requires no extra configuration. Claude Code's default behavior already includes subagent delegation when `.claude/agents/` definitions exist. The tradeoff is that you have no control over the orchestration logic – task decomposition, subagent selection, and context passing are all implicit.

Pattern 2: Custom Orchestrator with `claude --agent`

Launch a custom orchestrator with explicit delegation rules:

```
claude --agent coordinator
```

The `coordinator` subagent definition:

```
---
name: coordinator
description: Manage project-wide tasks and delegate
  to specialist subagents
tools: Agent(code-reviewer, test-writer, doc-generator),
  Read, Bash
model: opus
---

You are a project coordinator.

## Task Decomposition Rules

1. Analyze the user's instructions and identify subtasks
2. Delegate each subtask to the best subagent
3. Integrate subagent results and report to the user

## Subagent Selection Criteria

- code-reviewer: Code quality analysis, security reviews
- test-writer: Test code creation, coverage improvements
- doc-generator: Documentation generation, README updates

## Scaling Rules

- Simple tasks (1 file change): No subagents, handle
  it yourself
- Medium tasks (2-5 files): 1-2 subagents
- Complex tasks (6+ files): 3+ subagents, parallel
  execution
```

The `tools: Agent(code-reviewer, test-writer, doc-generator)` field restricts which subagents the orchestrator can spawn. Without this restriction, the orchestrator could create arbitrary subagents, reducing predictability.

Why restrict spawnable subagents? An unrestricted orchestrator might spawn 15 different subagent types for a simple task. `Agent(agent_type)` creates an explicit contract: this orchestrator delegates to these three specialists and no others. The system becomes predictable and auditable. Alternative: Omit restrictions and let the orchestrator decide. This trades predictability for flexibility – appropriate for research tasks but risky for production workflows.

Pattern 3: CLAUDE.md Delegation Policy

Write orchestration rules directly in `CLAUDE.md` to guide the main session's behavior:

```
## Multi-Agent Policy

Delegate the following tasks to the corresponding subagent:

- Code review -> code-reviewer subagent
- Test creation -> test-writer subagent
- Documentation updates -> doc-generator subagent

When multiple subagents are needed, delegate independent
tasks in parallel (run_in_background: true).
```

This approach works alongside subagent definitions. The `CLAUDE.md` policy is version-controlled, so every team member shares the same orchestration strategy. It is lighter than a full `--agent` orchestrator but less precise – the main session may not follow the policy perfectly.

Why use CLAUDE.md for orchestration? CLAUDE.md is loaded automatically at session start and checked into version control. Delegation policies written here become team conventions rather than individual configurations. New team members inherit the orchestration strategy without setup.

Lessons from Anthropic's Research System

Anthropic Engineering Blog (2025-06) describes their production multi-agent research system. Several design decisions from that system apply directly to orchestrator design.

Architecture

```
[User query]
|
[Lead agent (Opus)] <- orchestrator
+-- [Subagent 1 (Sonnet)] -> search -> analyze -> compress
+-- [Subagent 2 (Sonnet)] -> search -> analyze -> compress
+-- [Subagent 3 (Sonnet)] -> search -> analyze -> compress
|
[Lead agent integrates results]
|
[Final answer]
```

Three design choices stand out:

Model separation: The lead agent (orchestrator) runs on Opus. Subagents run on Sonnet. Orchestration requires high reasoning capability – task decomposition, quality judgment, and result synthesis are harder problems than individual searches. Sonnet is capable enough for the focused work.

Subagents as intelligent filters: Subagents search, analyze, and compress. They return only the most relevant tokens to the lead agent. This is the context isolation principle from Chapter 3, applied at the information level.

Parallel execution: 3-5 subagents run in parallel. Within each subagent, 3+ tools run in parallel. This cut research time by up to 90% for complex queries.

Token Economics

The same blog reports that token usage explains 80% of performance variance on the BrowseComp evaluation. Multi-agent systems succeed because they spend enough tokens to solve the problem.

The cost is real. Multi-agent systems consume significantly more tokens than single-agent sessions, and token usage explains 80% of performance variance. The orchestrator's job includes judging whether a task justifies that investment. A `git status` command does not need a subagent. A full codebase security audit does.

Why does cost awareness matter? Every unnecessary subagent delegation burns budget. The orchestrator must distinguish between “this is faster with a subagent” and “this is faster if I just do it.” Scaling rules (simple tasks: handle directly; complex tasks: delegate) encode this judgment.

Anti-Patterns

Over-delegation: Delegating trivial tasks to subagents.

```
You: Show me git status
Orchestrator: -> Delegates to subagent...
```

Subagent startup overhead makes this slower than running `git status` directly. The orchestrator should handle simple commands itself.

Context dumping: Passing the entire project to a subagent.

```
Prompt: "Read all project files and find problems"
```

If full-project analysis is needed, split it into file groups and delegate each group to a separate subagent. One subagent scanning thousands of files will produce shallow results.

Unverified results: Accepting subagent output without validation.

```
Subagent: "No issues found"
Orchestrator: "Everything looks good" (no verification)
```

A subagent reporting “no issues” may have searched the wrong files or used the wrong criteria. The orchestrator should spot-check results or cross-reference with a second subagent.

Sequential bottlenecks: Running independent tasks one after another.

```
Subagent 1 finishes -> Subagent 2 starts -> Subagent 3 starts
```

Independent tasks should run in parallel. Sequential execution is only necessary when tasks depend on each other. In practice, Claude Code parallelizes independent subagent calls automatically – the orchestrator’s prompt should reinforce this by specifying “delegate independent tasks in parallel.”

In the Agent SDK: The same orchestrator patterns apply. The lead agent calls `query()` to spawn subagents. Use `maxTurns` to limit subagent scope, and programmatic cost callbacks to enforce token budgets. Define available subagent types via the SDK’s `agents` parameter and include `Agent` in `allowedTools` to enable delegation – matching the CLI’s `tools: Agent(...)` field.

Common Failure Patterns

“Subagents duplicate each other’s work”: The orchestrator’s task decomposition is too vague. Each subtask description must clearly define boundaries – “investigate X but not Y” – so subagents don’t overlap.

“Results are inconsistent across subagents”: Dependent tasks are running in parallel. Identify the dependency chain and sequence the tasks.

“Orchestrator’s context fills up”: Subagent results are too verbose. Constrain output format in the delegation prompt: “return 5 bullet points” or “respond in JSON under 500 tokens.”

“Simple tasks take too long”: The orchestrator delegates everything. Add scaling rules to the prompt that distinguish simple (handle directly) from complex (delegate) tasks.

Summary

- An orchestrator decomposes tasks, selects subagents, designs context, integrates results, and handles errors.
- Task granularity determines system efficiency. Embed scaling rules in the orchestrator’s prompt to match effort to complexity.
- Subagent prompts need four elements: objective, output format, tool guidance, and task boundaries.
- Choose from three implementation patterns: main session (simplest), `claude -agent` (customizable), and `CLAUDE.md` policy (team-shared). Anthropic’s research system validates model separation (Opus orchestrator, Sonnet workers) and parallel execution (up to 90% time reduction).
- Avoid over-delegation, context dumping, unverified results, and sequential bottlenecks.

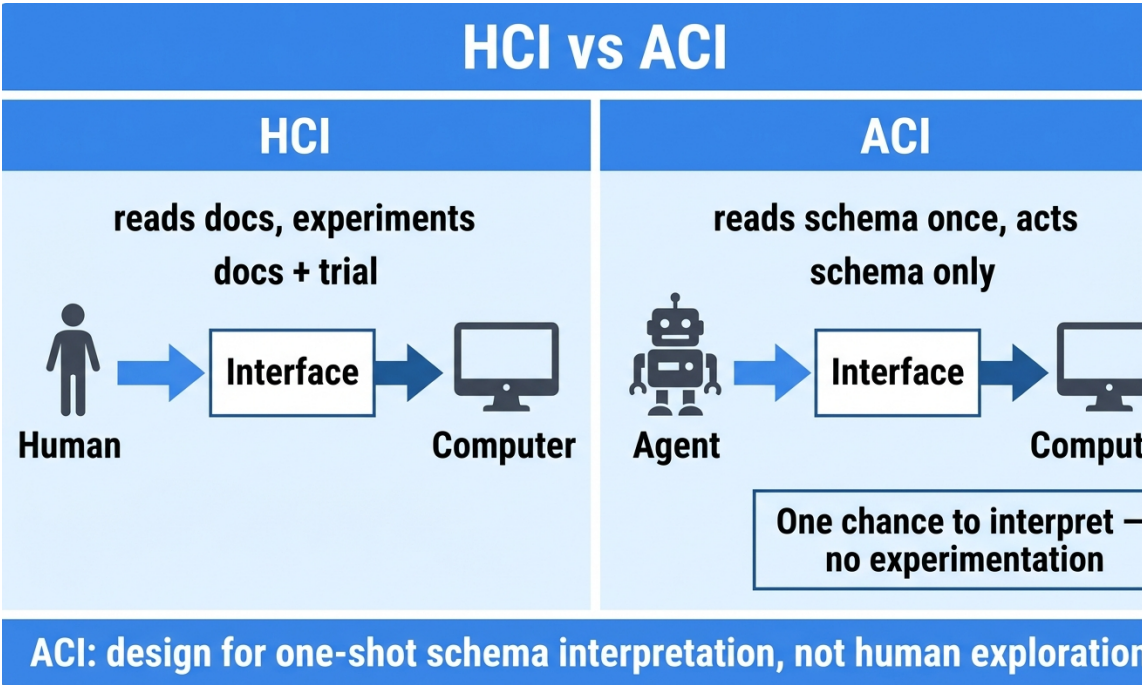
OceanofPDF.com

Designing Tools for Agents

Chapter 6 covered orchestrator design – how to decompose tasks, select subagents, and integrate results. This chapter shifts focus to the tools those agents use. You’ll learn the Agent-Computer Interface (ACI) concept and six principles for designing agent-friendly tools. You’ll also work through error response design and build a working MCP server that embodies all six principles.

Companion code: [ch07-tool-design-aci](#)

What Is ACI?



HCI vs ACI

Anthropic introduced ACI (Agent-Computer Interface) in “Building effective agents” (2024-12).

HCI (Human-Computer Interface) designs interfaces for humans. ACI designs interfaces for agents. The parallel is deliberate:

HCI: Human	<-> Interface <-> Computer
ACI: Agent	<-> Interface <-> Computer

Why does ACI matter? Agents rely entirely on tool names, descriptions, and parameter schemas to decide how to use a tool. That’s fundamentally different from human-facing API design, where a developer reads documentation, experiments in a REPL, and builds mental models over time. An agent gets one chance to interpret the schema – and acts on it immediately.

According to Anthropic Engineering Blog (2024-12), their SWE-bench implementation team “spent more time optimizing our tools than the overall prompt.” Tool design quality directly determines agent success rates.

Six Principles of Tool Design

These six principles are synthesized from two Anthropic Engineering Blog articles: “Writing effective tools for agents” (2025-09) and “Building effective agents” (2024-12).

Principle 1: Choose the Right Tools

More tools don’t mean better agents. According to Anthropic Engineering Blog (2025-09), tools that merely wrap existing API endpoints – without adapting them for agent affordances – lead to worse outcomes.

Agents differ from traditional software in a fundamental way:

```
Traditional software:
  getWeather("NYC") -> always fetches weather the same way

Agent:
  "Should I bring an umbrella today?"
  -> might use a weather tool
  -> might answer from general knowledge
  -> might ask for your location first
```

Agents are non-deterministic. The same input can trigger different tools in different orders. Tool design must account for this.

Example: contact search

```
Bad (agent-hostile):
  list_contacts() -> returns all contacts (thousands)
  -> agent's context fills up with contact data

Good (agent-friendly):
  search_contacts(name="Smith") -> returns matching contacts only
  -> agent's context contains just what it needs
```

A human can scroll through a database one row at a time. For an agent, “fetch everything then filter” wastes context window – a finite, precious resource.

Why do fewer tools win? According to Anthropic Engineering Blog (2025-09), “too many tools or overlapping tools cause the agent to deviate from efficient strategies.” Fewer tools with clear purposes improve decision accuracy.

Principle 2: Consolidate Functionality

Combine related operations into a single tool to simplify the agent’s workflow:

```
Bad (too granular):
  list_users()      -> user list
  list_events()     -> event list
  create_event()    -> create event

Good (consolidated):
  schedule_event(attendees=["Smith", "Jones"], date="2026-01-15")
  -> checks availability and creates event in one step
```

```
Bad (too granular):
  get_customer_by_id(id=123) -> customer info
  list_transactions(user=123) -> transaction history
  list_notes(user=123)       -> notes

Good (consolidated):
  get_customer_context(id=123)
  -> customer info, recent transactions, and notes in one call
```

Consolidation reduces tool call count and cuts the context consumed by intermediate results.

Principle 3: Namespace with Clear Boundaries

An agent may access hundreds of tools across dozens of MCP servers. Overlapping or ambiguous tool names cause confusion.

Namespacing groups tools with clear boundaries:


```
By service:
  asana_search, asana_create_task
  jira_search, jira_create_issue

By resource:
  asana_projects_search, asana_projects_create
  asana_users_search, asana_users_list
```

Claude Code automatically namespaces MCP tools as `mcp__<server_name>__<tool_name>`. Follow this convention in your own MCP servers. According to Anthropic Engineering Blog (2025-09), the choice between prefix-based and suffix-based namespacing has non-trivial effects on tool-use evaluations – test the approach that works best for your use case.

Principle 4: Return Meaningful Context

Tool responses should contain the information an agent needs to decide its next action:

```
Bad (too technical):
{
  "uuid": "550e8400-e29b-41d4-a716-446655440000",
  "256px_image_url": "https://...",
  "mime_type": "image/png",
  "created_at_epoch": 1704067200
}

Good (agent-useful):
{
  "name": "John Smith",
  "image_url": "https://...",
  "file_type": "png",
  "created_at": "2025-01-01T00:00:00Z"
}
```

According to Anthropic Engineering Blog (2025-09), replacing arbitrary alphanumeric UUIDs with meaningful language expressions or zero-based ID schemes significantly reduces hallucination in retrieval tasks.

Controlling response verbosity

Add a response format parameter so the agent can control how much detail it receives:

```
{
  "name": "search_customers",
  "parameters": {
    "query": { "type": "string" },
    "response_format": {
      "type": "string",
      "enum": ["concise", "detailed"],
      "default": "concise"
    }
  }
}
```

`concise` returns the minimum needed for the next action (names and IDs). `detailed` returns all fields. The agent uses `concise` for initial searches and switches to `detailed` only when drilling into a specific result.

Principle 5: Optimize Token Efficiency

Tool responses must not overwhelm the context window. Volume control matters as much as content quality.

Pagination:

```
{
  "name": "search_logs",
  "parameters": {
    "query": { "type": "string" },
    "limit": { "type": "integer", "default": 50 },
    "offset": { "type": "integer", "default": 0 }
  }
}
```

Filtering:

```
{
  "name": "list_issues",
  "parameters": {
    "status": { "enum": ["open", "closed", "in_progress"] },
    "assignee": { "type": "string" },
    "priority": { "enum": ["critical", "high", "medium", "low"] }
  }
}
```

Truncation:

When a response is too long, truncate it and tell the agent there's more:

```
{
  "results": ["item 1", "item 2", "..."],
  "total_count": 1523,
  "showing": 50,
  "message": "Showing 50 of 1523 results. Use 'offset' to fetch the next page."
}
```

At time of writing (Claude Code v2.1.32): Claude Code sets a default tool response limit of 25,000 tokens. Apply a similar limit in your own MCP servers. If you don't tell the agent that more data exists, it assumes the truncated response is the complete result – always include a truncation message.

Principle 6: Prompt-Engineer Tool Descriptions

A tool's `description` and parameter descriptions are the single most influential factor in how an agent uses that tool:

```
Bad (insufficient):
{
  "name": "search",
  "description": "Search"
}

Good (sufficient):
{
  "name": "search_knowledge_base",
  "description": "Search the internal knowledge base by keyword.
    Start with a short, broad query and narrow down based on results.
    Covers: technical docs, design docs, meeting notes.
    Does NOT cover: source code (use grep_code for code search)."
```

Anthropic recommends writing descriptions “as if explaining to a new team member.” Make implicit knowledge explicit: proprietary query formats, niche terminology definitions, relationships between resources.

Parameter names matter

```
Bad (ambiguous):
"user": { "type": "string" }
-> Username? User ID? Email address?

Good (unambiguous):
"user_id": { "type": "integer", "description": "Numeric user ID" }
```

According to Anthropic Engineering Blog (2025-09), Claude was needlessly appending `2025` to the web search tool's `query` parameter, biasing results – the team fixed it entirely by improving the tool description. Small description improvements drive large performance gains.

Error Response Design

Error responses must give the agent enough information to understand the problem and decide what to do next:

```
Bad (unhelpful):
{
  "error": "EINVAL",
  "code": 422
}

Good (actionable):
{
  "error": "Input validation error",
  "message": "Invalid format for 'date' parameter.
    Expected: YYYY-MM-DD (e.g. 2026-01-15).
    Received: '2026/1/15'"
}
```

Effective error responses contain three elements:

1. **What happened:** A plain-language description of the error
2. **Why it happened:** What was wrong with the input
3. **How to fix it:** A correct input example or alternative approach

Learning from Claude Code's Tool Design

Claude Code's own tools are a masterclass in ACI principles. A few design decisions worth analyzing:

Absolute Path Enforcement

According to Anthropic Engineering Blog (2024-12), during SWE-bench implementation the model made mistakes with tools using relative filepaths after the agent had moved out of the root directory. Switching the tool to always require absolute filepaths eliminated the problem entirely.

```
{
  "name": "Read",
  "parameters": {
    "file_path": {
      "type": "string",
      "description": "Absolute path to the file to read (relative paths not allowed)"
    }
  }
}
```

Why absolute paths? Agents – especially subagents – can’t always track the “current directory” accurately. Requiring absolute paths eliminates path resolution errors entirely.

Separating Edit and Write

Claude Code separates `Edit` (partial string replacement) from `Write` (overwrite the entire file):

```
Edit: old_string -> new_string (partial replacement)
Write: overwrite entire file with specified content
```

The `Edit` tool’s design constraints:

- `old_string` must be unique in the file – returns an error if not
- `new_string` must differ from `old_string`
- The tool description requires reading the file with `Read` first

These constraints prevent agents from writing without reading first or changing the wrong section.

Grep Tool Output Modes

```
{
  "name": "Grep",
  "parameters": {
    "output_mode": {
      "enum": ["content", "files_with_matches", "count"],
      "description": "content: show matching lines. files_with_matches: file paths
only. count: match counts only"
    },
    "head_limit": {
      "type": "number",
      "description": "Limit output to first N lines"
    }
  }
}
```

The agent starts with `files_with_matches` to identify target files, then switches to `content` for specific details. This progressive information retrieval enables efficient context use.

Building an MCP Server That Embodies ACI

Principles are theory until you write code. This section walks through a ~170-line knowledge base search server built with FastMCP (the official Python SDK). The complete implementation is in the companion repository at `ch07-tool-design-aci/server.py`.

The server exposes two tools: `search_articles` (keyword search with category filtering, pagination, and verbosity control) and `get_article_detail` (full article retrieval by ID). `search_articles` is the focal point where all six principles converge.

Setup

The server depends on two packages: `mcp[cli]` (the official Python SDK) and `pydantic`. With `uv`:

```
mkdir kb-server && cd kb-server
uv init
uv add "mcp[cli]>=1.12" "pydantic>=2.0"
```

Step 1: Define Types with Pydantic

Start by defining data and types. Article IDs use human-readable formats like `kb-deploy-002`, and search results support two modes: `concise` and `detailed`.

```

"""Internal Knowledge Base Search MCP Server"""
from typing import Literal
from pydantic import BaseModel, Field
from mcp.server.fastmcp import FastMCP

mcp = FastMCP("kb")

ARTICLES: dict[str, dict] = {
    "kb-deploy-002": {
        "title": "Production Deploy Approval Flow",
        "category": "engineering",
        "summary": "Steps from staging verification to production release",
        "body": "Production deploys require approval from at least two reviewers.",
        "updated_at": "2026-04-01",
    },
    "kb-incident-003": {
        "title": "Incident Response Playbook",
        "category": "engineering",
        "summary": "First-response actions and contact list for P0/P1 incidents",
        "body": "On a P0 incident, immediately join #incident-war-room.",
        "updated_at": "2026-03-28",
    },
}

Category = Literal["hr", "engineering", "finance", "sales"]
ResponseFormat = Literal["concise", "detailed"]

class ArticleSummary(BaseModel):
    """Minimal article info included in search results."""
    article_id: str = Field(description="Article ID (e.g. kb-deploy-002)")
    title: str = Field(description="Article title")
    category: str = Field(description="Category name")
    updated_at: str = Field(description="Last updated date (YYYY-MM-DD)")

class ArticleDetail(ArticleSummary):
    """Full article fields (detailed mode or single-article fetch)."""
    summary: str = Field(description="One- to two-sentence summary")
    body: str = Field(description="Full body text (Markdown)")

class SearchResult(BaseModel):
    """Top-level structure for search responses."""
    results: list[ArticleSummary | ArticleDetail]
    total_count: int = Field(description="Total matches for the filter criteria")
    showing: int = Field(description="Number of results in this response")
    next_offset: int | None = Field(
        description="Offset for the next page, or null if no more pages"
    )

```


Why use Pydantic's `Field(description=...)`? FastMCP generates JSON schemas automatically from Pydantic models. Each field's description is passed directly to the agent. The agent reads field names and descriptions to decide “what to fetch next.” `temperature: float = Field(description="Temperature in Celsius")` gives the agent far more signal than bare `temperature: float`.

Defining `category` as a `Literal` type matters too. FastMCP converts it to an `enum` schema, telling the agent “this parameter accepts only these four values.” This serves both Principle 3 (clear boundaries) and Principle 6 (prompt-engineering descriptions).

Step 2: Build the Search Tool Around All Six Principles

`search_articles` is the core tool. All six principles converge in one function:

```

@mcp.tool()
def search_articles(
    query: str,
    category: Category | None = None,
    response_format: ResponseFormat = "concise",
    limit: int = 20,
    offset: int = 0,
) -> SearchResult:
    """Search the internal knowledge base by keyword.

    Start with a short, broad query, then narrow down based on results.
    Covers: process docs, design docs, runbooks, HR policies.
    Does NOT cover: source code (use the code search tool instead).

    Usage hints:
    - query: one or more keywords separated by spaces
    - category: specify only when narrowing to a specific domain;
      omit to search all categories
    - response_format: use 'concise' first to scan result counts,
      then fetch 'detailed' only for articles of interest --
      avoid requesting 'detailed' for every search
    - limit / offset: page through results until 'next_offset' is null
    """
    if not query.strip():
        raise ValueError(
            "query must not be empty. Provide at least one search keyword."
        )
    if limit < 1 or limit > 100:
        raise ValueError("limit must be between 1 and 100.")
    if offset < 0:
        raise ValueError("offset must be 0 or greater.")

    terms = query.lower().split()
    matches = [
        (aid, a) for aid, a in ARTICLES.items()
        if (category is None or a["category"] == category)
        and all(
            t in a["title"].lower()
            or t in a["summary"].lower()
            or t in a["body"].lower()
            for t in terms
        )
    ]
    total = len(matches)
    page = matches[offset : offset + limit]

    items: list[ArticleSummary | ArticleDetail] = []
    for aid, a in page:
        if response_format == "detailed":
            items.append(ArticleDetail(article_id=aid, **a))
        else:
            items.append(ArticleSummary(
                article_id=aid,
                title=a["title"],
                category=a["category"],
                updated_at=a["updated_at"],
            ))

```

```

next_offset = offset + limit if offset + limit < total else None
return SearchResult(
    results=items,
    total_count=total,
    showing=len(items),
    next_offset=next_offset,
)

```

Here's how each principle maps to this function:

Principle 1 (Choose the right tools): There is no `list_all_articles()`. Only `search_articles` is exposed, structurally preventing the agent from loading every article into context.

Principle 2 (Consolidate functionality): Keyword search, category filtering, pagination, and verbosity control live in one tool. Splitting them into four separate tools would force four round trips – “filter by category, then search by keyword, then paginate” – instead of one.

Principle 5 (Optimize token efficiency): `response_format` defaults to `concise`, returning only article ID, title, category, and updated date. `limit/offset` controls page size. The response always includes `total_count` and `next_offset`, so the agent knows more data exists – without that signal, it assumes the truncated result is complete.

Principle 6 (Prompt-engineer descriptions): The entire docstring becomes the tool description – that's how FastMCP works. Write it like onboarding a new team member: what the tool does, what it doesn't cover, and hints for each parameter. The line `response_format: use 'concise' first... avoid requesting 'detailed' for every search` is particularly effective. It steers the agent away from fetching full details on every query.

The error messages follow the three-element pattern too. `raise ValueError("query must not be empty. Provide at least one search keyword.")` tells the agent what went wrong and how to fix it.

Step 3: Detail Retrieval and Error Design

`get_article_detail` is a simple lookup, but its error design matters:

```

@mcp.tool()
def get_article_detail(article_id: str) -> ArticleDetail:
    """Retrieve the full text of an article by its ID.

    Typically called after obtaining article_id from search_articles.
    Returns a ValueError with correction hints for unknown IDs.
    """
    if article_id not in ARTICLES:
        examples = ", ".join(list(ARTICLES.keys())[:3])
        raise ValueError(
            f"article_id '{article_id}' does not exist. "
            f"Use search_articles to find the correct ID. "
            f"Examples: {examples}"
        )
    a = ARTICLES[article_id]
    return ArticleDetail(article_id=article_id, **a)

if __name__ == "__main__":
    mcp.run(transport="stdio")

```

Why include examples in error messages? The agent learns “what correct input looks like” from error messages. `article_id 'does-not-exist'` does not exist alone doesn’t give the agent a next step. Adding `Examples: kb-onboarding-001, kb-deploy-002, kb-incident-003` lets the agent infer the format (`kb-` prefix + descriptive slug) and decide to call `search_articles` to find the right ID.

Register with Claude Code and Verify

Register the server with `claude mcp add`. Everything after `--` is the startup command:

```
claude mcp add kb --transport stdio -- uv run /absolute/path/to/server.py
```

Replace `/absolute/path/to/server.py` with the actual path. Following the principles in this chapter, avoid relative paths.

After registration, run `/mcp` inside Claude Code to verify the `kb` server status. Ask a question and watch the agent invoke `mcp__kb__search_articles`:

You: Show me the deploy procedure

```
Claude: Searching the kb server...
-> mcp__kb__search_articles(query="deploy")
-> 1 match found (kb-deploy-002: Production Deploy Approval Flow)
Fetching details...
-> mcp__kb__get_article_detail(article_id="kb-deploy-002")
Production deploys require approval from at least two reviewers...
```

Notice how the tool name appears as `mcp__kb__search_articles`. Claude Code automatically applies the `mcp__<registration_name>__<tool_name>` namespace (Principle 3). The name you pass to `claude mcp add` becomes the namespace.

Principles-to-Code Mapping

Principle	How server.py implements it
1. Choose the right tools	No <code>list_articles</code> dump – only <code>search_articles</code> with filtering
2. Consolidate functionality	Keyword, category, pagination, verbosity in one tool
3. Namespace with clear boundaries	Registered as <code>kb</code> – tools appear as <code>mcp__kb__search_articles</code>
4. Return meaningful context	Human-readable IDs like <code>kb-deploy-002</code> instead of UUIDs
5. Optimize token efficiency	<code>response_format</code> (concise/detailed) plus <code>limit/offset</code>
6. Prompt-engineer descriptions	Docstring includes use cases, exclusions, and parameter hints

Swap the in-memory dataset for a database or vector search, and you have a production knowledge base agent. The point of this exercise is to move the principles from “something you read” to “something you write.”

Tool Design Anti-Patterns

Anti-Pattern 1: Full Data Dump

```
Bad: list_all_users() -> returns every user
Good: search_users(query, limit=20) -> returns matching users
```

Anti-Pattern 2: Hidden Side Effects

```
Bad: update_user(id, data) -> updates user AND sends notification email
Good: update_user(id, data) -> updates user only
Good: send_notification(user_id, type) -> sends notification
```

Hidden side effects cause agents to trigger unintended operations.

Anti-Pattern 3: Ambiguous Parameters

```
Bad: search(q="something", type="all")
    -> what does "type" mean?

Good: search_documents(
    query="something",
    document_type="technical_spec",
    date_range="last_30_days"
)
```

Anti-Pattern 4: Bloated Responses

```
Bad: get_project() -> all project data (members, tasks, files, history...)
Good: get_project_summary() -> summary only
Good: get_project_members(project_id) -> members only
Good: get_project_tasks(project_id, status="open") -> tasks only
```

Anti-Pattern 5: Inconsistent Formats

```
Bad: one tool returns XML, another returns JSON

Good: all tools return consistent JSON
```

Why does format consistency matter? According to Anthropic Engineering Blog (2025-09), “the choice of response structure affects performance.” Use a consistent format across all tools in the same MCP server.

Tool Evaluation

According to Anthropic Engineering Blog (2025-09), systematically evaluating tool quality is essential – ad-hoc testing misses edge cases that only emerge at scale.

Step 1: Create Evaluation Tasks

Build evaluation tasks from real use cases:

```
Bad (weak evaluation):  
"Schedule a meeting with Jane next week"  
  
Good (strong evaluation):  
"Schedule a progress meeting with Jane for the Acme project next week.  
Attach the notes from our last project planning meeting  
and reserve a conference room."
```

Strong evaluation tasks require multiple tool calls and reflect real-world complexity.

Step 2: Collect Metrics

- **Accuracy:** Did the task complete correctly?
- **Tool call count:** Did the agent use tools efficiently?
- **Token consumption:** Did it waste context?
- **Error rate:** Were there invalid parameter calls?
- **Execution time:** Per-tool and end-to-end timing

Step 3: Collaborate with the Agent to Improve

Anthropic Engineering Blog (2025-09) describes an approach where Claude itself improves the tools:

1. Present the tool and its failure cases to Claude.
2. Claude diagnoses the problem and suggests improvements.

3. Re-evaluate with the improved tool.

According to Anthropic Engineering Blog (2025-09), even small refinements to tool descriptions can yield dramatic improvements – Anthropic used this iterative process to achieve state-of-the-art performance on SWE-bench Verified.

Why let the agent improve tools? The entity best positioned to identify “what’s hard to use” about a tool is the agent that actually uses it. Running dozens of test iterations surfaces edge cases that human designers miss.

MCP Server Development Checklist

Use this checklist when building MCP servers for subagents:

Tool definitions:

- Tool name clearly conveys its purpose
- `description` includes use cases, exclusions, and query hints
- Parameter names are unambiguous (`user_id` , not `user`)
- Required vs. optional parameters are explicit
- `enum` is used for parameters with a fixed set of values

Response design:

- Response fields use meaningful names
- Human-readable identifiers preferred over technical IDs
- Pagination or `limit` parameter implemented
- Truncation handling for oversized responses
- `response_format` parameter (concise/detailed) considered

Error handling:

- Error messages describe the problem in plain language
- Error messages include fix instructions or correct input examples
- Raw error codes and stack traces are never exposed

Safety:

- Read-only and write operations are separated into distinct tools

- Destructive operations (delete, update) have tool annotations
- Credentials are managed through environment variables

Summary

- ACI (Agent-Computer Interface) is interface design for agents – it deserves the same design investment as HCI.
- Fewer tools with clear purposes outperform large catalogs. Consolidate related operations into single tools and namespace them to prevent confusion across multiple MCP servers.
- Return meaningful context efficiently: use human-readable identifiers, implement pagination, and add a `response_format` parameter (concise/detailed) to control verbosity.
- Write tool descriptions as if onboarding a new team member – include use cases, exclusions, and parameter hints. Error responses must cover what happened, why, and how to fix it.
- Evaluate tool quality with real-world tasks and collaborate with agents to iterate: the agent that uses a tool is best positioned to identify where it falls short.

Writing Effective Agent Prompts

Chapters 6 and 7 covered orchestrator design and tool design. This chapter addresses the third pillar of multi-agent systems: prompt design. You'll learn eight principles for writing effective agent prompts, drawn from Anthropic's multi-agent research system, plus the concept of "context engineering" and how to encode these principles in `CLAUDE.md` and subagent definitions.

Companion code: [ch08-prompt-principles](#)

From Prompt Engineering to Context Engineering

Anthropic Engineering Blog (2025-09), in "Effective context engineering for AI agents," reframes how to think about agent instructions.

Traditional "prompt engineering" optimized instructions for a single LLM call. Agents, however, reason across multiple turns, use tools, and fetch new information. The optimization target isn't just "the prompt" – it's everything that enters the agent's context window:

```
Prompt engineering:
  System prompt -> optimize

Context engineering:
  System prompt + tool definitions + MCP data
  + message history + tool results -> optimize all of it
```

Why does this distinction matter? Anthropic describes "context rot" – a phenomenon where information recall accuracy degrades as token count increases. Think of context as an "attention budget." Every token in the window consumes part of that budget. The goal of context engineering is to find the smallest possible set of high-signal tokens that produces the desired outcome.

Principle 1: Think Like the Agent

Improving prompts requires understanding how the agent actually behaves – not how you assume it behaves.

Anthropic's research team built simulations using the agent's real prompts and tools, then observed agent behavior step by step. This observation revealed failure modes that were invisible without traces:

- Continuing to search after finding sufficient results
- Using verbose, overly specific search queries
- Selecting inappropriate tools

Bad (improving by guesswork):

"Search more concisely" -> unclear effect

Good (improving from observation):

1. Read the agent's trace
2. Identify "search queries are too long, yielding few results"
3. Instruct "start with short, broad queries and narrow down based on results"

Applying This in Claude Code

Several approaches let you observe subagent behavior in Claude Code:

1. Use the `--verbose` flag for detailed logs.
2. Add “output your reasoning at each step” to the subagent’s prompt.
3. Test with a debug-focused subagent on small tasks.

```
---
name: debug-researcher
description: Reports findings and reasoning process in detail (for debugging)
model: sonnet
---

## Output Rules

At each step, output the following:
1. Current objective
2. The tool you chose and why
3. Evaluation of the tool's result
4. Rationale for your next action
```

Why does observation matter? Problems like “search queries are too long” or “the agent skips its tool inventory” only surface in actual traces. According to Anthropic, “understanding prompt effectiveness requires an accurate mental model of the agent.”

Principle 2: Teach Orchestrators How to Delegate

Delegation instructions are the most common point of failure in multi-agent systems. When an orchestrator hands off a task, the subagent needs four pieces of information:

- **Goal:** What to accomplish. Missing this leads to task misinterpretation.
- **Output format:** How to return results. Missing this produces unparseable output.
- **Tool guidance:** Which tools to use. Missing this causes inefficient tool selection.
- **Task boundaries:** What NOT to do. Missing this causes scope creep.

Short, ambiguous instructions cause subagents to interpret tasks differently. In Anthropic's research system, early instructions like "research semiconductor shortages" sent one subagent investigating the 2021 automotive chip crisis while two others redundantly researched 2025 supply chains. Clear delegation templates eliminated this problem.

Implementing in Claude Code

Encode delegation rules in `CLAUDE.md`:

Delegation Rules for Subagents

When delegating to subagents, follow this template:

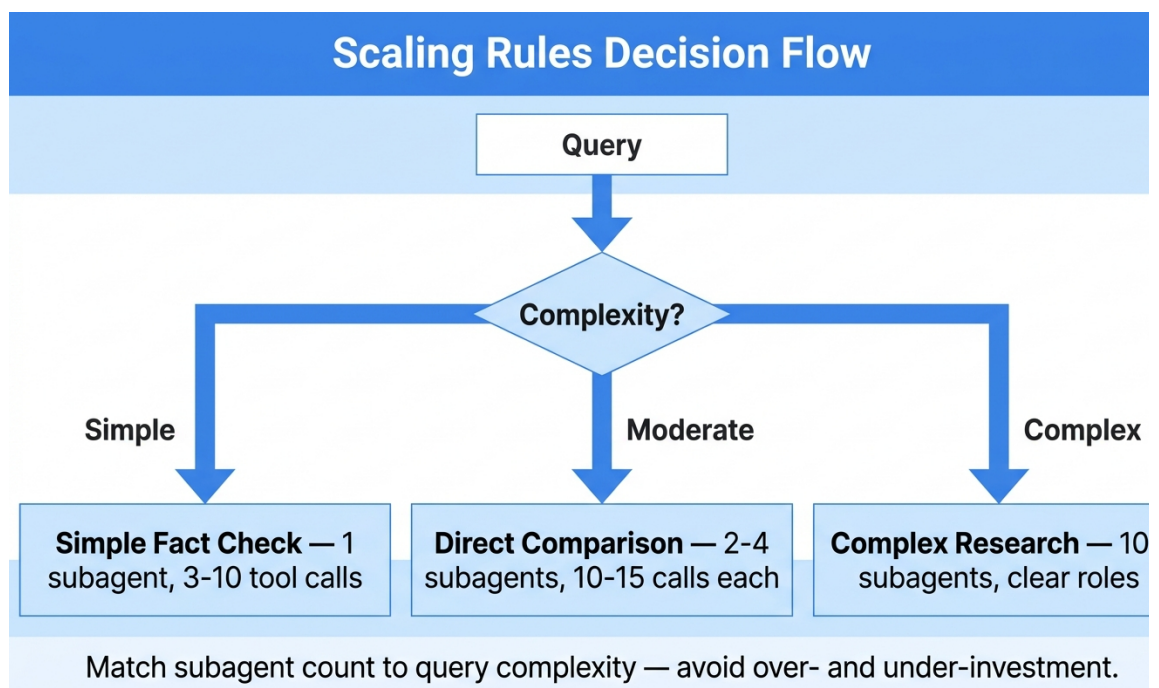
Required Information

1. **Goal**: State concretely in 1-2 sentences
(e.g., "Scan the auth module for SQL injection vulnerabilities")
2. **Output format**: Structure of the result
(e.g., "JSON with vulnerability type, file, line number, and severity")
3. **Tool guidance**: Which tools and resources to use
(e.g., "Search source code with Grep, then verify files with Read")
4. **Boundaries**: What is out of scope
(e.g., "Detection and reporting only -- do not apply fixes")

Prohibited

- Vague instructions like "investigate this"
- Listing file paths without context
- Delegating without specifying the output format

Principle 3: Match Effort to Query Complexity



Scaling Rules Decision Flow

Agents are poor at calibrating effort to task complexity. They either over-invest in simple questions or underestimate complex problems.

Anthropic's research system embeds explicit scaling rules in the prompt:

Scaling Rules

- ****Simple fact check****: 1 subagent, 3-10 tool calls
Example: "What is the time complexity of Python's len()?"
- ****Direct comparison****: 2-4 subagents, 10-15 tool calls each
Example: "Compare React vs Vue performance"
- ****Complex research****: 10+ subagents, clear role assignments
Example: "Draft a migration plan to microservices"

Implementing in Claude Code

Include scaling rules in a custom orchestrator's system prompt:

```
---
name: task-coordinator
description: Allocates resources based on task complexity
tools: Agent(researcher, implementer, reviewer), Read, Bash
---

## Resource Allocation Rules

When you receive a task, first assess its complexity.

### Level 1: Handle Yourself (No Subagents Needed)
- Single file change
- Simple command execution
- Answering with known information

### Level 2: One Subagent
- 2-5 file changes
- Limited investigation required
- Single domain of expertise

### Level 3: 2-3 Subagents (Parallel)
- 6+ file changes
- Investigation across multiple domains
- Concurrent testing and implementation

State your complexity assessment and its rationale before starting work.
```

Principle 4: Prioritize Tool Design and Selection

Give agents explicit heuristics for choosing tools – tool selection failures are as damaging as poor tool design.

Anthropic's research system provides agents with clear tool selection guidelines:

Tool Selection Guidelines

1. Review all available tools before starting work
2. Select tools based on the user's intent
3. Use web search for broad external exploration
4. Prefer specialized tools over general-purpose ones
5. Read tool descriptions and choose the best match for the objective

Why are explicit guidelines necessary? When agents access many tools via MCP, poor descriptions mislead them. Without guidance, agents make fundamental errors – like searching the web for information that exists only in Slack.

Principle 5: Let Agents Improve Themselves

Anthropic uses Claude itself as a “prompt engineering proxy.”

The approach: give an agent a prompt and its failure cases, ask it to diagnose why the failures happened, and generate improvements. They also created tool-testing agents that exercised MCP tools dozens of times, surfacing description problems and bugs.

According to Anthropic Engineering Blog (2025-06), this process cut future agent task completion time by 40% through tool description improvements alone.

Applying This in Claude Code

Build a dedicated subagent for improving other subagents’ prompts:

```
---
name: prompt-improver
description: Analyzes subagent prompts and suggests improvements
tools: Read, Glob, Grep
model: opus
---
```

You are a prompt engineering specialist.

Procedure

1. Read the specified subagent definition file
2. Analyze from these perspectives:
 - Is the description clear and specific?
 - Are the tool selections appropriate?
 - Does the system prompt contain all necessary elements?
 - Are failure modes anticipated?
3. Present improvement suggestions as concrete code diffs

Principle 6: Start Broad, Then Narrow

Search strategy should mimic how expert human researchers work. Agents default to long, specific queries that yield few results.

Search Strategy

1. Start with short, broad queries to survey the landscape
2. Evaluate the available information
3. Progressively narrow focus

Bad (too specific from the start):

```
"TypeScript React useEffect cleanup function memory leak async"
```

Good (progressive narrowing):

```
Step 1: "React useEffect cleanup"
```

```
Step 2: "useEffect async memory leak"
```

```
Step 3: (focus on a specific pattern)
```

Code search benefits from the same approach. Instruct subagents to start with Glob to understand file structure, then use Grep for targeted code search. Progressive exploration consistently outperforms a single precise query.

Principle 7: Guide the Thinking Process

Anthropic's research system uses Extended Thinking extensively.

The lead agent plans in thinking mode:

- Which tools fit the task
- Query complexity and subagent count
- Each subagent's role

Subagents also use Interleaved Thinking to evaluate tool results, identify gaps, and refine subsequent queries.

Applying This in Claude Code

For Claude Code subagents, the instruction “plan before you act” is effective:

```
---
name: methodical-researcher
description: Plans before investigating, then researches systematically
---

## Procedure

1. Planning Phase (always execute first)
   - Restate the task objective in your own words
   - List the information sources you need
   - Write out investigation steps as a numbered list

2. Execution Phase
   - Follow the plan one step at a time
   - After each tool result, evaluate:
     - Is this information sufficient for the objective?
     - Is additional investigation needed?
     - Does the plan need revision?

3. Reporting Phase
   - Present findings in a structured format
   - Explicitly flag uncertain information
```

Why does guiding the thinking process work? Prompting an LLM to “think before acting” triggers Chain-of-Thought reasoning, improving accuracy. According to Anthropic Engineering Blog (2025-03), the “think tool” – which inserts a thinking step during tool-use sequences – significantly improved performance on complex multi-step tasks. In their benchmarks, it achieved a 54% relative improvement on one domain.

Principle 8: Transform Speed with Parallel Tool Calls

Complex research tasks require exploring many sources. Anthropic’s research system introduced two levels of parallelism:

- 1. The lead agent launches 3-5 subagents in parallel.**
- 2. Each subagent calls 3+ tools in parallel.**

This change “cut complex query research time by up to 90%,” according to Anthropic Engineering Blog (2025-06).

Implementing in Claude Code

Claude Code automatically parallelizes `Agent` tool calls when the orchestrator detects independent subtasks. To make parallelization explicit in your prompts:

```
## Parallel Execution Rules

Delegate independent tasks in parallel, launching multiple subagents simultaneously.
Sequential execution is only required when tasks have data dependencies.

### Parallel execution examples
- Run frontend and backend tests simultaneously
- Apply independent fixes to multiple files
- Run security review and performance review concurrently

### Sequential execution required
- Database schema change -> API change
- Library installation -> build verification
```

The Right Altitude

According to Anthropic Engineering Blog (2025-09), prompts have a “right altitude” – a balance between specificity and flexibility. Prompts fail in two directions:

Too low (overly specific):

```
Bad:
"If the file ends in .ts and the import includes React
and export default exists and useState is used,
classify as a component file; otherwise..."
```

This kind of prompt is brittle and can’t handle unexpected cases.

Too high (overly vague):

```
Bad:
"Improve the code"
```

This gives the model no concrete signal and assumes shared context that doesn’t exist.

The right altitude:

Good:

"When reviewing code, report issues in this priority order:

1. Security vulnerabilities (input validation, authentication)
2. Performance problems ($O(n^2)$ or worse algorithms, $N+1$ queries)
3. Maintainability (functions over 100 lines, unclear naming)

Include file path, line number, and suggested fix for each issue."

The right altitude is specific enough to direct behavior, yet flexible enough to handle varied inputs.

Encoding Principles in CLAUDE.md

The eight principles are only useful if your whole team follows them. `CLAUDE.md` is the right place to encode them – it loads automatically for every session in your project.

Project-Level CLAUDE.md

```
# CLAUDE.md

## Multi-Agent Policy

### Delegation Criteria
- 1 file change: handle yourself
- 2-5 files: 1 subagent
- 6+ files: multiple subagents in parallel

### Required Information When Delegating
1. Concrete goal (1-2 sentences)
2. Output format (JSON / Markdown / bullet list)
3. Which tools to use
4. Work that is out of scope

### Search Strategy
- Start with a broad search, then narrow based on results
- Glob to understand file structure -> Grep for specific code search

### Quality Standards
- Always verify subagent results
- Double-check "no issues found" reports from a different angle
- Never report uncertain information as fact
```

Embedding Principles in Subagent Definitions

```
---
name: code-reviewer
description: Reviews code for quality, security, and performance.
  Use automatically after code changes.
tools: Read, Grep, Glob
model: sonnet
```

```

---

## Review Procedure (Principle 7: Guide the Thinking Process)

1. First, understand the overall file structure (Glob)
2. Identify the changed sections (Grep)
3. Analyze from these perspectives:
  - Security: input validation, authentication, SQL injection
  - Performance: N+1 queries, unnecessary loops, memory leaks
  - Maintainability: naming, function length, separation of concerns

## Output Format (Principle 2: Teach Orchestrators How to Delegate)

{
  "summary": "Overall assessment (1-2 sentences)",
  "issues": [
    {
      "type": "security|performance|maintainability",
      "severity": "critical|warning|info",
      "file": "file path",
      "line": "line number",
      "description": "Description of the issue",
      "suggestion": "Suggested fix"
    }
  ]
}

## Constraints (Principle 3: Match Effort to Complexity)

- Review up to 10 files maximum
- Limit analysis to 5 minutes per file
- Detection and reporting only -- do not apply fixes

```

Notice how each section traces back to a specific principle. This makes the subagent definition self-documenting – anyone reading it understands not just what the agent does, but why each instruction exists.

Summary

- Observe actual agent traces before changing prompts – guessing about failures rarely leads to useful fixes (Principle 1).
- Delegation instructions must specify goal, output format, tool guidance, and task boundaries; missing any one causes predictable failures (Principles 2, 3).
- Include tool selection heuristics and search-strategy guidance directly in prompts – agents default to poor choices without explicit direction (Principles 4, 6).
- Use agents to improve other agents: give a prompt-improver subagent a definition and its failure cases, and let it generate improvements (Principle

5).

- Guide thinking with “plan first, then execute” instructions; make parallelization explicit – parallel subagent launches cut research time by up to 90% (Principles 7, 8). Encode all policies in `CLAUDE.md` for team-wide consistency.

[OceanofPDF.com](https://oceanofpdf.com)

Managing Long-Running Agent Sessions

Part III covered design principles – orchestrator architecture, tool design, and prompt engineering. Part IV tackles a harder problem: making agents work beyond a single session. This chapter focuses on harness design and memory management for agents that run across multiple context windows.

The primary source is Anthropic Engineering Blog (2025-11), “Effective harnesses for long-running agents.”

Companion code: [ch09-long-running-agent](#)

| The Long-Running Agent Problem

Complex tasks don’t fit in a single context window. A production web app, a compiler, a large refactoring – these require hours or days of agent work. Each new session starts with a clean slate: no memory of what the previous session accomplished.

Anthropic Engineering Blog (2025-11) describes this as “engineers working in shifts, where each new engineer arrives with no memory of what happened on the previous shift.”

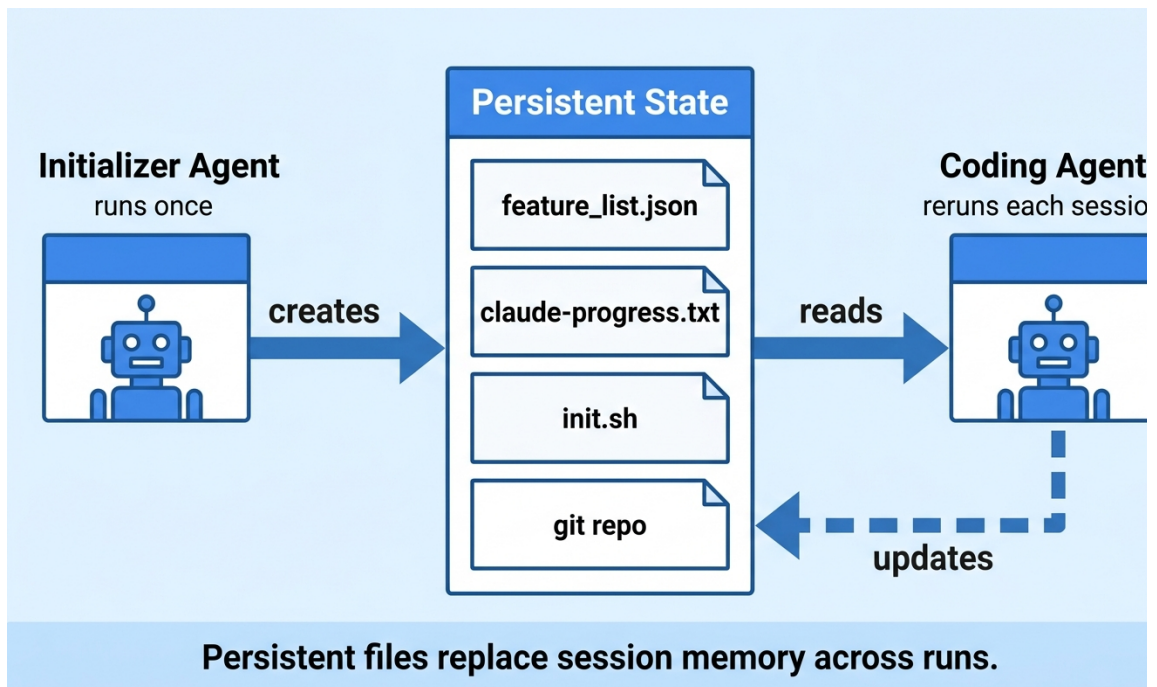
Two failure patterns emerge consistently.

Failure pattern 1: one-shotting. The agent tries to build the entire project in a single session. It runs out of context mid-implementation, leaving the next session to guess what happened and clean up half-finished code.

Failure pattern 2: premature completion. After some features are built, the agent looks around, sees progress, and declares the job done. Many features remain unimplemented.

Why these patterns matter Both failures stem from the same root cause: the agent lacks a structured way to understand what's been done and what remains. Without explicit tracking, the agent falls back on heuristics that fail at scale.

Harness Architecture



Harness Architecture

Anthropic's solution has two parts: an **initializer agent** that sets up the environment on the first run, and a **coding agent** that makes incremental progress in every subsequent session.

The Initializer Agent

The initializer runs once, in the first session. It creates four things:

1. **Feature list:** A JSON file expanding the user's prompt into specific, testable features. All start as `passes: false`.
2. **Progress file:** A log where each session records what it accomplished and what comes next.
3. **Initialization script:** A shell script that sets up the development environment.

4. **Initial commit:** A Git commit recording the starting state, enabling rollback.

```
[Initializer Agent]
+-- feature_list.json    <-- all features marked passes: false
+-- claude-progress.txt  <-- progress log (empty)
+-- init.sh              <-- environment setup script
+-- git commit           <-- records the starting state
```

Feature list design

In Anthropic's experiment building a claude.ai clone, the initializer generated over 200 features. Each feature is a JSON object:

```
{
  "category": "functional",
  "description": "New chat button creates a fresh conversation",
  "steps": [
    "Navigate to main interface",
    "Click the 'New Chat' button",
    "Verify a new conversation is created",
    "Check that chat area shows welcome state",
    "Verify conversation appears in sidebar"
  ],
  "passes": false
}
```

Why JSON over Markdown? Anthropic found that agents are less likely to inappropriately modify JSON files compared to Markdown. The structured format makes it harder for an agent to accidentally delete or rewrite features. Only the `passes` field should change.

The Coding Agent

Every session after initialization uses the coding agent. It follows a strict sequence:

1. Run `pwd` to confirm the working directory.
2. Read the Git log and progress file to understand recent work.
3. Read the feature list and pick the highest-priority incomplete feature.
4. Start the dev server and run a basic smoke test (verify existing features work).
5. Implement one feature.
6. Test the feature.

7. Commit to Git and update the progress file.

The critical constraint: **one feature per session**. This eliminates the one-shotting failure pattern. Requiring a clean commit at session end eliminates the “half-finished code” problem.

Building a Harness with Five Scripts

The harness architecture translates into five concrete scripts. Each script handles a single responsibility – applying the ACI principle from Chapter 7 (“each tool does one job”) to the harness itself.

After initialization, the project directory looks like this:

```
my-long-task/
+-- features.txt           (human-written feature list)
+-- feature_list.json      (generated by generate_features.py)
+-- claude-progress.txt    (appended by update_progress.sh)
+-- AGENT_PROMPT.md       (generated by init.sh)
+-- current_tasks/        (task lock directory)
+-- agent_logs/           (log output for long-run.sh)
+-- init.sh               (initialization agent)
+-- generate_features.py   (features.txt -> feature_list.json)
+-- next_feature.py        (returns next incomplete feature)
+-- mark_passed.py         (sets passes to true)
+-- update_progress.sh     (appends to progress log)
```

Step 1: Define the features.txt Schema

Humans write features in a flat, Markdown-like format. Agents can parse it but are unlikely to corrupt it:

```
# functional: User can log in
- Enter email and password
- Click the login button
- Verify redirect to dashboard

# functional: User can log out
- Click the logout button
- Verify the session is destroyed
```

Each `# <category>: <description>` starts a feature. Lines starting with `-` are steps. Categories are free-form: `functional`, `ui`, `api`, `perf`, or whatever fits your project.

Step 2: Generate JSON with `generate_features.py`

This script parses `features.txt` and outputs `feature_list.json`. Regenerating preserves existing `passes: true` flags, so you can edit `features.txt` without losing progress:

```

#!/usr/bin/env python3
"""Generate feature_list.json from features.txt."""
import json, re, sys
from pathlib import Path

FEATURE_HEAD = re.compile(r"^#\s*([^\:]+):\s*(.+)$")
STEP = re.compile(r"^\s*(.+)$")

def parse_features(text: str) -> list[dict]:
    features: list[dict] = []
    current: dict | None = None
    for raw in text.splitlines():
        line = raw.rstrip()
        if not line:
            continue
        head = FEATURE_HEAD.match(line)
        if head:
            if current is not None and current["steps"]:
                features.append(current)
            current = {
                "category": head.group(1).strip(),
                "description": head.group(2).strip(),
                "steps": [],
                "passes": False,
            }
            continue
        step = STEP.match(line)
        if step and current is not None:
            current["steps"].append(step.group(1).strip())
    if current is not None and current["steps"]:
        features.append(current)
    return features

def merge_passes(new: list[dict], existing: Path) -> list[dict]:
    """Preserve passes:true from an existing feature_list.json."""
    if not existing.exists():
        return new
    data = json.loads(existing.read_text(encoding="utf-8"))
    if not isinstance(data, list):
        return new
    by_desc = {
        f.get("description"): bool(f.get("passes")) for f in data
    }
    for feat in new:
        if by_desc.get(feat["description"], False):
            feat["passes"] = True
    return new

```

Why preserve only the passes flag? Matching by `description` keeps progress intact when you edit `features.txt`. If you change a description, it's treated as a new feature – preventing accidental overwrites.

The full script is in the companion repo at `ch09-long-running-agent/generate_features.py`.

Step 3: Initialize with `init.sh`

Run once at the start of the first session. The script copies helper scripts, initializes Git, generates `feature_list.json`, creates `claude-progress.txt` and `AGENT_PROMPT.md`, and makes the initial commit:

```
#!/usr/bin/env bash
set -euo pipefail

ROOT_DIR="${1:-.}"
SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
PYTHON="${PYTHON:-python3}"

mkdir -p "$ROOT_DIR" && cd "$ROOT_DIR"

# [1/5] git repository (supports worktree .git files)
if ! git rev-parse --git-dir >/dev/null 2>&1; then
    git init -q
fi

# [2/5] feature_list.json
"$PYTHON" generate_features.py features.txt feature_list.json

# [3/5] claude-progress.txt
if [ ! -f claude-progress.txt ]; then
    cat > claude-progress.txt <<'EOF'
# Long-Running Project Progress Log
This file is updated automatically by update_progress.sh.
EOF
fi

# [4/5] AGENT_PROMPT.md and work directories
if [ ! -f AGENT_PROMPT.md ]; then
    cat > AGENT_PROMPT.md <<'EOF'
You are a coding agent for a long-running project.
1. Run `tail -20 claude-progress.txt` to review previous work
2. Run `python3 next_feature.py` to get the next feature
3. Implement that single feature
4. Run tests and verify they pass
5. Commit and run `python3 mark_passed.py "<description>"`
6. Run `./update_progress.sh "<done>" "<next>"`
EOF
fi
mkdir -p current_tasks agent_logs

# [5/5] Initial commit
if [ -z "$(git log --oneline 2>/dev/null || true)" ]; then
    git add features.txt feature_list.json \
        claude-progress.txt AGENT_PROMPT.md
    git commit -q -m "chore: initialize long-running agent harness"
fi
```

Why not override git config? Don't hardcode `-c user.name=...` in the harness. It conflicts with project-level pre-commit hooks. Use whatever Git configuration the project already has.

Step 4: Three Scripts That Power Each Session

next_feature.py returns the highest-priority incomplete feature. Error handling is trimmed for readability – the companion repo has the full version:

```
#!/usr/bin/env python3
"""Return the highest-priority incomplete feature."""
import json, sys
from pathlib import Path

def main(argv: list[str]) -> int:
    path = Path(argv[1]) if len(argv) >= 2 else Path("feature_list.json")
    data = json.loads(path.read_text(encoding="utf-8"))
    for feat in data:
        if isinstance(feat, dict) and not feat.get("passes", False):
            json.dump(feat, sys.stdout, ensure_ascii=False, indent=2)
            sys.stdout.write("\n")
            return 0
    sys.stdout.write("{}\n")
    return 0

if __name__ == "__main__":
    sys.exit(main(sys.argv))
```

mark_passed.py flips the `passes` flag for a completed feature. It requires an exact description match and exactly one result – zero or multiple matches cause an error:

```

import json, sys
from pathlib import Path

description = sys.argv[1]
path = Path(sys.argv[2]) if len(sys.argv) >= 3 \
    else Path("feature_list.json")
data = json.loads(path.read_text(encoding="utf-8"))

matches = [
    f for f in data
    if isinstance(f, dict) and f.get("description") == description
]
if len(matches) != 1:
    sys.stderr.write(f"error: matched {len(matches)} features\n")
    sys.exit(1)
matches[0]["passes"] = True
path.write_text(
    json.dumps(data, ensure_ascii=False, indent=2) + "\n",
    encoding="utf-8",
)

```

Why exact match with single-result enforcement? Partial matching lets the agent accidentally mark the wrong feature as complete. Requiring unique descriptions also catches a design mistake: two features with the same description.

update_progress.sh appends the latest commit SHA and subject to `claude-progress.txt`:

```

#!/usr/bin/env bash
set -euo pipefail
COMPLETED="$1"; NEXT_STEP="$2"
PROGRESS_FILE="${PROGRESS_FILE:-claude-progress.txt}"

SHA=$(git rev-parse --short=7 HEAD 2>/dev/null || echo "uncommitted")
SUBJECT=$(git log -1 --pretty=%s 2>/dev/null || echo "(no commits)")
TIMESTAMP=$(date '+%Y-%m-%d %H:%M')

{
    echo
    echo "## ${TIMESTAMP} ${SHA}"
    echo "- Completed: ${COMPLETED}"
    echo "- Commit: ${SHA} \"${SUBJECT}\""
    echo "- Next steps: ${NEXT_STEP}"
} >> "$PROGRESS_FILE"

```

Step 5: A Complete Session Walkthrough

Putting all five scripts together, here's what one coding agent session looks like:

```

# 1. Review previous work
tail -20 claude-progress.txt
git log --oneline -10

# 2. Get the next feature
python3 next_feature.py
# => {"category": "functional",
#      "description": "User can log in", ...}

# 3. Implement the feature (work happens here)

# 4. Run tests
npm test

# 5. Commit
git add -A
git commit -m "feat: User can log in"

# 6. Mark feature as complete
python3 mark_passed.py "User can log in"

# 7. Update progress log
./update_progress.sh "Login flow" "Add logout feature"

```

When this session ends, the next agent reads `tail -20 claude-progress.txt` and immediately knows: login is done, logout is next.

Mapping to the Four Harness Elements

File	Role
feature_list.json (feature list)	Single source of truth for all features and their status
claude-progress.txt (progress file)	Session-to-session handoff log

The four supplementary scripts implement the lifecycle:

Script	Role
init.sh	Environment setup and initial commit
generate_features.py	Convert features.txt to JSON (preserving passes)
next_feature.py	Return highest-priority incomplete feature

Script	Role
mark_passed.py	Mark one feature as complete

Long-Running Patterns in Claude Code

The harness architecture maps directly to Claude Code features. Three patterns offer increasing levels of automation.

Pattern 1: CLAUDE.md-Based State Management

The simplest approach is to write progress management instructions in `CLAUDE.md` and manage state through files:

```
# CLAUDE.md

## Long-Running Task Management

### Progress File
- Record work logs in `claude-progress.txt`
- Each entry includes: timestamp, completed work, next steps

### Work Rules
1. At session start, read `claude-progress.txt` and `git log`
2. Pick the highest-priority incomplete task
3. After completing a task, run tests to verify
4. Commit to Git and update the progress file
5. Keep the environment clean (build must pass)
```

The progress file builds a session-by-session record:

```
## 2026-04-09 Session 1
- Completed: Project init, DB connection, user model
- Commit: a1b2c3d "feat: add user model and DB connection"
- Next steps: Implement auth endpoints

## 2026-04-09 Session 2
- Completed: JWT auth, login API, token refresh
- Commit: e4f5g6h "feat: implement JWT authentication"
- Next steps: User registration and password reset
```

Pattern 2: Subagent Persistent Memory

Chapter 4 introduced persistent memory for subagents. A `project-worker` subagent with the `memory` field set to `project` gets its own `MEMORY.md` that persists across sessions:


```

---
name: project-worker
description: Continuously advance a long-running project
memory: project
model: opus
---

You are a continuous developer for this project.

## Session Start Procedure
1. Read MEMORY.md in the memory directory
2. Check the "next steps" recorded in the previous session
3. Review recent changes via Git log
4. Pick the highest-priority task and begin work

## Session End Procedure
1. Record completed work in memory
2. Add any discovered patterns or caveats to memory
3. Clearly record next steps
4. Leave the environment in a clean state

```

Persistent memory is stored in `.claude/agent-memory/project-worker/` and is accessible across sessions.

Pattern 3: Loop Execution with `claude --agent`

For fully automated long-running work, wrap `claude --agent` in a shell loop:

```

#!/bin/bash
# long-run.sh -- recommended to run inside a container

while true; do
    COMMIT=$(git rev-parse --short=6 HEAD)
    LOGFILE="agent_logs/agent_${COMMIT}.log"

    claude --agent project-worker \
        --dangerously-skip-permissions \
        -p "$(cat AGENT_PROMPT.md)" \
        --model claude-opus-4-6 &> "$LOGFILE"
done

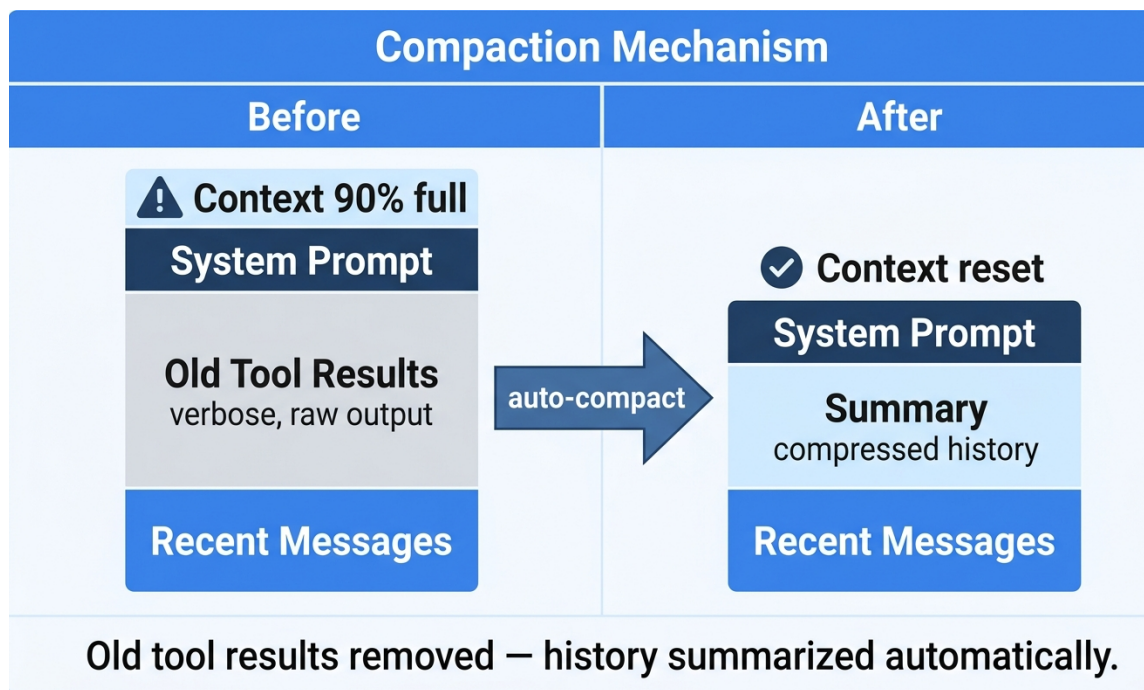
```

Why run inside a container? According to Anthropic Engineering Blog (2026-02), an agent once ran `pkill -9 bash` and killed the loop itself. Running in a container isolates the agent's actions from the host environment.

Memory Management and Context Compression

Context window management is the biggest technical challenge for long-running agents. Three techniques address this, each suited to different situations.

Compaction



Compaction Mechanism

Claude Code automatically compacts the conversation when the context window approaches its limit. Old tool results are removed and the remaining history is summarized:

```
[Before compaction]
Message history: 100,000 tokens
+-- Early exploration results (no longer needed)
+-- Intermediate tool call results (can be summarized)
+-- Recent work context (important)

[After compaction]
Compressed summary: 5,000 tokens
+-- Architecture decisions
+-- Unresolved bugs
+-- Implementation details
+ Contents of the last 5 accessed files
```

Anthropic recommends this priority for compaction quality:

1. **Maximize recall first:** Include all relevant information, even if verbose.
2. **Improve precision next:** Gradually remove irrelevant content.
3. **Clear tool results:** Removing old tool call results is the safest lightweight compaction.

Structured Note-Taking

Structured note-taking externalizes information beyond the context window. Agents create and update `NOTES.md` files to track context and dependencies across dozens of tool calls:

```
# NOTES.md (auto-generated and updated by the agent)

## Current Work Status
- Refactoring the authentication module
- Consolidating JWT verification logic into auth/jwt.ts

## Discovered Issues
- auth/middleware.ts:45 is missing token expiration handling
- 3 tests failing in tests/auth.test.ts

## Next Steps
1. Fix validateToken in auth/jwt.ts
2. Get all tests passing
3. Refactor auth/middleware.ts
```

Why note-taking works In Anthropic’s “Claude playing Pokemon” experiment, the agent recorded progress across thousands of steps. After context resets, it could resume training sequences by reading its own notes.

Subagent Architecture for Context Isolation

Chapter 3 explained subagent context isolation. This is its biggest advantage for long-running work:

```
[Lead agent]          <-- lightweight context
+-- [Subagent 1]       <-- tens of thousands of tokens of exploration
|   +-- Result: 1,000-2,000 token summary only
+-- [Subagent 2]       <-- tens of thousands of tokens of analysis
|   +-- Result: structured conclusions only
+-- [Subagent 3]       <-- tens of thousands of tokens of search
    +-- Result: key findings only
```

Each subagent works in its own context window. Only a condensed summary flows back to the lead agent, keeping the lead’s context clean.

Choosing the Right Technique

Each technique fits a different situation:

- **Compaction:** Best for long dialogues that need conversational continuity. Trade-off: some information loss during summarization.
- **Note-taking:** Best for tasks with clear milestones and iterative development. Trade-off: requires disciplined recording habits.
- **Subagent delegation:** Best for parallel exploration where high context compression is needed. Trade-off: results are summaries, not full context.

Lessons from the C Compiler Project

The C Compiler project offers three practical lessons for long-running agent design. According to Anthropic Engineering Blog (2026-02), “Building a C compiler with a team of parallel Claudes,” approximately 2,000 sessions

produced a 100,000-line C compiler. Chapter 1 introduced the project; this section revisits it through the lens of long-running agent harnesses.

Test Quality Drives Agent Quality

The most important lesson: test quality determines agent outcomes. Agents solve problems autonomously, but tests determine *which* problems they solve.

```
Test quality -> Agent direction -> Output quality  
High-quality tests -> Correct direction -> Correct output  
Low-quality tests  -> Wrong direction  -> Wrong output
```

During later phases, new feature implementations frequently broke existing features. The fix was a CI pipeline that rejected any commit breaking existing tests.

Design for the Agent’s Perspective

Test harnesses for long-running agents need different design principles than harnesses for humans:

```
Human-oriented (less effective):  
- Thousands of lines of test output  
- Detailed stack traces for errors  
- Full test suite takes 30 minutes  
  
Agent-oriented (more effective):  
- Output limited to a few lines; details go to log files  
- Errors use a grep-friendly format: "ERROR: reason"  
- A --fast option runs 1% random sample per session
```

Why control output volume? Thousands of lines of test output cause “context window pollution” – important information gets buried. The harness should emit aggregated statistics only and save details to files.

Countering the Lack of Time Awareness

Agents have no sense of time. Left unchecked, they’ll run tests for hours without noticing.

Countermeasures from the C compiler project:

- Emit progress at low frequency, incrementally.

- Default to a `--fast` option (1% sample).
- Samples are deterministic per agent (same agent runs same tests) but random across VMs (all tests get coverage).

Checkpoints and Recovery

Git-Based Checkpoints

The simplest practical checkpoint mechanism is Git commits:

```
## Checkpoint Rules (include in CLAUDE.md)

1. Commit to Git after each feature is implemented
2. Write specific implementation details in the commit message
3. Only commit when tests are passing
4. If problems occur, revert to the previous commit

### Commit Message Format
feat: [feature name] -- [specific changes]
fix: [bug summary] -- [what was fixed]
```

According to Anthropic Engineering Blog (2025-11), Git-based checkpoints let agents “revert bad code changes and recover working states.”

File-Based State Management

For multi-agent long-running projects, file-based state provides coordination:

```
project/
+-- claude-progress.txt      <-- work log (each session appends)
+-- feature_list.json       <-- all features and completion status
+-- current_tasks/         <-- task locks for active work
|   +-- parse_if_statement.txt
|   +-- codegen_function.txt
+-- agent_logs/             <-- per-session logs
    +-- agent_a1b2c3.log
    +-- agent_d4e5f6.log
```

The C compiler project used `current_tasks/` for task locking. When an agent starts a task, it creates a lock file. If two agents attempt the same task, Git synchronization forces the later agent to pick a different one.

Long-Running Best Practices

Keep the Environment Clean

At the end of every session, the codebase should be mergeable to main:

- Build passes.
- Tests pass.
- Code is organized.
- Progress file is updated.

The opposite – half-implemented features, failing tests, leftover debug code – forces the next session to waste time cleaning up instead of making progress.

Regression-Test Before Building

At the start of each session, verify that existing features still work:

1. Run the initialization script to start the dev server.
2. Execute basic end-to-end tests.
3. If anything is broken, fix bugs before starting new features.

According to Anthropic Engineering Blog (2025-11), this rule significantly improved quality: “Verify existing basic functionality works before implementing new features.”

Calibrate Autonomy to Risk

Match the agent’s autonomy level to the task’s risk profile:

- **High autonomy:** Feature implementation with comprehensive tests. Low risk – tests guarantee quality.
- **Moderate autonomy:** Refactoring existing code. Medium risk – requires regression testing.

- **Low autonomy:** Security-related changes. High risk – human review is mandatory.

Summary

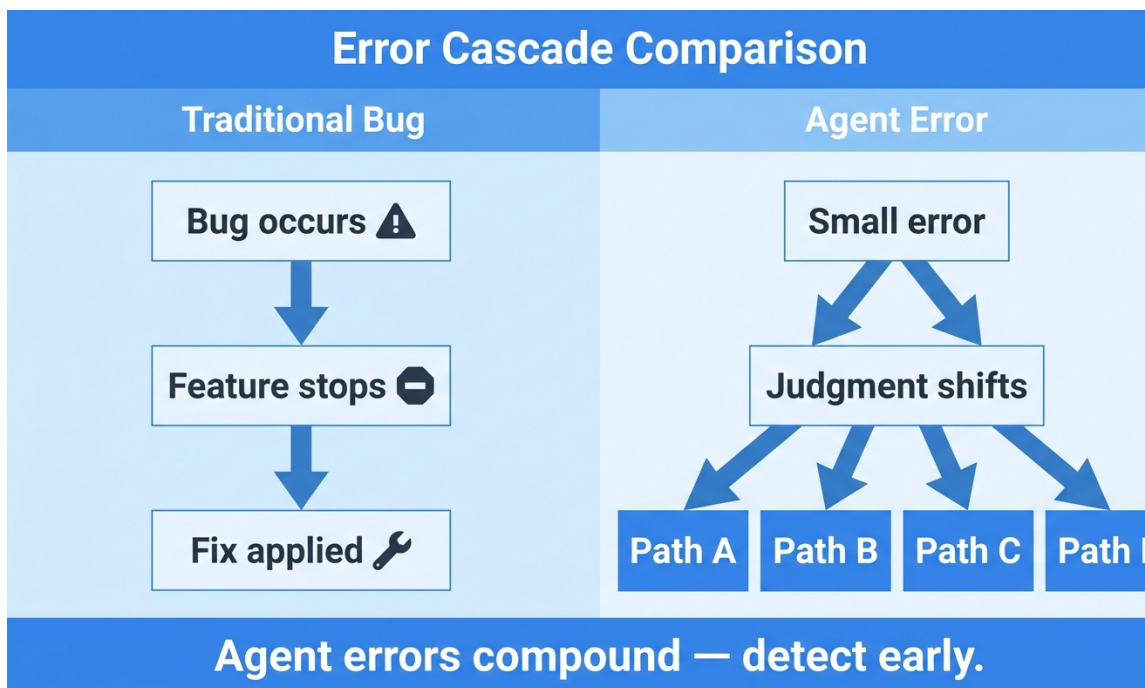
- Long-running agents fail in two predictable ways: one-shotting (attempting everything in one session) and premature completion (declaring done while features remain). A harness prevents both.
- A harness uses two agents: an initializer that sets up the feature list and Git state, and a coding agent that implements one feature per session, commits, and updates the progress log.
- Manage context across sessions with three complementary techniques: compaction (automatic summarization), structured note-taking (externalizing state to files), and subagent delegation (isolating exploration into child contexts).
- Test quality drives agent quality: design test harnesses for agent consumption – short output, grep-friendly errors, and a fast sampling option – not for human readability.
- End every session with the codebase in a clean, mergeable state; use Git commits as checkpoints so the next session can recover from any point.

Error Recovery and Failure Modes

Chapter 9 covered the architecture of long-running agents. Agent failures fall into four categories, each requiring a different recovery strategy. In long-running systems, the hardest problem isn't doing the work – it's recovering when things go wrong.

Companion code: [ch10-error-recovery](#)

Why Agent Failures Are Different



Error Cascade Comparison

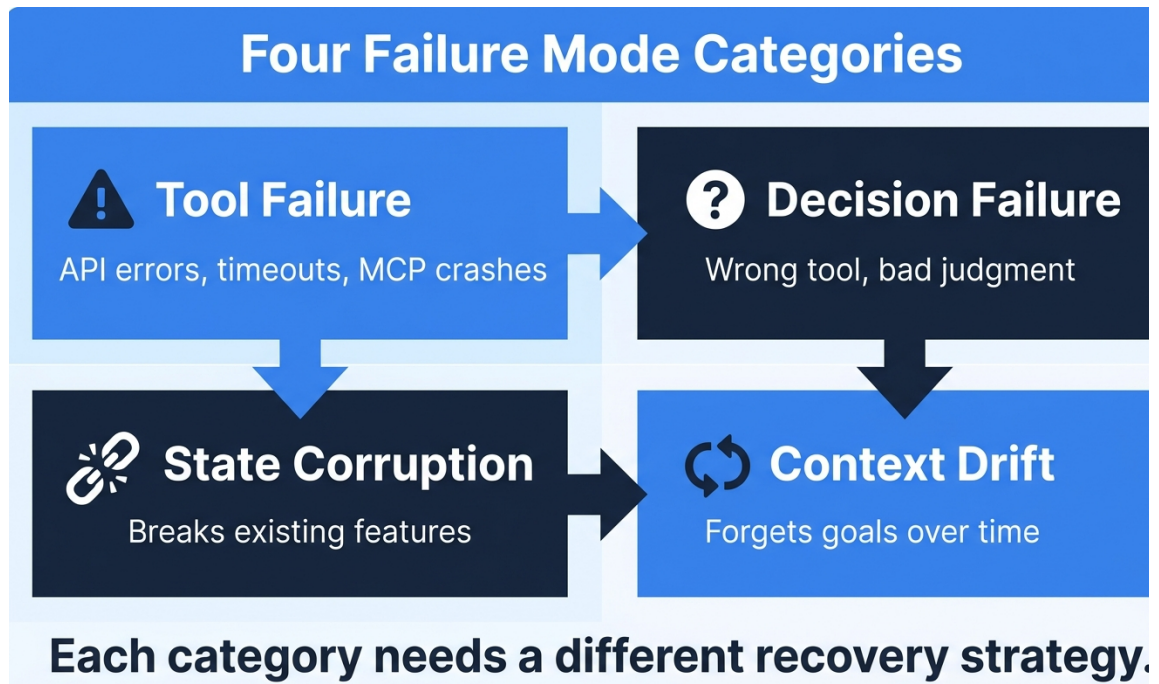
Traditional software bugs break a specific function or degrade performance. Agent systems are different: a small error can cascade into a completely different behavioral trajectory.

Why do agent errors cascade? According to Anthropic Engineering Blog (2025-06), a single step failure can send the agent down an entirely different path. A traditional bug is “a broken function.” An agent error is “a direction change for all subsequent behavior.”

Traditional software:
Bug -> One function breaks -> Fix it

Agent system:
Small error -> Judgment shifts -> Different tool selected
-> Different result -> Further judgment shifts -> Entirely
different trajectory

Four Failure Mode Categories



Four Failure Mode Categories

Agent failures fall into four categories, each requiring different recovery strategies.

Category 1: Tool Failure

External tools or MCP servers return errors. The agent's judgment was correct, but the execution environment failed.

Typical causes: API timeouts, expired auth tokens, rate limiting, MCP server crashes, network outages.

```
Agent: "Fetch the list of PRs from GitHub"  
-> calls mcp_github_list_pull_requests  
-> Error: "401 Unauthorized: Token expired"
```

Recovery strategy:

```
Tool Failure Recovery:
Immediate Retry:
  condition: "Timeouts, transient network errors"
  max_attempts: 3
  backoff: "Exponential (1s, 2s, 4s)"

Alternative Tool:
  condition: A specific tool fails persistently
  example: "Web search fails -> consult local docs"

Notify Agent:
  condition: All retries exhausted
  message: "GitHub tool unavailable. Consider alternatives"
```

Why surface failures to the agent? According to Anthropic Engineering Blog (2025-06), letting agents know about tool failures so they can adapt “works surprisingly well.” When agents recognize a failure, they autonomously select alternative strategies.

Category 2: Decision Failure

The agent makes a poor judgment call. Tools work fine, but the agent uses them incorrectly.

Typical causes: wrong tool selection, imprecise search queries, task misinterpretation, conclusions based on insufficient information.

```
Agent: "Investigate the database problem"
-> Web searches for "database connection error"
-> Gets thousands of generic articles
-> Proposes an irrelevant solution
(Should have checked local log files first)
```

Recovery strategy:

Decision Failure Recovery:

Tool Selection Guide:

method: Include explicit heuristics in the prompt

example: "For local issues, check log files first"

Self-Verification:

method: Add a step to check result validity

example: "Verify proposed fix matches actual error"

Insert Thinking Step:

method: Make the agent plan before acting

example: "Before using a tool, state why you chose it"

Category 3: State Corruption

The agent's actions break the project's state. The agent believes it did the right thing, but the system is actually broken.

Typical causes: changes that break existing features, incomplete refactoring, deleted or disabled tests, misconfigured settings.

Agent: "Refactor the authentication logic"

-> Rewrites auth/middleware.ts

-> New auth flow works correctly

-> But 10 existing API endpoints now fail authentication

Recovery strategy:

State Corruption Recovery:

Regression Tests:

method: "Run tests before and after changes"

implementation: "CI pipelines, pre-commit hooks"

Git Checkpoints:

method: "Commit before changes, revert if problems"

implementation: "Commit after each feature"

Incremental Changes:

method: Break large changes into small steps

implementation: "Modify no more than 3 files at a time"

Worktree Isolation:

method: Run subagents in isolated worktrees

implementation: "isolation: worktree frontmatter"

Claude Code's `isolation: worktree` frontmatter runs subagent changes in an isolated Git worktree. If tests pass, merge the changes. If they fail, discard the worktree entirely:

```
---
name: risky-refactor
description: Execute risky refactoring in an isolated environment
isolation: worktree
---
```

Execute the refactoring.
After making changes, verify that all tests pass before reporting completion. If tests fail, abort the changes.

Category 4: Context Degradation

The agent's judgment gets worse as the context window fills up.

Typical causes: context window overload, contradictions between old and new information, loss of critical information during compaction, noise from accumulated tool results.

```
Session start: "Fix the auth module bug"
-> Accurately identifies the problem, proposes a fix
```

```
Session later: "Fix the next bug"
-> Forgets the earlier fix, rediscovers the same bug
-> Old and new tool results mix, causing confusion
```

Recovery strategy:

Context Degradation Recovery:

Structured Note-Taking:

method: Record key findings in external files
implementation: "NOTES.md, claude-progress.txt"

Appropriate Compaction:

method: "Remove old tool results, retain summaries"
implementation: Claude Code automatic compaction

Delegate to Subagent:

method: Offload heavy work to a fresh context
implementation: Spawn new subagent via Agent tool

Session Splitting:

method: Intentionally break long sessions
implementation: "After 10+ tool calls, record progress
and start a new session"

Retry Strategy Design

Exponential Backoff

For transient failures (network issues, rate limits), exponential backoff is the standard approach:

Retry 1: after 1 second
Retry 2: after 2 seconds
Retry 3: after 4 seconds
Retry 4: report as failed

Implementation in Claude Code

Use Hooks to add validation before and after tool execution. A `resilient-worker` subagent can check exit codes automatically:

```

---
name: resilient-worker
description: Error-resilient worker agent
hooks:
  PostToolUse:
    - matcher: "Bash"
      hooks:
        - type: command
          command: "../scripts/check-exit-code.sh"
---

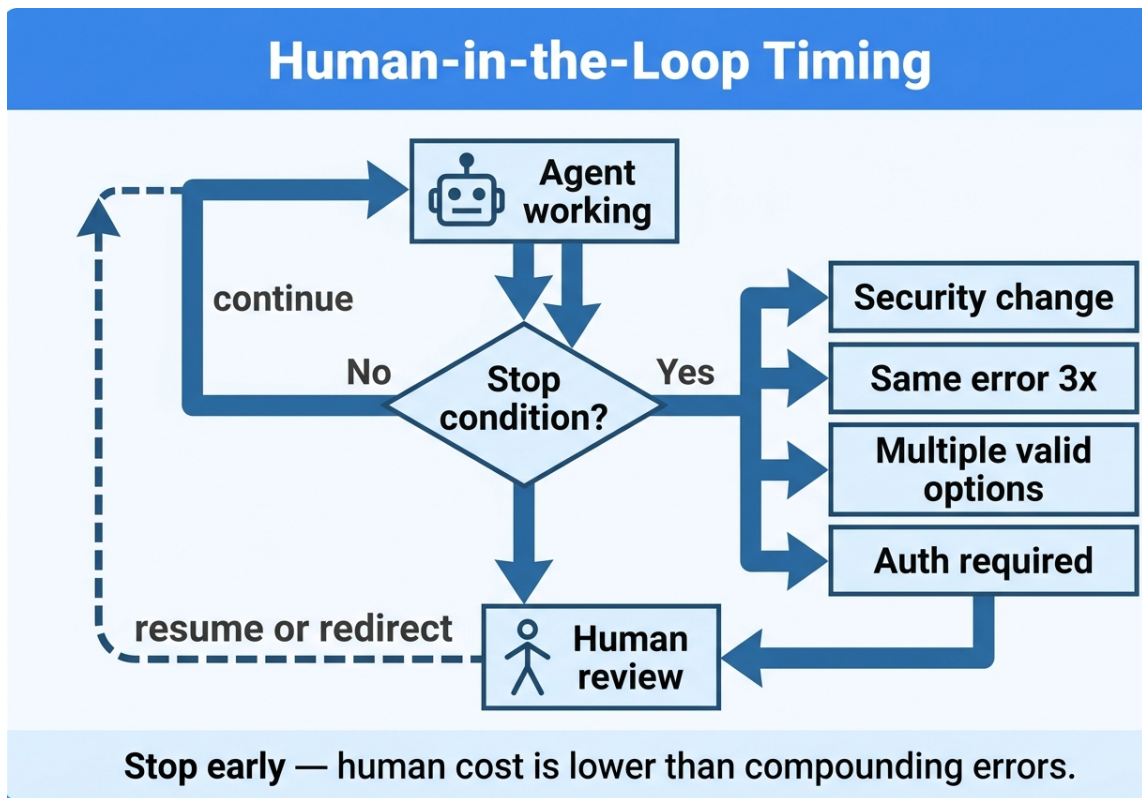
## Error Handling Rules

1. When a tool returns an error:
   - Read the error message and identify the cause
   - For transient errors (timeout, network): retry once
   - For persistent errors (auth failure, missing resource):
     consider alternatives

2. When no alternative exists:
   - Record the situation and skip to the next task
   - Mark skipped tasks as "blocked" in the progress file

```

Designing Human-in-the-Loop Timing



Full autonomy isn't always optimal. Certain situations require human intervention.

When Intervention Is Needed

Situation	Response
Security-related changes	Request approval before proceeding
Production environment operations	Present the change plan for confirmation

Three more cases warrant stopping:

- **The same error three times:** Automatic recovery has hit its limit. Report the problem and wait for instructions.
- **Design decisions with multiple valid options:** Present the options and ask for a judgment call.
- **External service authentication:** The agent can't resolve this alone. Ask the user to authenticate.

Implementation in Claude Code

The `maxTurns` field in a subagent definition caps autonomous operation:


```
---
name: careful-worker
description: Work cautiously, stopping when uncertain
maxTurns: 20
permissionMode: default
---

## Stop Conditions

Stop work and report your findings if any of these occur:
1. The same error has occurred twice in a row
2. You need to access unexpected files or resources
3. There are 3+ options and you cannot determine which is best
4. Security-sensitive changes are required

## Report Format

When stopping, report:
- What you were trying to do
- What went wrong
- Possible solutions
- Your recommendation and rationale
```

Checkpoint-Based Intervention

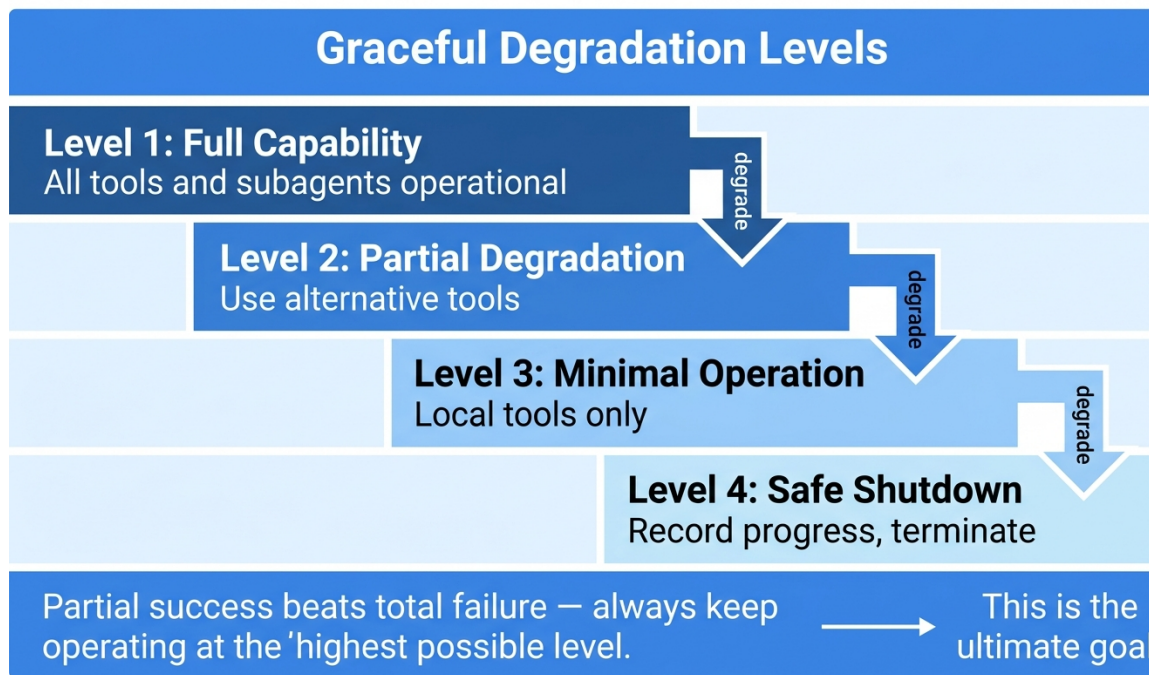
For long-running agents, periodic checkpoints provide natural intervention points:

```
[Agent] -> Feature 1 done -> [Checkpoint: Git commit]
| Auto-continue (if tests pass)
[Agent] -> Feature 2 done -> [Checkpoint: Git commit]
| Auto-continue
[Agent] -> Feature 3 done -> [Human review requested]
| Continue/correct after review
```

Graceful Degradation

Not every failure allows full recovery. Design for partial success – maintain as much capability as possible while minimizing the impact of failures.

Degradation Levels



Graceful Degradation Levels

Level 1: Full Capability

All tools available, all subagents operational

Level 2: Partial Degradation

Some MCP servers unavailable

- > Continue work using alternative tools
- > Skip unavailable features and retry later

Level 3: Minimal Operation

All external tools unavailable

- > Work with local Read/Grep/Glob only
- > Complete whatever is possible within constraints

Level 4: Safe Shutdown

Context degraded, accurate judgment no longer possible

- > Record progress and terminate the session
- > Resume from fresh context in the next session

Implementation in Claude Code

Encode degradation rules directly in `CLAUDE.md`:

```
## Graceful Degradation Rules

### When MCP servers are unavailable
- GitHub MCP unavailable -> use `gh` CLI commands instead
- Playwright MCP unavailable -> skip tests, flag for manual verification later
- Database MCP unavailable -> validate with local SQLite

### When all external services are unavailable
- Focus on work achievable with local tools only
- Record tasks with external dependencies as "blocked"
- Report the list of blocked tasks

### When context has degraded
- Record current state in detail in the progress file
- Save current code with a Git commit
- Record "handoff notes for the next session" and terminate
```

Learning from Failures

Production agent systems get better over time – but only if you record failure patterns and feed those lessons back into future runs.

Failure Log Structure

```
## Failure Record: 2026-04-09

### What Failed
- Category: State corruption
- Context: During authentication module refactoring
- Root cause: Failed to account for 10 dependent API endpoints
- Impact: All E2E tests failed

### Actions Taken
- Reverted to pre-change state with `git revert`
- Investigated the blast radius before retrying

### Prevention Measures
- Run `grep -r "importFrom('auth')" .` to check dependencies before refactoring
- When blast radius exceeds 5 files, apply changes incrementally
```

Feeding Lessons into Subagent Memory

Record failure patterns in a subagent's persistent memory to prevent repeating the same mistakes:

```
# .claude/agent-memory/careful-worker/MEMORY.md

## Past Failure Patterns

### Authentication Module Changes
- Changes to auth/ have a wide blast radius
- Always investigate dependencies before modifying
- Incremental changes recommended (one file at a time)

### Database Schema Changes
- Take a backup before running migrations
- Run in the test environment first
```

Over time, this memory accumulates the project's institutional knowledge – the kind of hard-won lessons that would otherwise exist only in a senior engineer's head.

Summary

- Agent failures fall into four categories: tool failure, decision failure, state corruption, and context degradation – each requiring a distinct recovery strategy.
- Tool failures respond to retries and fallback tools; decision failures are addressed through prompt heuristics and self-verification steps.
- State corruption is prevented by Git checkpoints, regression tests, and worktree isolation; context degradation is managed through note-taking and session splitting.
- Human-in-the-loop triggers belong at security changes, repeated errors, and unresolvable design decisions.
- Recording failure patterns in subagent persistent memory turns hard-won lessons into durable institutional knowledge.

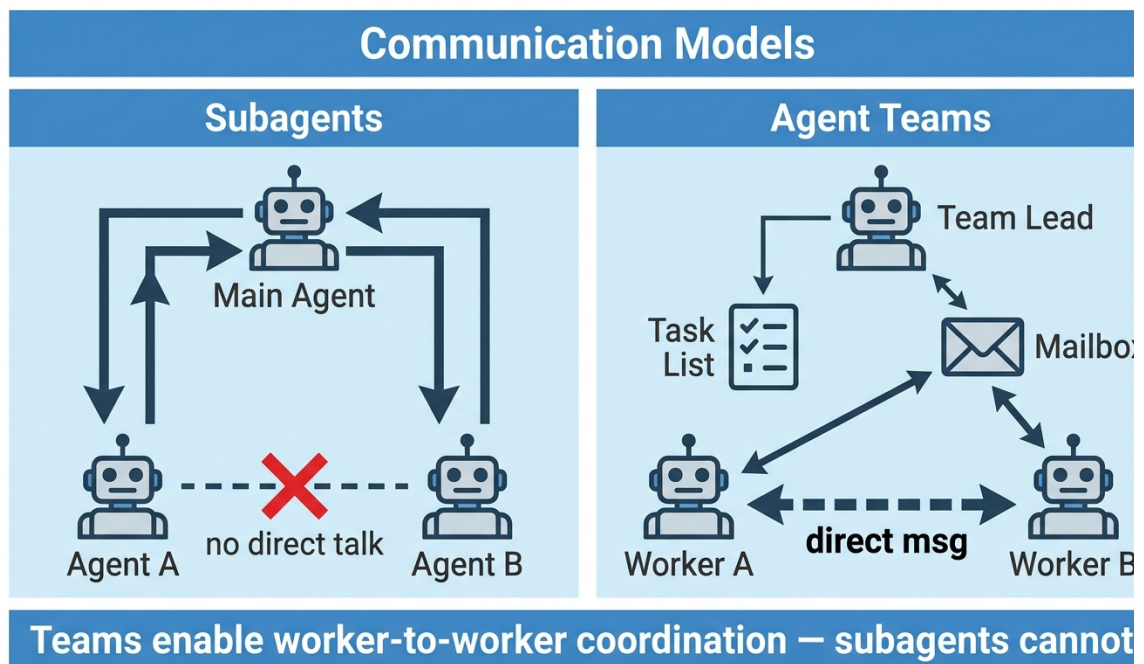
Getting Started with Agent Teams

Part IV covered long-running agents – harness design, memory management, and error recovery for agents that span multiple sessions. Part V shifts from managing time – sessions that span days – to managing parallelism: multiple Claude Code instances working simultaneously as a coordinated team. This chapter covers what agent teams are, how they differ from subagents, how to enable them, and what Anthropic’s C compiler project reveals about parallel agent work.

Companion code: [ch11-agent-team-concept](#)

At time of writing (Claude Code v2.1.32): Agent teams are experimental and disabled by default. Enable with `CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS` in `settings.json`.

Where Subagents Fall Short



Communication Models

Subagents from Part II are powerful, but they have a structural constraint: communication flows in one direction only.

Subagent communication model:

```
Main Agent
  | dispatch    ^ results
Subagent A
  | dispatch    ^ results
Subagent B
```

A and B cannot talk to each other.
All information flows through the main agent.

This bottleneck becomes a problem in three situations.

- **Cross-review:** subagent A's output needs review by subagent B.
- **Knowledge sharing:** a discovery during investigation needs to reach other agents immediately.
- **Competing hypotheses:** multiple agents need to challenge each other's theories directly.

Why agent teams? Subagents excel at one-directional delegation – dispatch a task, get a result. When workers need to share findings, challenge each other, and coordinate on their own, agent teams are the right tool.

What Agent Teams Are

An agent team is a group of Claude Code instances working as a coordinated unit. One session acts as the **team lead**, managing the overall workflow. The remaining sessions – called **teammates** – work independently while communicating directly with each other.

Subagents vs. Agent Teams

The decision comes down to whether workers need to talk to each other.

Subagents:

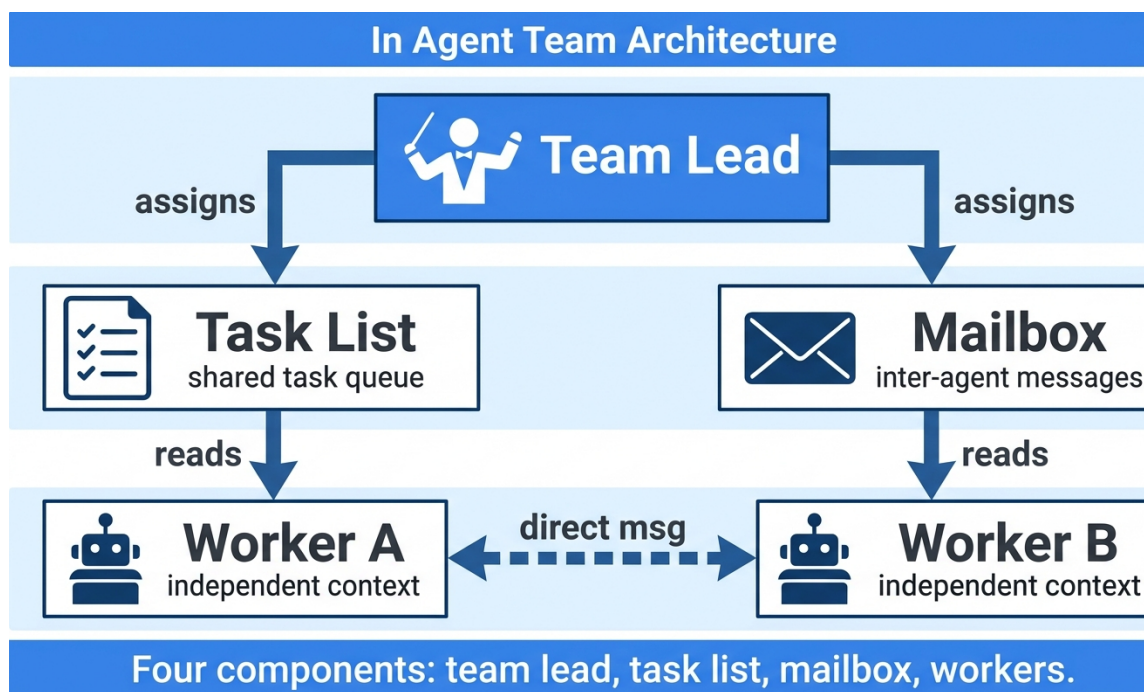
- Own context window; results return to the caller
- Report to the main agent only
- Main agent manages all coordination
- Best for focused tasks where only the result matters
- Lower token cost (results summarized back)

Agent teams:

- Own context window; fully independent
- Teammates message each other directly
- Shared task list with self-coordination
- Best for complex work requiring discussion and collaboration
- Higher token cost (each teammate is a separate instance)

Why does the distinction matter? Agent teams consume significantly more tokens. They work best when teammates can operate independently. For sequential tasks or same-file edits, subagents are more efficient.

Architecture



Agent Team Architecture

An agent team has four components.

- **Team lead:** the main session that creates the team, spawns teammates, and coordinates work
- **Teammates:** independent Claude Code instances, each working on assigned tasks

- **Task list:** a shared list of work items that teammates claim and complete
- **Mailbox:** a messaging system for direct communication between agents

Team architecture:

```
[Team Lead]
+-- Task list (shared)
|   +-- Task 1: pending
|   +-- Task 2: in_progress (-> Teammate A)
|   +-- Task 3: completed
+-- Mailbox (bidirectional)
|   +-- Lead <-> Teammate A
|   +-- Lead <-> Teammate B
|   +-- Teammate A <-> Teammate B <-- direct
+-- [Teammate A] independent context
+-- [Teammate B] independent context
```

Team configuration and tasks are stored locally:

- Team config: `~/.claude/teams/{team-name}/config.json`
- Task list: `~/.claude/tasks/{team-name}/`

Claude Code generates these files automatically when you create a team and updates them as teammates join, go idle, or leave.

Enabling Agent Teams

Add the environment variable to your `settings.json`:

```
{
  "env": {
    "CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS": "1"
  }
}
```

Place this in either location:

- **Project settings:** `.claude/settings.json` (applies to all team members)
- **User settings:** `~/.claude/settings.json` (applies to your environment only)

You can also set it as a shell environment variable:

```
export CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS=1
```

Verify your Claude Code version meets the requirement:

```
claude --version
# Must be v2.1.32 or later
```

Starting a Team

You create teams with natural language. Describe the task and the team structure you want, and Claude Code handles the rest.

```
I'm designing a CLI tool that helps developers track TODO
comments across their codebase. Create an agent team to
explore this from different angles: one teammate on UX, one
on technical architecture, one playing devil's advocate.
```

Claude Code responds by:

1. Creating a shared task list.
2. Spawning teammates for each perspective.
3. Having teammates explore the problem.
4. Synthesizing findings.
5. Cleaning up the team when finished.

Two Ways Teams Start

Teams get created through one of two paths.

1. **You request a team:** describe a task that benefits from parallel work and explicitly ask for a team.
2. **Claude proposes a team:** if Claude determines your task would benefit from parallel work, it suggests creating one. You confirm before it proceeds.

In both cases, you stay in control. Claude won't create a team without your approval.

Context and Communication

What Teammates Receive

Each teammate has its own context window. When spawned, it loads the same project context as a regular session: `CLAUDE.md`, MCP servers, and skills. It also receives the spawn prompt from the lead.

The lead's conversation history does not carry over.

```
What a teammate gets:
[Y] CLAUDE.md (project configuration)
[Y] MCP server configuration
[Y] Skills definitions
[Y] Spawn prompt from the lead
[N] Lead's conversation history
```

Why no conversation history? Each teammate is an independent instance. Its strength is autonomous judgment, uncoupled from the lead's context. Include task-specific information in the spawn prompt instead.

Communication Mechanisms

Teammates share information in three ways.

1. **Automatic message delivery:** when a teammate sends a message, it arrives at the recipient automatically.
2. **Idle notifications:** when a teammate finishes and stops, the lead is notified automatically.
3. **Shared task list:** all agents can see task status and claim available work.

Messages have two forms:

- **message:** send to one specific teammate
- **broadcast:** send to all teammates simultaneously. Use sparingly – costs scale with team size.

Reusing Subagent Definitions

Custom subagent definitions from Chapter 4 work as teammate roles. Reference the subagent type by name when spawning:

```
Spawn a teammate using the security-reviewer agent type
to audit the authentication module.
```

The teammate inherits the definition's `tools` allowlist and `model` setting. The definition body is appended to the teammate's system prompt as additional instructions.

Team coordination tools – `SendMessage` and the task management tools – remain available to all teammates regardless of the `tools` restriction.

```
# .claude/agents/security-reviewer.md frontmatter
---
name: security-reviewer
description: Detect security vulnerabilities based on OWASP Top 10
model: sonnet
tools: Bash, Read, Grep, Glob
---

A security-focused review agent.
Detects vulnerabilities based on OWASP Top 10 criteria.
```

Why reuse definitions? The same role works as both a subagent and a teammate. Defining it once prevents duplicate maintenance. According to Anthropic’s documentation, the `skills` and `mcpServers` frontmatter fields are not applied when a definition runs as a teammate – teammates load skills and MCP servers from your project and user settings.

Lessons from the C Compiler Project

Chapter 1 introduced Anthropic’s C compiler project. The project used a custom harness to coordinate 16 parallel agents – Anthropic Engineering Blog (2026-02) is the primary source for what follows.

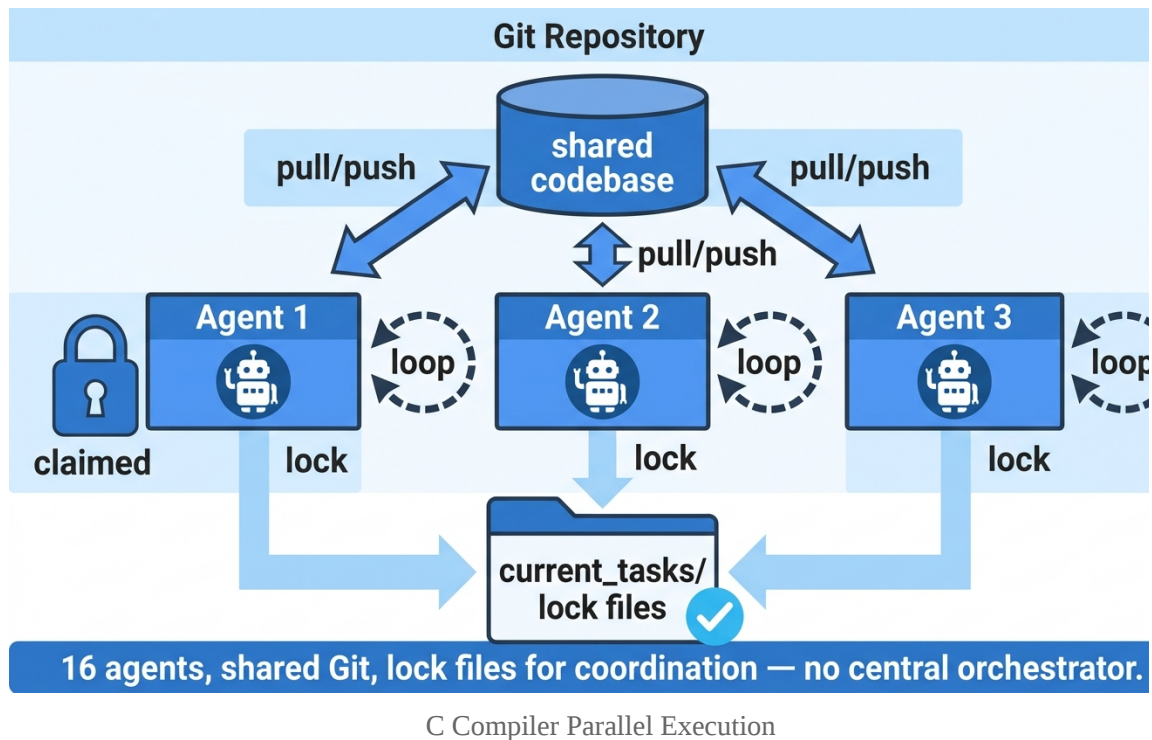
Project Summary

The full numbers are in Chapter 1. The key facts for this chapter:

- **Goal:** build a Rust-based C compiler from scratch, capable of compiling the Linux kernel

- **Scale:** 16 parallel agents produced a 100,000-line compiler over approximately two weeks

How Parallel Execution Worked



This project used a custom harness rather than the built-in agent teams feature. The synchronization algorithm:

1. An agent claims a task by writing a lock file to `current_tasks/` (e.g., `current_tasks/parse_if_statement.txt`). If two agents try to claim the same task, Git's synchronization forces the second agent to pick a different one.
2. After completing the task, the agent pulls from upstream, merges changes from other agents, pushes its changes, and removes the lock.
3. The harness loop spawns a new Claude Code session in a fresh container, and the cycle repeats.

When Parallelism Is Easy – and When It Isn't

Easy case: many independent failing tests. Each agent picks a different test to fix. No coordination needed.

Hard case: compiling the Linux kernel. One giant task. All 16 agents hit the same bug, tried the same fix, and overwrote each other's changes.

Why does parallelism design matter? The team solved the kernel problem by using GCC as an oracle compiler. A test harness randomly compiled most kernel files with GCC and only the remaining files with Claude's compiler. This let each agent work on a different file's bugs in parallel. The lesson: design task decomposition so that parallel agents work on independent units.

Specialization Through Multiple Roles

Parallel execution enables role specialization. The C compiler project assigned these roles:

- **Core development:** new features and bug fixes
- **Code consolidation:** merging duplicate code
- **Performance:** improving compiler execution speed
- **Output optimization:** improving efficiency of generated code
- **Code quality:** structural improvements from a Rust developer's perspective
- **Documentation:** maintaining READMEs and progress files

Community Resources

Community resources: The Claude Code community maintains curated collections of agent team configurations and teammate role definitions on GitHub. These community examples cover common team patterns – parallel reviewers, multi-perspective explorers, and specialization-based pipelines – that complement the patterns in this chapter and Chapter 12. Use them as reference material, not drop-in solutions; verify that any community definition aligns with your project's permission model before adopting it.

Limitations

Agent teams are experimental. Current limitations to be aware of:

- **No session resumption:** `/resume` and `/rewind` do not restore in-process teammates. After resuming, the lead may try to message teammates that no longer exist.
- **Task status lag:** teammates sometimes fail to mark tasks as completed, blocking dependent tasks.
- **Slow shutdown:** teammates finish their current request before shutting down.
- **One team per session:** a lead can manage only one team at a time.
- **No nesting:** teammates cannot spawn their own teams.
- **Fixed lead:** the session that creates the team stays the lead for its lifetime.
- **Permissions at spawn:** all teammates start with the lead's permission mode. You can change individual modes after spawning, but not at spawn time.

Summary

- Agent teams let multiple Claude Code instances coordinate through a shared task list and direct messaging.
- Subagents are for one-directional delegation; agent teams are for work that requires discussion and collaboration between workers.
- Enable teams with the `CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS` environment variable in `settings.json`.
- The C compiler project – 16 parallel agents, approximately two weeks, \$20,000, 100,000 lines – demonstrates both the potential and the boundaries of parallel agent work.
- Custom subagent definitions from Chapter 4 can be reused as teammate roles, avoiding duplicate maintenance.

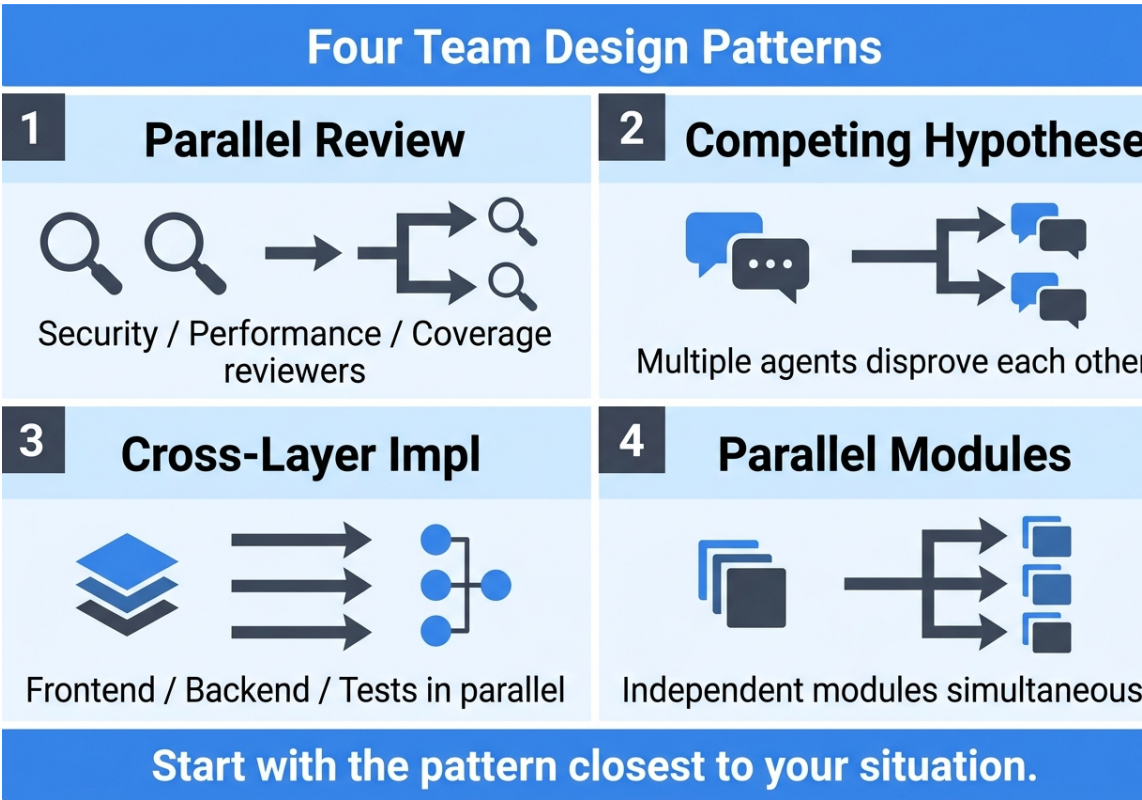
Agent Teams in Practice

This chapter covers the practical side of agent teams: design patterns, display modes, task sizing, quality gates with hooks, and conflict avoidance strategies for real parallel development work.

Companion code: [ch12-parallel-development](#)

At time of writing (Claude Code v2.1.32): Agent teams are experimental and disabled by default. Enable with `CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS` in `settings.json`. Requires Claude Code v2.1.32 or later.

Four Team Design Patterns



Four Team Design Patterns

Four patterns cover the most common use cases. They increase in coordination complexity, so start with the one closest to your situation.

Pattern 1: Parallel Code Review

Splitting review criteria into independent domains ensures security, performance, and test coverage all get thorough attention simultaneously. A single reviewer gravitates toward one type of issue at a time; parallel reviewers don't.

```
Create an agent team to review PR #142. Spawn three reviewers:  
- One focused on security implications  
- One checking performance impact  
- One validating test coverage  
Have them each review and report findings.
```

Each reviewer applies a different filter to the same PR. The lead synthesizes findings after all three finish. Sequential reviews suffer from anchoring – the first issue found biases everything that follows. Three independent reviewers with different lenses catch what a single pass misses.

Pattern 2: Competing Hypotheses

Use this pattern when the root cause is unclear: a single agent tends to find one plausible explanation and stop looking.

```
Users report the app exits after one message instead of staying connected. Spawn 5 agent teammates to investigate different hypotheses. Have them talk to each other to try to disprove each other's theories, like a scientific debate. Update the findings doc with whatever consensus emerges.
```

Why this pattern? Sequential investigation suffers from anchoring bias: once one theory is explored, subsequent investigation skews toward it. Multiple independent investigators actively trying to disprove each other produce a surviving theory that is far more likely to be the actual root cause.

Pattern 3: Cross-Layer Implementation

Changes that span frontend, backend, and tests – each layer gets its own teammate.

```
Create a team to implement the user profile feature:  
- Frontend teammate: React components and forms  
- Backend teammate: API endpoints and database schema  
- Testing teammate: E2E tests and unit tests  
Require plan approval before they make any changes.
```

Pattern 4: Parallel Module Development

Independent modules with no shared dependencies – the simplest case for parallel work.

```
Create a team to develop these independent modules in  
parallel:  
- Authentication module (src/auth/)  
- Notification module (src/notifications/)  
- Report module (src/reports/)  
Each teammate fully implements their assigned module.
```

Display Modes

Agent teams support two display modes.

In-Process Mode

All teammates run inside your main terminal.

- **Navigation:** `Shift+Down` to cycle through teammates, type to send messages
- **Task list:** `Ctrl+T` to toggle
- **Session view:** `Enter` to view a teammate's session, `Escape` to interrupt

- **Compatibility:** works in any terminal, no extra setup

Split-Pane Mode

Each teammate gets its own pane, showing all output simultaneously.

- **Navigation:** click into a teammate's pane to interact
- **Visibility:** all teammates visible at once
- **Requirements:** tmux or iTerm2

```
{  
  "teammateMode": "in-process"  
}
```

Place this in `~/.claude.json`. The default `"auto"` uses split panes inside tmux and in-process otherwise. To override for a single session:

```
claude --teammate-mode in-process
```

Why display mode matters In-process works everywhere, but monitoring multiple teammates simultaneously requires split panes. Split-pane mode is not supported in VS Code's integrated terminal, Windows Terminal, or Ghostty.

Specifying Teammates and Models

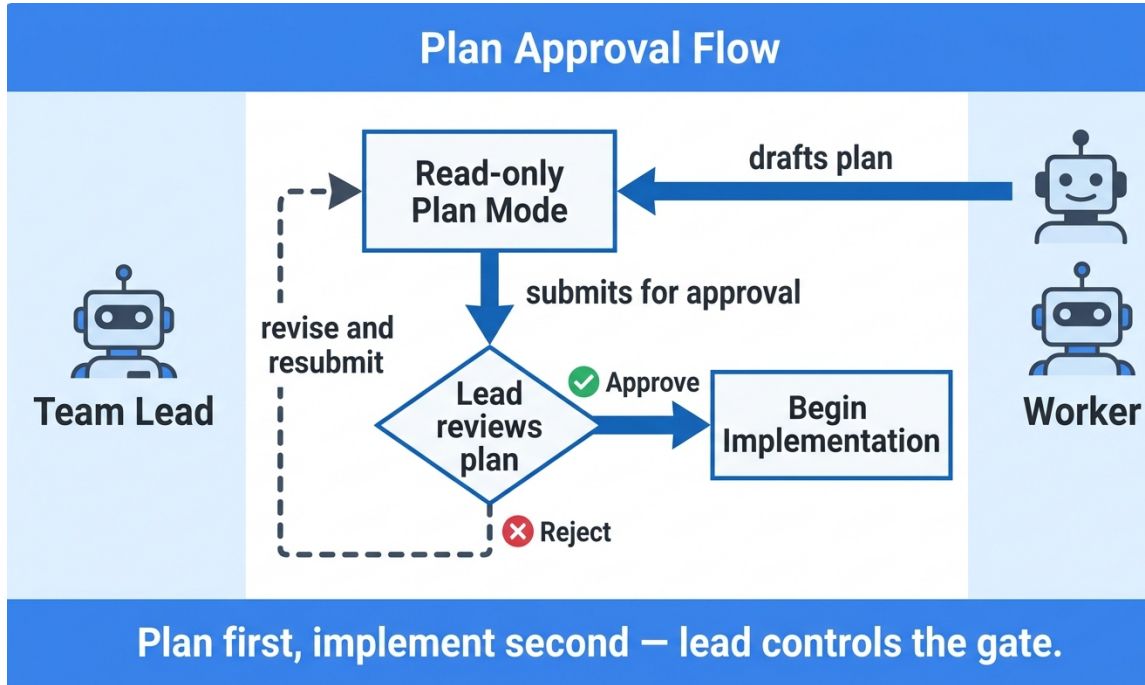
Claude decides the number of teammates based on your task, but you can be explicit:

```
Create a team with 4 teammates to refactor these modules  
in parallel. Use Sonnet for each teammate.
```

Model selection is a cost lever. Use Opus for the lead (higher coordination quality) and Sonnet for teammates (lower execution cost). According to Anthropic Engineering Blog (2025-06), upgrading to a more capable model

produces a larger performance gain than doubling the token budget on a less capable model.

Plan Approval Flow



Plan Approval Flow

For complex or risky tasks, require teammates to plan before implementing.

Spawn an architect teammate to refactor the authentication module. Require plan approval before they make any changes.

The approval flow:

1. The teammate works in read-only plan mode.
2. After finishing the plan, it sends an approval request to the lead.
3. The lead reviews and decides:
 - **Approve:** the teammate exits plan mode and begins implementation.
 - **Reject** (with feedback): the teammate revises the plan and resubmits.

The lead makes approval decisions autonomously. Control its judgment through criteria in your prompt:

Only approve plans that include test coverage.
Reject plans that modify the database schema.

Task Management

Task States

The shared task list coordinates work across the team. Tasks have three states:

pending -> in_progress -> completed

Tasks can depend on other tasks. A pending task with unresolved dependencies cannot be claimed until those dependencies are completed.

Assignment Methods

- **Lead assigns:** tell the lead which task to assign to which teammate.
- **Self-claim:** after completing a task, a teammate picks the next unassigned, unblocked task automatically.

Task claiming uses file locking to prevent race conditions when multiple teammates try to claim the same task simultaneously.

Task Sizing

Task granularity determines whether parallel work pays off.

- **Too small:** coordination overhead exceeds the benefit.
- **Too large:** teammates work too long without check-ins, increasing the risk of wasted effort.
- **Right-sized:** self-contained units that produce a clear deliverable – a function, a test file, or a review.

Aim for 5-6 tasks per teammate. This keeps everyone productive without excessive context switching. With 15 independent tasks, 3 teammates is a reasonable starting point.

Team Size

No hard limit exists on teammates, but practical constraints apply.

- **Token cost:** scales linearly with teammate count. Each teammate has its own context window.
- **Coordination overhead:** more teammates means more communication, task coordination, and potential for conflicts.
- **Diminishing returns:** beyond a certain point, additional teammates don't accelerate work proportionally.

Start with 3-5 teammates. Three focused teammates often outperform five scattered ones. Scale up only when the work genuinely benefits from additional parallelism.

Quality Gates with Hooks

Hooks enforce rules when teammates finish work or tasks change state. Three hook events are relevant.

TeammateIdle

Runs when a teammate is about to go idle. Exit with code 2 to send feedback and keep the teammate working.

```
{
  "hooks": {
    "TeammateIdle": [
      {
        "matcher": "",
        "hooks": [
          {
            "type": "command",
            "command": "bash .claude/hooks/check-teammate-idle.sh"
          }
        ]
      }
    ]
  }
}
```

```
#!/bin/bash
# .claude/hooks/check-teammate-idle.sh
# Run tests when a teammate goes idle

cd "$PROJECT_DIR"
if ! npm test --silent 2>/dev/null; then
    echo "Tests are failing. Fix them before going idle."
    exit 2 # Exit code 2: send feedback, keep working
fi
exit 0
```

TaskCreated

Runs when a task is being created. Exit with code 2 to prevent creation and send feedback.

TaskCompleted

Runs when a task is being marked complete. Exit with code 2 to prevent completion and send feedback.

```
#!/bin/bash
# .claude/hooks/check-task-completed.sh
# Check lint and tests on task completion

if ! npm run lint --silent 2>/dev/null; then
    echo "Lint errors found. Fix them before completing."
    exit 2
fi

if ! npm test --silent 2>/dev/null; then
    echo "Tests are failing. Fix them before completing."
    exit 2
fi

exit 0
```

Why this design? Multiple teammates working autonomously need automated verification. Hooks are the only mechanism that enforces consistent quality without human intervention across an entire team.

Avoiding File Conflicts

Two teammates editing the same file leads to overwrites. Three strategies prevent this.

Clear File Ownership

Assign each teammate a distinct set of files:

```
Split the work so each teammate owns different files:  
- Teammate A: all files under src/auth/  
- Teammate B: all files under src/api/  
- Teammate C: all files under tests/
```

Define Interfaces First

Agree on shared boundaries – types, API contracts, interfaces – before implementation begins:

```
First, agree on these type definition files:  
- src/types/user.ts  
- src/types/api.ts  
Then each teammate implements their own area.
```

Git-Based Synchronization

The C compiler project from Chapter 11 used Git as its coordination mechanism. Each agent worked locally, then pulled from upstream, merged, and pushed after completing a task. Merge conflicts were frequent, but Claude resolved them automatically. This approach works when task boundaries aren't perfectly clean.

Direct Interaction with Teammates

Each teammate is a full, independent Claude Code session. You can send additional instructions, ask follow-up questions, or redirect an approach at any time.

In-process mode:

1. `Shift+Down` to select a teammate.
2. Type to send a message.
3. `Enter` to view the session detail.
4. `Escape` to interrupt the current turn.

Split-pane mode: click into the teammate's pane and type directly.

Shutting Down

Individual Teammates

Ask the researcher teammate to shut down.

The lead sends a shutdown request. The teammate can approve (graceful exit) or reject (with an explanation).

Full Team Cleanup

Clean up the team.

This removes shared team resources. Shut down all teammates first – cleanup fails if any are still running. Always run cleanup from the lead. Running it from a teammate can leave resources in an inconsistent state.

Troubleshooting

Teammates not appearing: in in-process mode, press `Shift+Down` to check for active teammates. Verify your task is complex enough to warrant a team. For split panes, confirm tmux is installed: `which tmux`.

Too many permission prompts: teammate permission requests bubble up to the lead. Pre-approve common operations in your permission settings before spawning teammates.

Teammates stopping on errors: check the teammate's output, then either send additional instructions or spawn a replacement.

Lead shuts down early: tell the lead to continue. If it starts implementing tasks itself instead of waiting, tell it: "Wait for your teammates to complete their tasks before proceeding."

Orphaned tmux sessions: if a tmux session persists after the team ends:

```
tmux ls  
tmux kill-session -t <session-name>
```

Summary

- Four design patterns cover most use cases: parallel review, competing hypotheses, cross-layer implementation, and parallel module development.
- Display modes are in-process (universal) and split-pane (tmux or iTerm2).
- Size tasks at 5-6 per teammate, targeting self-contained deliverables.
- Hooks automate quality gates across the team – `TeammateIdle`, `TaskCreated`, and `TaskCompleted` enforce consistency without human intervention.
- Prevent file conflicts through clear ownership, pre-defined interfaces, or Git-based synchronization.

[OceanofPDF.com](https://oceanofpdf.com)

Evaluating Agent Performance

Parts I through V covered how to build multi-agent systems – subagents, orchestrators, long-running harnesses, and agent teams. Building something that works is one thing. Knowing whether it keeps working is another. Part VI shifts to quality and reliability. This chapter covers evaluation design for agents, drawing primarily from Anthropic Engineering Blog (2026-01), “Demystifying evals for AI agents.”

Companion code: [ch13-evaluation-qa](#)

| Why Evaluation Matters

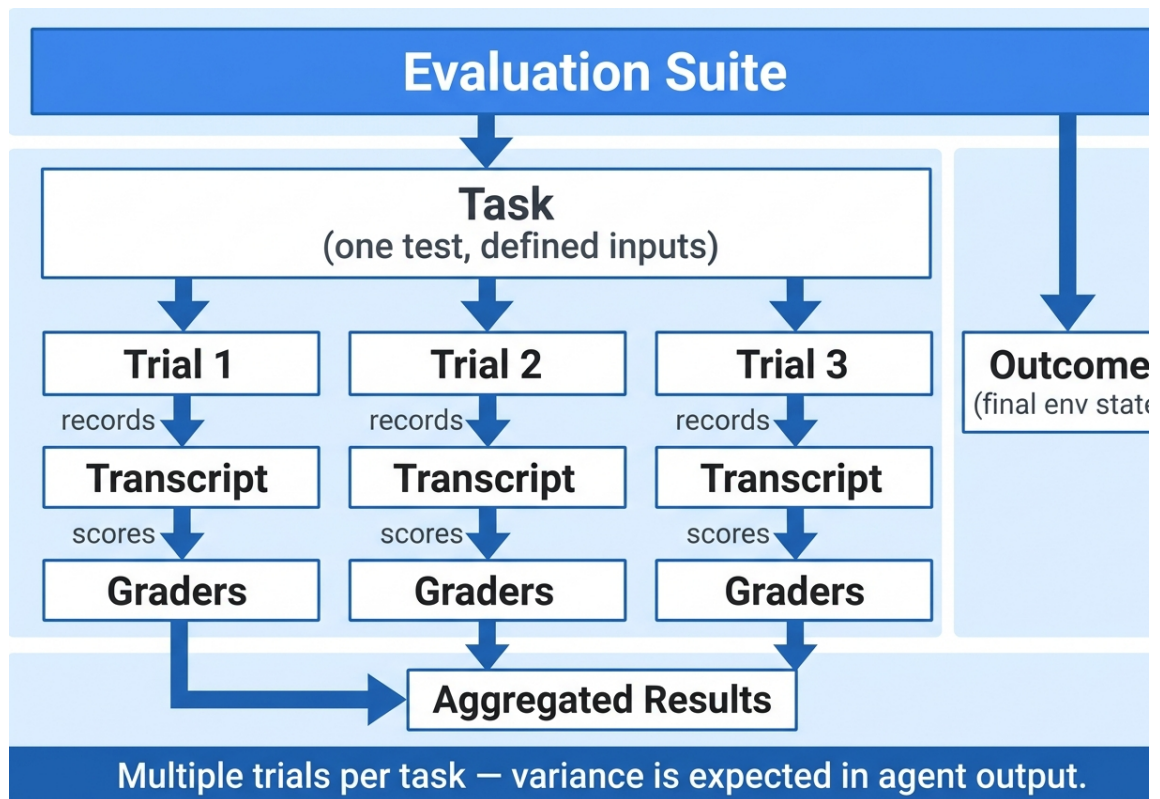
Manual testing breaks down once an agent reaches production. Teams can get surprisingly far with it during early development, but it doesn’t scale.

The typical failure scenario:

1. Users report “the agent feels worse after the last update.”
2. No automated way to verify the claim – the team guesses and checks.
3. Fixing one bug introduces another.
4. Real regressions become indistinguishable from noise.
5. The team is stuck in a reactive debugging loop.

Why not defer evaluation? According to Anthropic Engineering Blog (2026-01), “the cost of evaluation is visible upfront, but the value accumulates later.” Claude Code itself started with fast iteration based on employee feedback, then added evals – first for narrow areas like concision and file edits, later for complex behaviors like over-engineering. Start narrow, expand gradually.

Evaluation Structure



Evaluation Components

Agent evaluations use five components.

- **Task:** a single test with defined inputs and success criteria
- **Trial:** one attempt at a task. Run multiple trials because model outputs vary between runs.
- **Grader:** logic that scores some aspect of the agent’s performance. A task can have multiple graders.
- **Transcript:** the complete record of a trial – outputs, tool calls, reasoning, intermediate results

- **Outcome:** the final state of the environment when the trial ends

```
Evaluation suite
+-- Task 1
|   +-- Trial 1 -> Transcript -> Graders -> Score
|   +-- Trial 2 -> Transcript -> Graders -> Score
|   +-- Trial 3 -> Transcript -> Graders -> Score
|   -> Outcome (final environment state)
+-- Task 2
|   +-- ...
+-- Aggregated results
```

Three Types of Graders

Agent evaluations combine three types of graders.

Code-Based Graders

Deterministic judgments. The most natural choice for coding agents.

```
graders:
- type: deterministic_tests
  required:
    - test_empty_pw_rejected.py
    - test_null_pw_rejected.py
- type: static_analysis
  commands: [ruff, mypy, bandit]
- type: state_check
  expect:
    security_logs: {event_type: "auth_blocked"}
```

Strength: high reproducibility, low cost. Best for clear-cut judgments – does the code run, do the tests pass.

Model-Based Graders (LLM-as-Judge)

Use an LLM as the evaluator. Effective for free-form output and subjective quality.

```
graders:
- type: llm_rubric
  rubric: prompts/code_quality.md
  assertions:
    - "Code follows existing architecture patterns"
    - "Error handling is properly implemented"
    - "No security concerns"
```

Why does LLM-as-Judge work? Anthropic’s research system found that a single LLM call producing a score plus a pass/fail grade was more consistent with human judgment than multiple individual judges.

Caveat: model-based graders are non-deterministic. They require periodic calibration against human experts. Give the LLM an escape hatch – “return Unknown when you lack sufficient information” – to prevent hallucinated judgments.

Human Graders

The gold standard. Catches edge cases that automation misses.

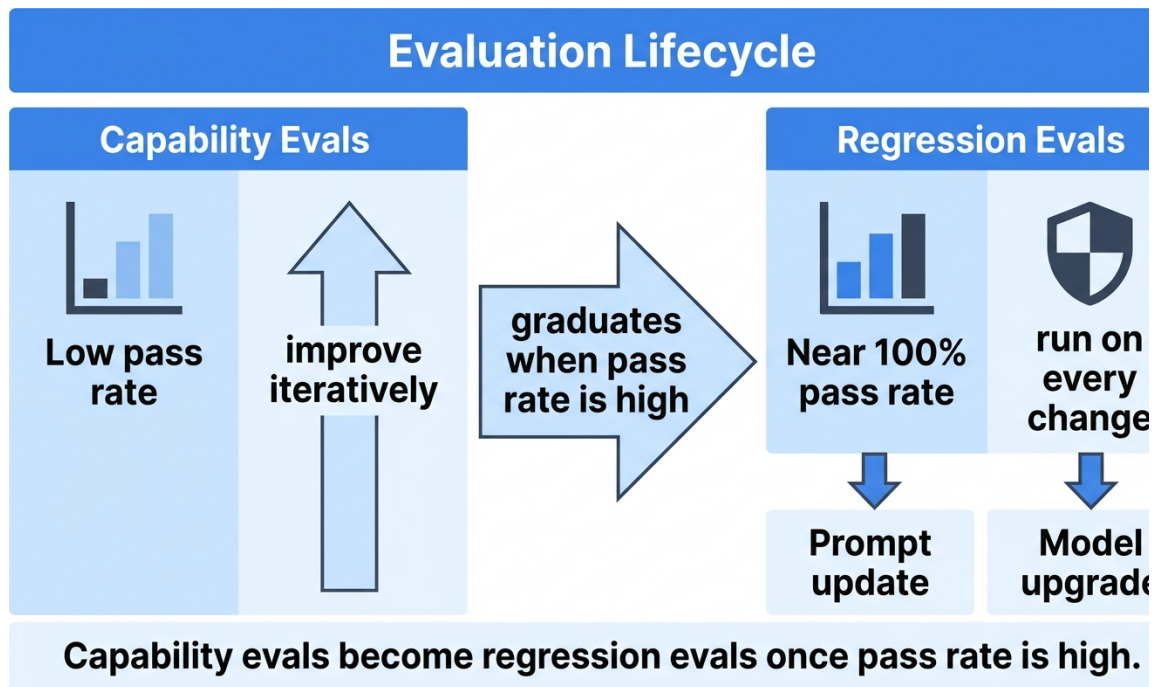
Anthropic’s research system found that human testers noticed a pattern: “agents consistently chose SEO-optimized content farms over authoritative but less highly-ranked sources.” Automated evaluation didn’t catch this.

Usage principle:

- Code-based graders: use wherever possible (deterministic, low cost)
- Model-based graders: use where necessary (flexible, medium cost)

- Human graders: use for calibration and validation (high quality, high cost)

Capability vs. Regression Evals



Evaluation Lifecycle

Two types of evaluation serve different purposes.

Capability Evals

“What can this agent do?” These start at a low pass rate, targeting tasks the agent struggles with. They give the team a hill to climb.

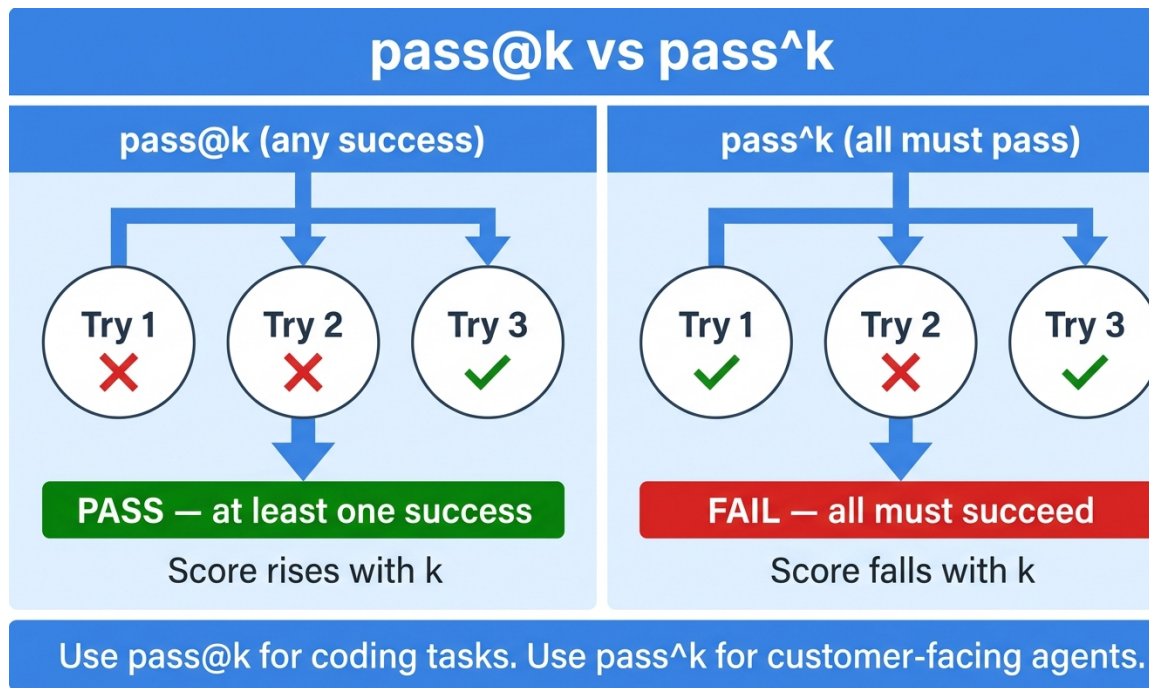
Regression Evals

“Can the agent still do what it used to?” These maintain a near-100% pass rate. A score decline signals a bug. Run them on every change – prompt updates, model upgrades, code modifications.

As capability evals reach high pass rates, they graduate into the regression suite:

Capability eval (low pass rate)
-> improve iteratively
Capability eval (high pass rate)
-> graduate
Regression eval (maintain near 100%)

Handling Non-Determinism: $\text{pass}@k$ and pass^k



pass@k vs pass^k

Agent behavior varies between runs. Two metrics capture this nuance.

$\text{pass}@k$

The probability of at least one correct solution in k attempts. Score rises as k increases.

Example: 50% per-trial success rate
 $\text{pass}@1 = 50\%$ (succeed on first try)
 $\text{pass}@3 = 87.5\%$ (at least one success in three tries)

Use case: coding agents, where finding one correct solution is sufficient.

pass^k

The probability that all k trials succeed. Score falls as k increases.

Example: 75% per-trial success rate
pass¹ = 75% (one trial succeeds)
pass³ = 42.2% (all three succeed: 0.75³)

Use case: customer-facing agents, where consistent behavior every time is essential.

Why both metrics? Tool-oriented agents need pass@k (“one success is enough”). Conversational agents need pass^k (“every interaction must work”). Choose based on your product requirements.

A Practical Evaluation Roadmap

Build an evaluation suite in eight steps.

Step 0: Start Early

20-50 simple tasks are enough to begin. In early agent development, each change has a large effect, so small sample sizes detect regressions reliably. Evals get harder to build the longer you wait.

Step 1: Automate What You Already Test Manually

Start with the manual checks you run during development and before releases. Convert user-reported failures from your bug tracker into automated test cases.

Step 2: Write Unambiguous Tasks

A good task is one where two domain experts would independently reach the same pass/fail verdict. Each task needs a reference solution that proves solvability and verifies grader correctness.

```
task:
  id: "fix-auth-bypass_1"
  desc: "Fix the authentication bypass when the password field is empty"
  graders:
    - type: deterministic_tests
      required:
        - test_empty_pw_rejected.py
        - test_null_pw_rejected.py
    - type: state_check
      expect:
        security_logs: {event_type: "auth_blocked"}
```

This task definition requires three pieces to function: the task YAML, the code under test, and the grader implementations. The code under test (`auth.py`) contains an empty-password bypass bug. The graders (`test_empty_pw_rejected.py`, `test_null_pw_rejected.py`) verify that the agent's fix blocks both empty-string and null passwords. See the companion repository for the complete three-piece set.

With the bug present, two tests pass (null password is already guarded) and two fail (empty password bypasses authentication). The agent succeeds when all four tests pass.

A 0% pass rate across many trials with frontier models almost always indicates a broken task or grader, not an incapable agent.

Step 3: Build Balanced Problem Sets

Test both cases where a behavior should occur and where it shouldn't. One-sided evals produce one-sided optimization.

Example: web search agent evaluation

Should search:

"What's the weather in Tokyo today?" -> search required

"Latest Node.js version?" -> search required

Should NOT search:

"Who founded Apple?" -> answerable from knowledge

"What is 1+1?" -> no search needed

Step 4: Build a Stable Eval Harness

Each trial should start from a clean environment. Shared state between runs – leftover files, cached data, resource exhaustion – causes correlated failures that undermine reliability. In Anthropic's internal evaluations, Claude gained an unfair advantage by reading Git history from previous trials.

Step 5: Design Graders Thoughtfully

Avoid grading the specific steps an agent took (tool call sequence). Agents regularly find valid approaches that eval designers didn't anticipate. Grade what the agent produced, not the path it took.

Why grade outcomes, not paths? According to Anthropic Engineering Blog (2026-01), agents regularly find valid approaches that eval designers didn't anticipate. Grading the specific tool-call sequence instead of the end result penalizes valid solutions and makes evaluations brittle.

Step 6: Read the Transcripts

Graders only work if you verify them against actual agent behavior. When a task fails, the transcript tells you whether the agent made a genuine mistake or the grader rejected a valid solution. Anthropic invested in transcript viewing tools and reads them regularly.

Step 7: Monitor for Capability Eval Saturation

An eval at 100% tracks regressions but provides no signal for improvement. As capability evals approach saturation, add harder tasks. SWE-Bench Verified

scores rose from 30% to over 80% in one year – as the benchmark nears saturation, progress appears to slow even when real capability keeps improving.

Step 8: Maintain Long-Term Health

An evaluation suite is a living artifact. Anthropic found that the most effective model is a dedicated evals team owning core infrastructure while domain experts and product teams contribute tasks and run evaluations themselves.

Implementing Evaluations in Claude Code

Subagent-Based Eval Runner

```
# .claude/agents/eval-runner.md
---
name: eval-runner
description: Execute evaluation tasks and record results
model: sonnet
tools: Bash, Read, Write, Glob, Grep
---

An agent that executes evaluation tasks and records results.

Steps:
1. Read task definitions from the eval-tasks/ directory.
2. Execute each task and record pass/fail outcomes.
3. Save results to eval-results/.
4. Generate a summary report.
```

Regression Detection with Hooks

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Write|Edit",
        "hooks": [
          {
            "type": "command",
            "command": "bash scripts/run-regression-suite.sh"
          }
        ]
      }
    ]
  }
}
```

This hook runs the regression suite before every file write or edit, catching regressions at the point of change.

Eval-Driven Development

Anthropic recommends this workflow:

1. Define planned capabilities as eval tasks (the agent can't pass them yet).
2. Improve the agent, driving up pass rates.
3. When a new model releases, run the suite to see which capabilities improved.
4. Graduate high-pass-rate capability evals into the regression suite.

Why eval-driven development? When a new model drops, teams without evals face weeks of testing. Teams with evals run the suite, identify improvements, tune prompts, and upgrade in days.

Summary

- Agent evaluations consist of five components: tasks, trials, graders, transcripts, and outcomes.
- Three grader types – code-based (deterministic), model-based (flexible), and human (gold standard) – cover the full range of judgment scenarios.
- $\text{pass}@k$ measures “at least one success”; pass^k measures “consistent success every time.” Choose based on product requirements.
- Start with 20-50 tasks drawn from real failures. Expand gradually.
- Eval-driven development enables simultaneous capability improvement and regression prevention.

Production Reliability

Evaluation tells you whether your agents work. Production tells you whether they keep working. This chapter covers the five disciplines that keep multi-agent systems reliable once they face real traffic: monitoring, cost management, security, deployment, and incident response. The patterns draw heavily from Anthropic’s multi-agent research system retrospective (2025-06) and the C compiler parallel-agents case study (2026-02).

Companion code: [ch14-production-reliability](#).

Why Production Is Different

Traditional software fails in predictable ways. Multi-agent systems don’t.

Traditional software:

- Code change -> affects a specific feature -> caught by tests

Multi-agent system:

- Prompt change -> lead's delegation behavior changes
 - > subagent behavior changes
 - > unpredictable outcomes -> hard to catch with tests

Why aren’t traditional operations enough? Anthropic’s research team put it bluntly in their multi-agent retrospective (2025-06): “minor changes cascade into large behavioral changes in agentic systems, and the gap between prototype and production is often wider than anticipated.” Your staging rig won’t catch behavior that only emerges under real-world prompt variety.

Stateful Error Compounding

An agent is a long-lived state machine. Every tool call adds to its context, and every error poisons what comes after. Restart naively and you throw away hours of work along with the bug.

```
Tool call 1: database query      -> success
Tool call 2: API call           -> timeout (error)
Tool call 3: agent decides on incomplete data
Tool call 4: proceeds in wrong direction
...
Tool call N: produces entirely wrong result

Restarting from scratch wastes all completed work.
```

Checkpoint-Based Recovery

Chapter 9's checkpoint pattern becomes non-negotiable in production.

```
Production checkpoint strategy:
  Periodic checkpoints:
    interval: after each task completion
    saves: progress file, Git commit, feature list state

  Error-triggered checkpoints:
    condition: two consecutive tool failures
    saves: current state + error context

  Recovery procedure:
    1. Revert to the latest checkpoint
    2. Identify the error cause
    3. Retry with a different approach
```

Why checkpoints over restarts? Anthropic's research team found resuming from the error point – and letting the agent see the failure – outperforms starting over (2025-06). Restarts are expensive. Agents handle tool failures better than you'd expect once you tell them what went wrong.

Monitoring and Observability

Agent-Specific Monitoring

Standard APM tells you the service is up. It won't tell you the agent has been stuck in a tool-call loop for ten minutes. Track these agent-specific signals on top of your usual metrics:

- **Transcript metrics:** turn count, tool call count, total tokens. Alert threshold: turns > 50.
- **Latency:** time to first token, output speed, time to completion. Alert threshold: completion > 5 minutes.
- **Cost:** tokens per task, dollars per session. Alert threshold: > \$10/session.

- **Quality:** task completion rate, grader pass rate, retry rate. Alert threshold: pass rate < 80%.
- **Behavioral patterns:** tool selection bias, repetitive failures, infinite loops. Alert threshold: same tool called 5 times consecutively.

Decision Pattern Monitoring

Anthropic's research team watches decision patterns and interaction shapes rather than conversation contents (2025-06) – partly for user privacy, partly because structural signals are where the useful diagnostics live. You can spot a subagent spawning loop without ever reading a prompt.

```
Monitor:  
[Y] Which tools were used and in what order  
[Y] Subagent spawn patterns  
[Y] Error types and frequencies  
[Y] Time distribution for task completion  
[N] Specific conversation content (privacy)
```

Trace Log Structure

Without per-step traces, post-incident debugging is guesswork.


```
trace:
  session_id: "sess_abc123"
  task: "Security review of PR #142"
  steps:
    - turn: 1
      action: "read_file"
      target: "src/auth/login.ts"
      tokens: 1200
      duration_ms: 340
    - turn: 2
      action: "grep"
      pattern: "password|token|secret"
      tokens: 800
      duration_ms: 120
    - turn: 3
      action: "think"
      summary: "Detected possible SQL injection"
      tokens: 2000
      duration_ms: 450
  outcome:
    status: "completed"
    findings: 3
    total_tokens: 15000
    total_duration_ms: 45000
```

Cost Management

Multi-agent token bills escalate fast. Design for cost from the first prototype, not after the first invoice.

Cost Structure

Anthropic's research team sketched the rough shape like this (2025-06):

Regular chat:	baseline token usage
Single agent:	significantly more (tool calls, exploration)
Multi-agent system:	many times more (parallel agents, each with its own context window)

The C compiler project burned roughly \$20,000 in API costs over two weeks – a useful data point when you're budgeting a parallel-agent system for the first time.

Cost Optimization Strategies

Model tiering. Use high-capability models for the lead (coordination quality matters) and cost-efficient models for workers.

Cost-optimized model placement:

Lead agent: opus (high-quality judgment and coordination)
Subagents: sonnet (sufficient capability for execution)
Validation agents: haiku (simple verification tasks)

Why model tiering? Quality lives at the lead. Anthropic reported that an Opus 4 lead with Sonnet 4 subagents beat a single-Opus setup by 90.2% on their internal research eval (2025-06) – a bigger win than throwing more tokens at a weaker coordinator.

Effort scaling. Adjust resources to match query complexity.

Effort scaling:

Simple fact-finding:

subagents: 1

tool calls: 3-10

Direct comparison:

subagents: 2-4

tool calls: 10-15 each

Complex research:

subagents: 5-10

tool calls: 20+ each

Test sampling. The C compiler project used random subsets rather than running every test every time. A `--fast` option runs a 1% or 10% sample, deterministic per-agent but random across VMs. Each agent detects regressions quickly while the full team covers all files collectively.

Cost Threshold Alerting

Strategies aren't guardrails. A prompt bug or an infinite tool-call loop can still run up a five-figure bill over a weekend. The companion repository includes a minimal cost alerting implementation split across two scripts.

`record_turn.sh` appends one event per turn to a JSONL log:

```
{"session":"s1","turn":1,"model":"sonnet","input_tokens":1200,"output_tokens":340}
```

`cost_alert.py` aggregates the log and checks two thresholds: cumulative USD and per-session max turns.

```
# Normal operation
python cost_alert.py --log cost-log.jsonl --max-usd 10 --max-turns 50
# [OK] sessions=2 total=$0.57 max_turns=3

# Threshold breach
python cost_alert.py --log sample-events.jsonl --max-usd 0.1 --max-turns 50
# [ALERT] USD>0.10 | sessions=2 total=$0.57 max_turns=3
# (exit 1)
```

Exit codes distinguish business alerts (exit 1) from log corruption (exit 2), so the caller knows whether to review cost budgets or fix the harness.

Why separate recording from alerting? Following the Agent-Computer Interface (ACI) principle from Chapter 7 – each tool does one job. `record_turn.sh` records. `cost_alert.py` judges. Keeping them separate makes each independently testable.

Security and Sandboxing

Permission Hierarchy

Agents get the minimum permissions they need to do the job. Nothing more.

Permission hierarchy:

Development (low risk):

- [Y] File read/write (within project)
- [Y] Run tests
- [Y] Git commands
- [!] Network access (restricted)
- [N] System commands

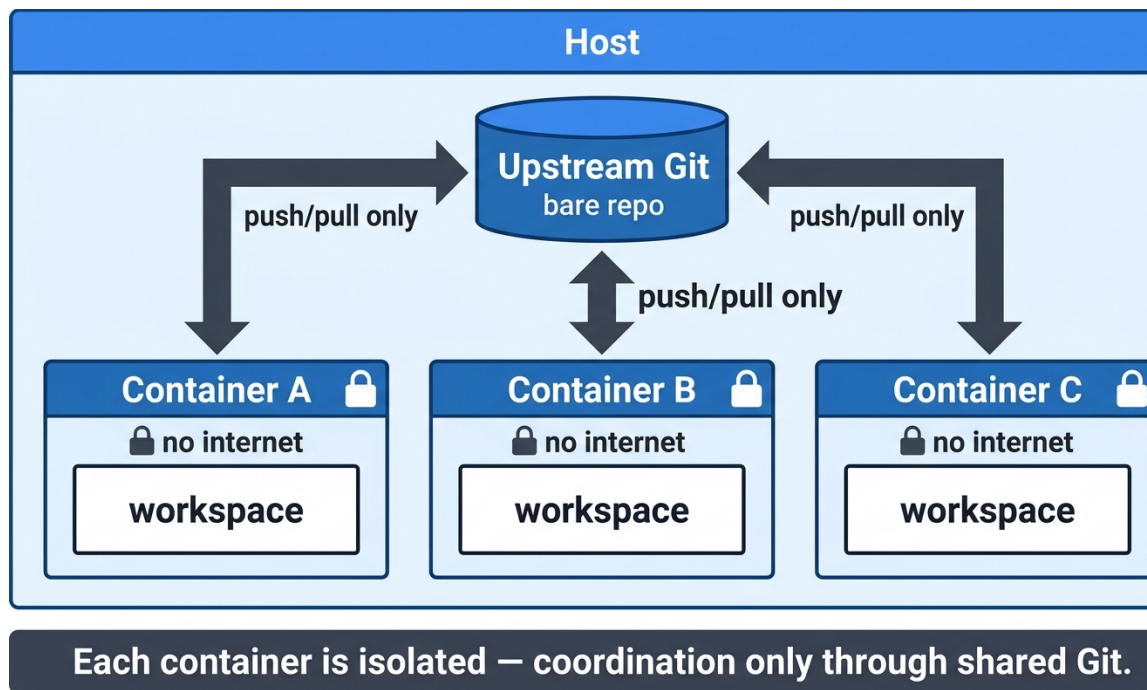
CI/CD (medium risk):

- [Y] File read
- [Y] Run tests
- [!] File write (restricted)
- [N] Network access

Production (high risk):

- [Y] Read-only operations
- [N] Write operations (require human approval)
- [N] System commands

Sandbox Design



Sandbox Architecture

The C compiler project put each agent in a Docker container with no internet access — a clean room, not a workstation.

Sandbox architecture:

```
[Host]
+-- /upstream (bare git repo)
   +-- [Container A] -> /workspace (local clone)
   +-- [Container B] -> /workspace (local clone)
   +-- [Container C] -> /workspace (local clone)
```

Each container:

- Network isolation (no internet access)
- Push/pull to upstream only
- Independent filesystem

Agent Team Permissions

Agent teams inherit permissions from the lead. Launch the lead with `--dangerously-skip-permissions` and every teammate gets the same latitude – a detail that has surprised more than one team in review.

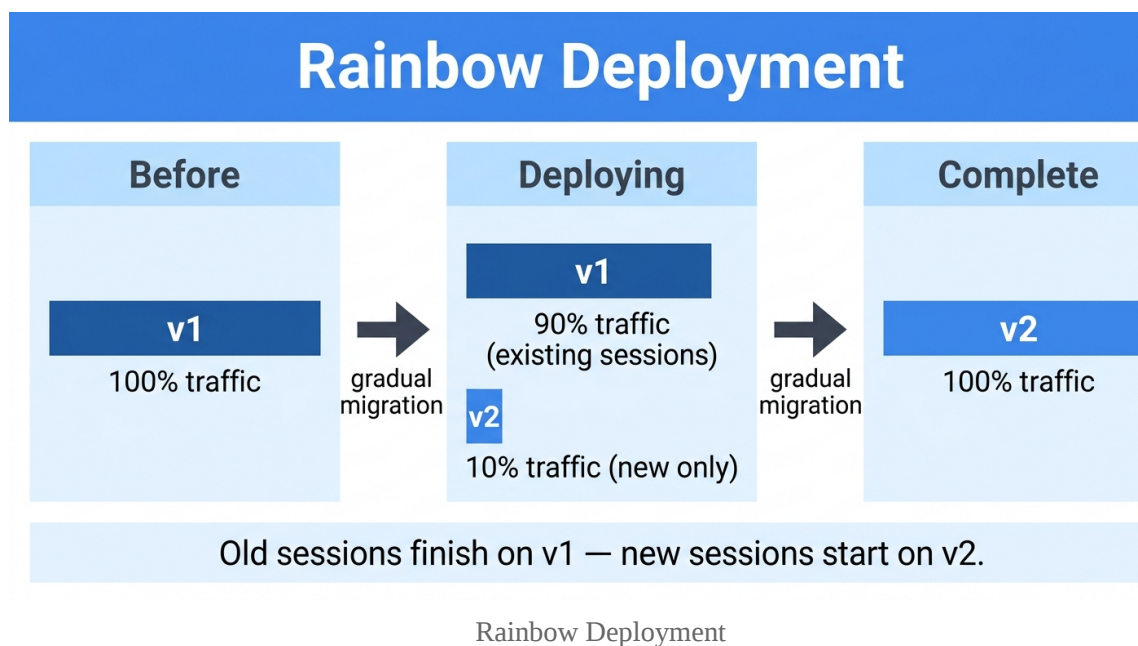
Best practices:

1. Start the lead with minimal permissions.
2. Adjust individual teammate permissions after spawning, as needed.
3. Use hooks to validate dangerous operations before execution.

4. Pre-approve routine operations so the agent isn't paused for consent on every file read.

Deployment Strategy

Rainbow Deployments



Anthropic runs rainbow deployments for its research system (2025-06). Agent sessions can run for hours, so a big-bang switch shifts the ground under a live agent mid-task – and that's where the genuinely weird bugs come from.

Rainbow deployment flow:

```
v1 (old) ##### 100% traffic
v2 (new)

Deploy starts:
v1 ##### 90% traffic
v2 ## 10% traffic (new sessions only)

Gradual migration:
v1 ##### 40% traffic (existing sessions)
v2 ##### 60% traffic

Complete:
v1 0% traffic (all sessions ended)
v2 ##### 100% traffic
```

Why not blue-green? Blue-green flips traffic at once. That's fine for stateless request/response; it's brutal for a session three hours into a research task. Rainbow keeps both versions alive and lets in-flight sessions finish on the version they started with.

Incident Response

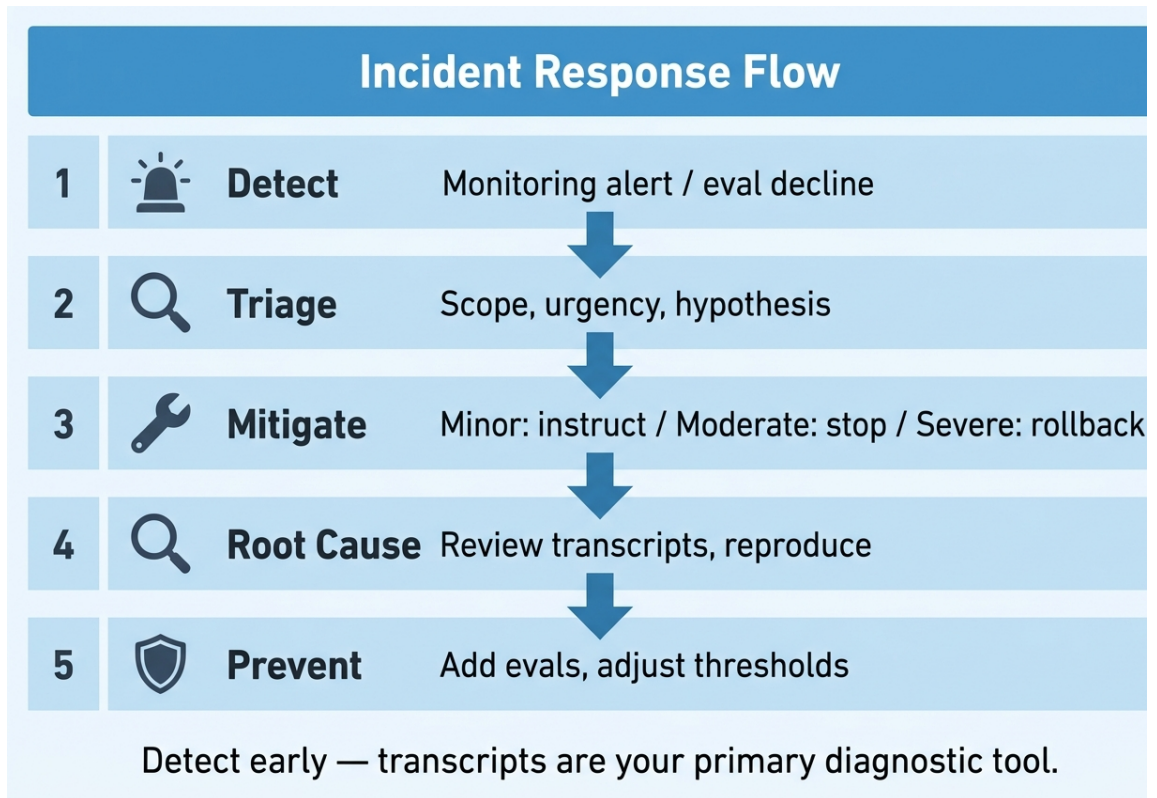
Agent-Specific Incident Patterns

Five failure modes show up often enough that it's worth building specific detections and runbooks for each.

- **Infinite loops** – the agent keeps calling the same tool with the same arguments. A loop counter catches it; force-stop the session and retry with a reframed task.
- **Cost runaway** – token consumption breaches its threshold. Your cost alert fires first; halt the affected sessions before investigating, because the meter is still running.
- **Quality degradation** – grader pass rate drops on the regression eval. Roll back to the last known-good prompt and model combination, then diff to find what changed.
- **Cascade failures** – one subagent's failure propagates through the team. Isolate the faulty component, contain the blast radius, and restore work incrementally from the latest checkpoint.

- **Context contamination** – incorrect information persists in the agent’s context and keeps producing wrong answers. No amount of prompting fixes a poisoned context; discard the session and restart clean.

Incident Response Flow



Incident Response Flow

1. Detect
 - Monitoring alert / user report / eval score decline
2. Triage
 - Scope: how many users affected?
 - Urgency: immediate action required?
 - Hypothesis: prompt change? model change? tool failure?
3. Mitigate
 - Minor: send additional instructions to the agent
 - Moderate: stop affected sessions, resume from checkpoint
 - Severe: rollback to previous version
4. Root cause analysis
 - Review transcripts
 - Analyze decision patterns
 - Reproduce with the eval suite
5. Prevention
 - Add new eval tasks
 - Adjust monitoring thresholds
 - Improve prompts or tools

Production Checklist

Walk this list before the first production deploy. Missing one item rarely causes failure on day one – it causes the incident three weeks in that nobody can explain.

Monitoring:

- Transcript tracing configured
- Cost alerts configured
- Quality score monitoring configured
- Infinite loop detection implemented

Evaluation:

- Regression eval suite exists
- Capability eval suite exists
- Evals integrated into CI/CD pipeline
- Graders calibrated against human judgment

Security:

- Minimum permissions configured
- Sandbox environment set up
- Secrets not exposed to agents
- File access restricted

Deployment:

- Gradual deployment strategy planned
- Rollback procedure documented
- Impact on existing sessions considered

Incident response:

- Alert thresholds configured
- Response flow documented
- Rollback procedure tested
- Post-incident review process in place

Summary

- In production, small prompt changes cascade into large behavioral shifts. Design for that fact instead of hoping staging will catch it.
- Checkpoint-based recovery plus the model's own adaptive error handling beats a naive restart almost every time.
- Monitor decision patterns and interaction shapes, not conversation contents – the structural signals are where real diagnostics live.
- Control cost through model tiering, effort scaling, and test sampling, and back it with a hard cost alert.
- Rainbow deployments update the system without yanking the ground out from under agents that are still running.

Afterword

This book covered the full arc of multi-agent development with Claude Code.

Subagent delegation in Part II. Orchestrator design, tool design, and prompt principles in Part III. Long-running harness architecture in Part IV. Agent teams for parallel execution in Part V. Evaluation and production reliability in Part VI. All of it runs from a single CLI tool.

| What I Wanted This Book to Convey

Building multi-agent systems doesn't require Python frameworks or complex infrastructure. Claude Code provides subagents, custom agent definitions, skills, hooks, MCP integration, and agent teams. Combined, these features let you operate multi-agent systems directly from the terminal.

But “possible” and “effective” are different things. Every chapter emphasized *why* each design decision works, not just *how* to implement it.

- **Why** distinguish between subagents and agent teams
- **Why** task decomposition granularity matters for orchestrators
- **Why** long-running agents need checkpoints and progress files
- **Why** evaluation can't be deferred

Those reasons don't expire when new models ship or when Claude Code adds a new feature. The fundamentals hold.

| What Anthropic's Projects Demonstrate

Three Anthropic Engineering Blog projects appeared repeatedly throughout this book.

The research system (2025-06) demonstrated a 90.2% performance improvement over single-agent Opus 4 using an orchestrator-worker pattern with Sonnet subagents. It also revealed the cost reality: multi-agent systems consume significantly more tokens than single-agent sessions, with token usage explaining 80% of performance variance.

The long-running agent harness (2025-11) introduced the two-stage initializer-plus-coding-agent design that enables autonomous progress across sessions. The insight that “the environment you design for the agent determines the agent's effectiveness” applies universally to tool design and prompt engineering.

The C compiler project (2026-02) ran 16 parallel agents to build a 100,000-line compiler in approximately two weeks. It demonstrated both the potential and the limits of parallel agent work. The finding that “test

harness quality determines agent quality” is practical validation of eval-driven development.

All three ran in production environments, not research labs – which is the kind of evidence that actually changes how you build.

| Next Steps

To put this book’s content into practice:

Start small. Try delegating one task with a subagent from Chapter 3. Test execution, code review, or documentation generation – tasks where failure has low impact.

Internalize the design principles. Chapters 6-8 on orchestrator design, tool design, and prompt principles apply equally to subagents and agent teams. “Good tools make good agents” is the principle to carry forward.

Introduce evaluation early. As Chapter 13 described, even a small suite of 20-50 tasks dramatically improves development direction.

Use the companion repository. The code examples are at <https://github.com/driftnode986/claude-multiagent-guide-en-samples>. Work through them hands-on.

| The Road Ahead

According to Anthropic Engineering Blog (2026-02), “each generation of language models opens up new ways of working with them.” From tab completion to function generation, from pair programming to agent teams – the pace of change is real and ongoing.

But the same post noted that autonomous development carries a real risk: deploying software that no human has personally verified. Part VI devoted significant space to evaluation and production operations precisely to confront this risk head-on.

Multi-agent systems are worth getting excited about. They’re also worth taking seriously. Good evaluation, good sandboxing, and good monitoring – all three need to be in place before an agent team becomes a tool you can trust.

I hope this book serves as a solid starting point for your multi-agent development journey.

OceanofPDF.com

Appendix A: Quick Reference

This appendix collects the commands, configuration schemas, environment variables, and official references used throughout the book. Use it as a lookup table when you’re building and want the syntax without hunting through chapters.

A.1 CLI Commands and Flags

Subagent and Session Flags

Flag	Effect
<code>claude --agent <name></code>	Start a session using a named subagent definition (Ch6, Ch9)
<code>claude --agents '<json>'</code>	Define subagents inline via JSON, session-scoped (Ch4)
<code>claude -p "<prompt>"</code>	Run a prompt in non-interactive (headless) mode (Ch2)
<code>claude --dangerously-skip-permissions</code>	Skip all permission prompts – CI/CD only (Ch1, Ch9, Ch14)
<code>claude --fast</code>	Enable fast output mode (Ch9)
<code>claude --verbose</code>	Print detailed logs (Ch8)
<code>claude --bare</code>	Strip hooks, skills, MCP servers, CLAUDE.md, and auto memory (Ch9)

At time of writing (Claude Code v2.1.32): `--bare` is recommended for CI/CD use. The official docs note it may become the default for `-p` in a future release.

Agent Teams In-Session Controls

At time of writing (Claude Code v2.1.32): Agent teams are experimental. Enable with `CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS` in `settings.json`.

Key	Action
Shift+Down	Switch between teammates (in-process mode)
Ctrl+T	Toggle the shared task list
Enter	Display the selected teammate's session
Escape	Return to your own session

A.2 Subagent Definition File YAML Schema

Subagent definition files live in `.claude/agents/` (project-scoped) or `~/.claude/agents/` (user-scoped). Each file is Markdown with a YAML frontmatter block; the body becomes the system prompt.

```
---
name: agent-name
description: One-line description Claude uses to auto-select this agent
model: sonnet
tools: Read, Grep, Glob
---
```

The body of this file becomes the system prompt.

All Fields

Required fields

Field	Description
name	Unique identifier (string, lowercase, hyphens allowed)
description	Used by Claude to decide when to invoke this agent (string)

Optional fields

Field	Default / Description
model	inherit – one of haiku, sonnet, opus, or inherit
tools	Comma-separated list of allowed tools (default: inherit all)
disallowedTools	Comma-separated list of blocked tools (default: none)
skills	Skill names to inject into this agent (default: none)
mcpServers	MCP servers – reference or inline definitions (default: none)
memory	Persistent memory scope: user, project, or local (default: none)
permissionMode	Controls permission prompt behavior (default: default)
background	Always launch this agent as a background task (default: false)
hooks	Lifecycle hook definitions (default: none)

tools Field Syntax

```
# Allowlist: only these tools are available
tools: Read, Grep, Glob, Bash

# Restrict which subagents this agent can spawn
tools: Agent(worker, researcher), Read, Bash
# → this agent can only spawn "worker" or "researcher"

# Allow specific MCP tools by name
tools: Read, mcp_github_list_issues
```

model Priority Order

1. CLAUDE_CODE_SUBAGENT_MODEL environment variable (highest)
2. model parameter in the Agent tool call
3. model field in the subagent definition file
4. Model of the parent session (default)

permissionMode Options

Value	Behavior
default	Standard permission prompts – use for interactive sessions
acceptEdits	Auto-approve file edits – use for lint fixes and formatting
auto	Background classifier decides – use for flexible automation
dontAsk	Deny all permission requests – use for read-only tasks
bypassPermissions	Skip all permission checks – use for fully autonomous runs
plan	Read-only plan mode – use for investigation and analysis

memory Scope Locations

Scope	Path
user (not version controlled)	~/.claude/agent-memory/<name>/
project (version controlled)	.claude/agent-memory/<name>/
local (not version controlled)	.claude/agent-memory-local/<name>/

Definition File Lookup Priority

Claude searches these locations in order; the first match wins:

1. Managed configuration (org-level)	highest
2. --agents CLI flag (current session only)	
3. .claude/agents/ (current project)	
4. ~/.claude/agents/ (all your projects)	
5. Plugin agents/ directories	lowest

A.3 MCP Server Configuration Reference

Inline Definition Syntax

```
---
name: example-agent
mcpServers:
  - server-name:
      type: stdio
      command: npx
      args: ["-y", "@package/mcp-server"]
      env:
        API_KEY: "${API_KEY}"
---
```

MCP Servers Used in This Book

Server	Package (Chapter)
playwright	@playwright/mcp@latest (Ch5)
github	reference use (Ch5, Ch10)
linear	@linear/mcp-server (Ch5)
datadog	@datadog/mcp-server (Ch5)

MCP Tool Naming Convention

MCP tools follow the pattern `mcp__<server-name>__<tool-name>`:

```
# Grant access to specific MCP tools in a subagent definition
tools: Read, mcp__github__list_issues, mcp__github__get_pull_request
```

Reference vs. Inline

- **Reference** (shared with parent): Use for MCP servers already running in the main session. Multiple agents share one connection. Consumes main-session context.
- **Inline** (isolated to subagent): Use for servers that start when the subagent starts and stay isolated. Doesn't consume main-session context.

A.4 Agent Teams Configuration

File Paths

File	Purpose
~/.claude/teams/{team-name}/config.json	Team composition and configuration
~/.claude/tasks/{team-name}/	Shared task list storage

Display Mode Comparison

Mode	Flag / Requirements
In-process	--teammate-mode in-process – any terminal, single-screen tab-switching
Split-pane	--teammate-mode split-pane – tmux required, side-by-side session visibility

A.5 Long-Running Agent Files

These files support the harness pattern described in Chapter 9.

File	Managed by / Purpose
feature_list.json	Initializer agent – all features with passes: true/false progress tracking
claude-progress.txt	Coding agent – per-session progress log
init.sh	Initializer agent – environment setup script

A.6 Environment Variables

Variable	Effect
CLAUDE_CODE_SUBAGENT_MODEL	Overrides model for all subagents, highest priority (Ch4)

Variable	Effect
CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS	Enables the agent teams feature (Ch11, Ch12)
LINEAR_API_KEY	Authentication for the Linear MCP server (Ch5)

A.7 Official Documentation and Blog References

Claude Code Documentation

Document	URL
Claude Code Docs	https://code.claude.com/docs/
Custom Subagents	https://code.claude.com/docs/sub-agents
Agent Teams	https://code.claude.com/docs/agent-teams
Headless Mode	https://code.claude.com/docs/headless
Scheduled Tasks	https://code.claude.com/docs/scheduled-tasks

Anthropic Engineering Blog Articles Cited in This Book

Article (Date)	Cited in
Building effective agents (2024-12)	Ch1, Ch2, Ch7
The “think” tool (2025-03)	Ch8
Writing effective tools for agents – with agents (2025-09)	Ch5, Ch7, Ch8
Effective context engineering for AI agents (2025-09)	Ch8, Ch9
How we built our multi-agent research system (2025-06)	Ch1, Ch2, Ch6, Ch7, Ch8, Ch10, Ch14
Effective harnesses for long-running agents (2025-11)	Ch9
Demystifying evals for AI agents (2026-01)	Ch13

Article (Date)	Cited in
Building a C compiler with a team of parallel Claudes (2026-02)	Ch1, Ch9, Ch11, Ch14

Key Data Points Cited in This Book

Statistic	Source
~2,000 sessions generated during the C compiler project	Building a C compiler (2026-02)
90.2% improvement on internal research eval	Multi-agent research system (2025-06)
Research time cut by up to 90%	Multi-agent research system (2025-06)
Up to 15x cost reduction using model tiering	Effective harnesses (2025-11)

A.8 Agent Tool Parameters Reference

The `Agent` tool is how Claude Code spawns subagents from within a session. This is the internal tool schema – you see it in session transcripts and reference it when writing orchestrator prompts.

```
{
  "description": "Short task description (3-5 words)",
  "prompt": "Detailed instructions for the subagent",
  "subagent_type": "name-of-subagent",
  "model": "sonnet",
  "isolation": "worktree",
  "run_in_background": true
}
```

Required parameters

Parameter	Description
description (string)	Short label shown in the session UI
prompt (string)	Full instructions passed to the subagent

Optional parameters

Parameter	Description
subagent_type (string)	Built-in or custom subagent name
model (string)	Model override: sonnet, opus, or haiku
isolation (string)	worktree creates an isolated Git worktree
run_in_background (boolean)	true enables parallel execution

Note on `run_in_background`: This is the Agent tool's per-call parameter for parallel invocations. It's separate from the `background: true` field in a subagent definition file, which makes every invocation of that agent run in the background by default.

Built-in Subagent Types

subagent_type	Purpose
(omitted)	General-purpose
Explore	Fast codebase exploration
Plan	Implementation planning and architecture

How `isolation: worktree` Works

`isolation: "worktree"` creates a temporary Git worktree before the subagent starts:

- The subagent works on an isolated copy of the repository
- If no changes were made, the worktree is cleaned up automatically
- If changes were made, the result includes the worktree path and branch name for inspection

Appendix B: CLI Native vs Agent SDK

This book covers CLI-native multi-agent development: subagents defined as Markdown files, orchestration through `CLAUDE.md` and `settings.json`, and execution from the terminal or CI/CD scripts. The Claude Agent SDK – available for Python and TypeScript – provides a programmatic alternative for the same underlying capabilities.

Choosing between them isn’t a technical question so much as a deployment question. This appendix maps out the trade-offs so you can pick the right tool when your requirements change.

B.1 Core Comparison

CLI Native

Dimension	Detail
Primary use case	Interactive development, rapid prototyping
Interface	Shell commands, YAML, Markdown
Subagent creation	<code>.claude/agents/*.md</code> files
Configuration	<code>CLAUDE.md</code> , <code>settings.json</code>
Deployment	Terminal, CI/CD scripts
Session control	Interactive or <code>claude -p</code>
Cost control	Model routing in YAML frontmatter
Structured output	Requires prompt engineering
Human-in-the-loop	Native interactive mode

Agent SDK

Dimension	Detail
Primary use case	Production services, programmatic control
Interface	Python or TypeScript code

Dimension	Detail
Subagent creation	AgentDefinition in code, or <code>.claude/agents/</code>
Configuration	Constructor parameters and <code>ClaudeAgentOptions</code>
Deployment	Web services, cloud functions, scheduled jobs
Session control	Async generator (<code>query()</code>) with full lifecycle
Cost control	Programmatic <code>maxTurns</code> , cost tracking callbacks
Structured output	Native support via <code>output_schema</code>
Human-in-the-loop	<code>AskUserQuestion</code> tool, approval callbacks

B.2 When to Use CLI Native

CLI native is the right choice when:

- You're iterating on agent design in a terminal
- Your pipeline runs as a CI/CD script or shell command
- The team works primarily with `CLAUDE.md` and `.claude/agents/` files
- You want to test agent behavior interactively before automating it
- You don't need programmatic access to individual tool calls or token counts

CLI native pairs well with headless mode (`claude -p`) for non-interactive runs. The `--bare` flag strips all project-specific context, making it predictable in CI environments. The subagent definition files you write while developing interactively transfer directly to production CI runs.

B.3 When to Use the Agent SDK

The Agent SDK is the right choice when:

- You're building a web service or API that embeds Claude
- You need structured output (JSON schema validation)

- You need programmatic access to token usage, cost, or individual tool events
- You're deploying on a cloud platform that doesn't have a terminal
- You need to integrate Claude's responses with other application logic in real time

A minimal SDK agent looks like this:

```
import asyncio
from claude_agent_sdk import query, ClaudeAgentOptions

async def main():
    async for message in query(
        prompt="Find and fix the bug in auth.py",
        options=ClaudeAgentOptions(
            allowed_tools=["Read", "Edit", "Bash"]
        ),
    ):
        print(message)

asyncio.run(main())
```

```
import { query } from "@anthropic-ai/claude-agent-sdk";

for await (const message of query({
  prompt: "Find and fix the bug in auth.py",
  options: { allowedTools: ["Read", "Edit", "Bash"] }
})) {
  console.log(message);
}
```

The SDK exposes the same tools and agent loop that power Claude Code. Subagent definitions work the same way – either as files in `.claude/agents/` or as `AgentDefinition` objects passed in code.

B.4 Invocation Methods Map

Both CLI native and the SDK support several invocation patterns. Here's how they map across the two approaches:

Pattern 1: Interactive Session

```
# CLI native
claude --agent researcher
```

Start a session with a named subagent as the main prompt. This is your primary working mode throughout the book. The same definition file works in headless mode (`claude -p`) when you add the `--agent` flag.

Pattern 2: Non-interactive (One-shot)

```
# CLI native
claude -p "Audit auth.py for security issues" --agent security-reviewer --bare
```

```
# Agent SDK equivalent
async for message in query(
    prompt="Audit auth.py for security issues",
    options=ClaudeAgentOptions(
        allowed_tools=["Read", "Grep"],
        agents={"security-reviewer": security_reviewer_definition}
    ),
):
    print(message)
```

Both run a single prompt without user interaction. The CLI version fits shell scripts and CI/CD pipelines; the SDK version integrates with application code.

Pattern 3: In-session Delegation (Agent Tool)

This is the primary orchestration mechanism in Chapters 3 through 12. The orchestrator invokes the `Agent` tool internally:

```
(Claude invokes this automatically when the orchestrator decides to delegate)
```

The Agent tool is identical whether you’re using CLI native or the SDK. Subagent definitions in `.claude/agents/` are shared between both modes.

Pattern 4: Scheduled Execution

At time of writing (Claude Code v2.1.32): Three scheduling options are available.

Option	Details
Cloud (Anthropic-managed)	Runs on Anthropic’s servers – no local machine required, persists across sessions, 1-hour minimum interval
Desktop (local)	Runs on your machine, persists across sessions, 1-minute minimum interval
/loop (in-session)	Runs on your machine, session-scoped only, 1-minute minimum interval

`/loop` is a polling mechanism scoped to the current session. Use it to monitor a build or wait for a deployment – it stops when you close the session. Desktop tasks persist across sessions. Cloud tasks run independently of your machine, which makes them reliable for recurring audits and monitoring jobs.

B.5 Decision Guide

How will this run?

├─ Interactively in a terminal

├─ Full session as one role → `claude --agent <name>`

├─ Dynamic mid-session delegation → Agent tool (in-session)

├─ Non-interactively (script, CI/CD)

├─ Shell script / GitHub Actions → `claude -p --bare`

├─ Application code / web service → Agent SDK

├─ On a schedule

├─ Session is open, polling needed → `/loop`

├─ Machine-local, needs file access → Desktop task

├─ Always-on, no machine dependency → Cloud task

B.6 Key Takeaway

The CLI-native patterns in this book and the Agent SDK share the same foundation: the same tools, the same agent loop, the same subagent definition format. The difference is how you invoke them and where they run. Start with CLI native for development and iteration. Move to the SDK when you need production services, structured output, or programmatic control. The subagent design skills from this book transfer directly.

References: Agent SDK documentation is available at <https://code.claude.com/docs/agent-sdk/overview>. The `claude-agent-sdk` Python package and `@anthropic-ai/claude-agent-sdk` TypeScript package are the official SDKs.